

Data Science Team Training

Second edition

Stephen D. Turner, Ph.D.

Contents

Preface	13
--------------------------	-----------

I

Technical

1	Version Control	17
1.1	GitHub Setup	17
1.1.1	Creating a GitHub Account	17
1.1.2	Adding an SSH Key	17
1.1.3	Cloning a Repository via SSH	17
1.1.4	Pulling and Pushing	17
1.2	Branching and Merging	18
1.2.1	Internal Organizational Workflow	18
1.2.2	Contributing to External Repositories	19
1.3	Git in Positron	20
1.4	Merge Conflicts	20
1.4.1	What conflict markers look like	21
1.4.2	Resolving conflicts	21
1.5	.gitignore	22
1.5.1	Why it matters for data science	22
1.5.2	Syntax	22
1.5.3	A starting point for R and Python projects	22
2	Data Organization	25
2.1	Data Organization in Spreadsheets	25
2.1.1	Be Consistent	25
2.1.2	Make It a Rectangle	25
2.1.3	One Thing Per Cell	25
2.1.4	Fill in All Cells	25
2.1.5	Write Dates as YYYY-MM-DD	25
2.1.6	Choose Good Names for Things	26
2.1.7	No Calculations in Raw Data Files	26
2.1.8	Don't Use Formatting as Data	26
2.1.9	Create a Data Dictionary	26
2.1.10	Save Data in Plain Text	26
2.1.11	Make Backups	26
2.2	File Naming	27
2.2.1	Three Principles	27
2.2.2	Recommended Pattern: YYYY-MM-DD-slug	27
2.3	Project Directory Structure	27
2.3.1	Simple project-oriented workflow	27
2.3.2	Version controlling code with data on a shared drive	28
2.4	File Formats	29
2.5	Additional Best Practices	29

3	Data Validation and QA	31
3.1	Failing Fast	31
3.2	Structured Validation with pointblank	32
3.2.1	Basic Workflow	32
3.2.2	Adding More Checks	32
3.2.3	Common Checks for Public Health Data	33
3.2.4	Stopping on Failure	33
3.3	Where to Put Validation	34
4	Testing Analysis Code	35
4.1	Start with Functions	35
4.2	First Tests with testthat	35
4.3	Choosing Expectations	36
4.4	Edge Cases	37
4.5	Regression Tests	38
4.6	Organizing Tests in a Project	40
4.7	Tests and Validation Are Different Checks	41
4.8	Further Reading	41
5	Debugging and Getting Help	43
5.1	Reading the Error Message	43
5.2	Locating the Failure with traceback()	44
5.3	Pausing Inside a Function with browser()	45
5.4	When the Code Runs but the Answer Is Wrong	46
5.5	Shrinking the Problem: Minimal Reproducible Examples	47
5.6	Asking for Help	47
5.7	Further Reading	48
6	Reproducible Reporting	49
6.1	Python and Other Languages	49
6.2	Document Anatomy	49
6.3	One Input, Many Output Formats	50
6.4	Code Is the Report	51
6.5	Paths and Project Structure	52
6.6	Parameterized Reports	52
6.6.1	Declaring Parameters	52
6.6.2	Using Parameters	53
6.6.3	Rendering with Different Parameters	53
6.7	Consistent Branding with brand.yml	54
7	Accessibility	55
7.1	Accessibility Requirements	55
7.2	Alt Text for Figures	56
7.3	Color and Contrast	57
7.4	Accessible Tables	58

7.5	HTML Accessibility Checks	58
7.6	PDF Accessibility	59
7.6.1	Enabling PDF Standards	59
7.6.2	Validating with veraPDF	60
7.6.3	Known Limitations	60
7.7	Authoring Workflow	60
8	Coding with AI	61
8.1	Code Completion with GitHub Copilot	61
8.2	Posit Assistant	61
8.2.1	Older Positron Assistant and Databot instructions	61
8.3	Claude Code	61
8.4	Cost, Privacy, and Model Choice	62
8.4.1	Capability	62
8.4.2	Cost	62
8.4.3	Privacy	62
9	Dashboards	63
9.1	Dashboard Anatomy	63
9.1.1	YAML Front Matter	63
9.1.2	Rows and Columns	64
9.1.3	Cards	64
9.1.4	Value Boxes	64
9.1.5	Tabsets	64
9.2	Building a Dashboard	65
9.2.1	Project Structure	65
9.2.2	Loading Data	65
9.2.3	Value Boxes	65
9.2.4	Static plots (ggplot2)	66
9.2.5	Interactive plots (plotly)	67
9.2.6	Tables (DT)	67
9.3	Previewing and Rendering	68
9.4	Hosting on GitHub Pages	68
9.4.1	Repository Setup	68
9.4.2	Publishing	68
10	BI Platforms	69
10.1	Dividing the Work	69
10.2	Pre-Aggregating Before Import	69
10.2.1	Default Aggregation	71
10.3	Publishing Programmatically	71
10.3.1	Obtaining Permissions	72
10.4	Power BI and SharePoint	72
10.5	Version Control for BI Artifacts	72
10.6	Concurrent Editing	73
10.7	Metric Definitions	73
10.8	Checks Before Publishing	74

11	Automation and Scheduling	75
11.1	What to Automate (and What Not To)	75
11.2	Scheduling on Your Own Machine	75
11.3	Continuous Automation with GitHub Actions	76
11.4	Multi-Step Pipelines with targets	77
11.5	Running in a Reproducible Environment	78
11.6	Knowing When It Breaks	78
11.7	Sensitive Data and Automation	79
11.8	Further Reading	79
12	Package Loading	81
12.1	How R's search path works	81
12.2	Load only what you need	82
12.3	Use <code>::</code> for one-off functions	83
12.4	The <code>conflicted</code> package	83
12.5	Team implications	85
13	Managing Package Dependencies	87
13.1	The Cost of a Dependency	87
13.2	Measuring Dependency Weight	87
13.3	Meta-Packages	88
13.4	Vetting a Package	88
13.5	Auditing an Existing Project	89
13.6	Lifecycle Stages	89
13.7	Version Consistency Across a Team	89
13.8	Writing Your Own Instead	89
13.9	Organizing Your Own Packages	90
13.10	Further Reading	90
14	Package Development	91
14.1	Why build a package?	91
14.2	The package development workflow	92
14.3	Resources	92
15	Reproducible Environments	95
15.1	Package Environments with <code>renv</code>	95
15.1.1	How <code>renv</code> Works	95
15.1.2	Core Workflow	96
15.1.3	Working as a Team	96
15.1.4	What <code>renv</code> Does Not Solve	97
15.2	Containers with <code>Docker</code>	97
15.2.1	Key Concepts	97
15.2.2	Why <code>Docker</code> for R?	97
15.2.3	The <code>Rocker</code> Project	97
15.3	<code>pracpac</code> : R Packaging with <code>Docker</code>	99

15.3.1	What pracpac Does	99
15.3.2	Installation	99
15.3.3	Workflow	99
15.3.4	Use Cases	100
15.4	Choosing Your Approach	101
15.5	Further Reading	101
16	Databases from R	103
16.1	DBI: The Common Interface	103
16.2	DuckDB	104
16.2.1	Setting up	104
16.2.2	Loading data	104
16.2.3	Querying with SQL	105
16.3	dbplyr: dplyr Syntax on Any Database	105
16.3.1	Lazy tables	105
16.3.2	Writing queries with dplyr	106
16.3.3	Inspecting generated SQL	107
16.4	duckplyr: DuckDB as a dplyr Backend	107
16.4.1	How duckplyr differs from dbplyr	108
16.5	Connecting to Institutional Databases	108
16.5.1	SQL Server	108
16.5.2	PostgreSQL	109
16.5.3	Storing credentials safely	109
16.5.4	Reusing Query Patterns	109
16.6	Practical Guidance	109
17	Choosing and Paying for Data Infrastructure	111
17.1	Identifying the Problem	111
17.2	Sizing the Requirement	111
17.3	Row Stores and Column Stores	112
17.4	Managed Platforms	112
17.5	Consumption Pricing	112
17.6	Build Versus Buy	113
17.7	Funding the Cost	113
17.8	Protected Data on a Managed Platform	114
17.9	Migrating From Flat Files	114
17.10	Practical Guidance	115
18	APIs and Public Data Sources	117
18.1	What an API Request Looks Like	117
18.2	Where Public Health Data Lives	117
18.3	Use a Package When One Exists	118
18.4	Building Requests with http2	118
18.4.1	Pagination	119
18.4.2	Retries and rate limits	119
18.5	API Keys and Tokens	119
18.6	Separate the Pull from the Analysis	119

18.7	Validate What Comes Back	119
18.8	Download and Reuse	120
19	Migrating from Legacy Workflows	123
19.1	Why Migrate, and Why Not	123
19.2	Inventory and Triage	123
19.3	Reading Legacy Data Formats	124
19.4	Translating SAS Code	125
19.4.1	Check merge and missing-value semantics	125
19.4.2	AI-assisted translation	126
19.5	Verifying Parity	126
19.6	Running in Parallel and Cutting Over	127
19.7	Excel and Access Workflows	127
19.8	The People Side	128
19.9	Further Reading	128
20	Code Style	129
20.1	The Tidyverse Style Guide	129
20.2	Formatting with Air	129
20.2.1	Setting Up Format on Save in Positron	129
20.2.2	Before and After	130
20.3	Linting with lintr	131
20.3.1	Getting Started	131
20.3.2	Examples	131
21	Documentation	133
21.1	READMEs	133
21.1.1	A Minimal README Template	133
21.1.2	README for Shared Utility Code	133
21.2	Inline Comments	134
21.2.1	Sectioning Longer Scripts	134
21.3	Documenting Functions with roxygen2	134
21.3.1	When Functions Are More Than Utilities	135
21.4	Project-Level Documentation	135
21.4.1	Data Dictionaries	135
21.4.2	Decision Logs	135
21.4.3	Methods Documentation	136
21.5	Documentation as a Team Practice	136

II

Nontechnical

22	Building and Sustaining a Data Science Team	139
22.1	Roles on a Public Health Data Science Team	139
22.2	Hiring and Position Descriptions	139
22.3	Growing Skills on the Team You Have	140
22.4	Onboarding and Knowledge Continuity	140

22.5	Where the Team Sits	140
22.6	Retention and Sustainability	141
22.7	Further Reading	141
23	Onboarding and the Team Handbook	143
23.1	Why Keep a Handbook	143
23.2	Contents of the Handbook	143
23.3	Testing Setup Documentation	143
23.4	Conventions	144
23.5	Update Protocols	144
23.6	Practice Repositories and Training Data	144
23.7	Cross-Training	145
23.8	Handoff and Offboarding	145
23.9	Adapting an Existing Handbook	145
23.10	Maintaining the Handbook	145
23.11	The IT Liaison	146
23.12	Further Reading	146
24	Project Management	147
24.1	Defining the Project	147
24.1.1	Start with the question	147
24.1.2	Project charters	147
24.2	Managing Scope	147
24.3	Collaborating as a Team	148
24.4	Working with Your IT Team	148
24.4.1	Start early	148
24.4.2	Be specific about what you need	148
24.4.3	Secure data environments	149
24.5	Communicating Findings	149
24.5.1	Know your audience	149
24.5.2	Communicate uncertainty honestly	149
24.5.3	Match the output format to the need	149
24.6	Further Reading	149
25	Working with IT and Data Governance	151
25.1	Data Environments in Public Health	151
25.2	Data Use Agreements	151
25.2.1	What DUAs Cover	151
25.2.2	Practical Guidance	152
25.3	HIPAA and Protected Health Information	152
25.3.1	What HIPAA Covers	152
25.3.2	The Minimum Necessary Standard	153
25.3.3	De-identification	153
25.4	Getting Data Access	153
25.4.1	Framing Requests	153
25.5	Working in Secure Data Environments	154
25.5.1	Common Constraints	154

25.5.2	Adapting Your Workflow	154
25.6	Disclosure Review and Small Number Suppression	154
25.6.1	When Disclosure Review Applies	154
25.6.2	Small Number Suppression	155
25.6.3	What Disclosure Review Examines	155
26	AI Governance in a Health Agency	157
26.1	The Need for a Written Policy	157
26.2	Contents of an Agency AI Policy	157
26.3	Formal Review Processes	158
26.4	Protected Data and AI Tools	158
26.5	Cloud and Self-Hosted Inference	159
26.6	Transparency in Modeling Tools	159
26.7	Generated Code	159
26.8	Reproducibility	160
26.9	Staff Capability	160
26.10	Further Reading	160
27	Peer Review for Data Analysis	163
27.1	Scope of Analytical Peer Review	163
27.2	Code Review Mechanics	163
27.2.1	What to Look for in a Code Review	163
27.2.2	Giving and Receiving Feedback	164
27.3	Checklists for Analytical Review	164
27.4	AI as a Review Aid	165
27.5	High-Stakes Analyses	165
27.6	Building a Review Culture	165
28	Communicating Findings	167
28.1	Lead with the Finding	167
28.1.1	The “So What” Test	167
28.2	Audience and Format	168
28.2.1	Decision-Makers	168
28.2.2	Program Staff	168
28.2.3	Technical Peers	168
28.2.4	An Executive Summary Template	168
28.3	Choosing the Right Output	169
28.4	Visualizing to Communicate	169
28.4.1	Match Chart Type to Message	169
28.4.2	Make the Finding Clear	170
28.5	Communicating Uncertainty	170
28.5.1	Natural Language Over Statistical Formalism	170
28.5.2	Null Results Are Findings	170
28.5.3	Preliminary and Provisional Data	171
28.5.4	Data Quality Problems	171
28.5.5	Suppression	171
	Appendices	173

	References	175
A	Additional Resources	177
A.1	Learning R	177
A.2	Going Deeper in R	177
A.3	Visualization	177
A.4	Modeling and Statistics	178
A.5	Public Health Data Science	178
A.6	Databases from R	178
A.7	APIs and Public Data	178
A.8	Data Infrastructure	178
A.9	BI Platforms	179
A.10	Package Dependencies	179
A.11	Testing and Validation	179
A.12	Automation and Reproducible Environments	179
A.13	Accessibility	179
A.14	AI Tools and Governance	179
A.15	Team Practice and Onboarding	180
A.16	Migrating from Legacy Tools	180
A.17	Online Courses	180
A.18	Version Control	180
A.19	Quarto	180
B	Spending a Training Budget	181
B.1	Principles	181
B.2	Skills Worth Buying	181
B.3	Courses and Subscriptions	181
B.4	Books	182
B.5	Consulting	182
B.6	Procurement Constraints	182
B.7	A Worked Plan	183
B.8	Requesting Next Year's Funding	183
	Bibliography	185

Preface

Public health teams produce recurring analyses, reports, and dashboards, often with limited staff and time. This book covers practices that make that work easier to repeat, review, and share: version control, organized data, documented code, reproducible reports, and clear team responsibilities.

This book grew out of the [CSTE Data Science Team Training \(DSTT\)](#) program, where I serve as a project coach working with public health agencies across the country to build data science capacity in the public health workforce. The material here includes code, resources, workshop notes, and practical guidance accumulated and refined over several years of coaching teams at local and state health departments.

The technical chapters cover the organization, verification, and delivery of analytical work. The nontechnical chapters cover staffing, onboarding, project planning, data governance, review, and communication.

The testing, debugging, and API chapters (Chapter 4., Chapter 5., Chapter 18.) include worked examples with the code and explanations together. The targets section (Section 11.4) includes every file needed to build and run a small reporting pipeline.

While this material was created with DSTT participants in mind, it is intended to be broadly useful to anyone working at the intersection of data science and public health, whether you are a current or former DSTT participant, a public health practitioner looking to strengthen your analytical skills, or someone new to data science in a public health context.

Appendix A. lists further reading and training. Appendix B. covers choosing paid training and meeting procurement deadlines.

Cite this book:

Turner, Stephen D. (2026). *Data Science Team Training*, 2nd ed. Retrieved from <https://dstt.stephenturner.us/>.

```
@book{turner2026dstt,  
  title   = {Data Science Team Training},  
  author  = {Turner, Stephen D.},  
  date    = {2026},  
  edition = {2},  
  url     = {https://dstt.stephenturner.us/}  
}
```

This work is licensed under [CC BY-NC-SA 4.0](#).



Technical

1	Version Control	17
2	Data Organization	25
3	Data Validation and QA	31
4	Testing Analysis Code	35
5	Debugging and Getting Help	43
6	Reproducible Reporting	49
7	Accessibility	55
8	Coding with AI	61
9	Dashboards	63
10	BI Platforms	69
11	Automation and Scheduling	75
12	Package Loading	81
13	Managing Package Dependencies	87
14	Package Development	91
15	Reproducible Environments	95
16	Databases from R	103
17	Choosing and Paying for Data Infrastructure	111
18	APIs and Public Data Sources	117
19	Migrating from Legacy Workflows	123
20	Code Style	129
21	Documentation	133

1. Version Control

Git is a distributed version control system that tracks changes to files over time, allowing you to review history, revert mistakes, and collaborate without overwriting each other's work. For data science teams, version control is essential for reproducibility (knowing exactly which code produced which results), collaboration (multiple people working on the same project safely), and auditability (a clear record of what changed and why). GitHub is the most widely used platform for hosting Git repositories and adds features like pull requests, code review, and issue tracking on top of Git's core capabilities.

1.1 GitHub Setup

1.1.1 Creating a GitHub Account

Visit github.com and sign up for a free account. The free tier includes unlimited public and private repositories and is sufficient for most team workflows.

1.1.2 Adding an SSH Key

SSH keys let you authenticate with GitHub without entering your username and password on every push or pull. To generate a key:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Accept the default file location (`~/.ssh/id_ed25519`) and optionally set a passphrase. Then copy the public key:

```
cat ~/.ssh/id_ed25519.pub
```

In GitHub, go to **Settings** > **SSH and GPG keys** > **New SSH key**, paste the output, give it a descriptive title (e.g., your machine name), and save.

Verify the connection works:

```
ssh -T git@github.com
```

You should see a message like `Hi username! You've successfully authenticated.`

1.1.3 Cloning a Repository via SSH

When cloning a repository, prefer the SSH URL over HTTPS. On any GitHub repository page, click **Code** and select the **SSH** tab to get the URL.

```
git clone git@github.com:org/repo.git
```

Using SSH avoids repeated password prompts and works with SSH agent forwarding for server-based workflows.

1.1.4 Pulling and Pushing

After cloning, your local repository is linked to the remote (called `origin` by default). To sync your local branch with the latest changes from the remote:

```
git pull
```

After committing changes locally, push them to the remote:

```
git push
```

Git tracks the relationship between your local branch and the corresponding remote branch automatically, so these short-form commands will work once the tracking relationship is established.

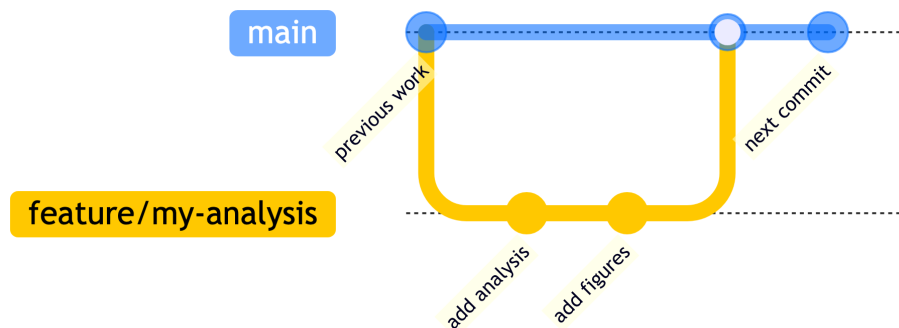
1.2 Branching and Merging

Branches let multiple people work on different features or fixes simultaneously without interfering with each other. The `main` branch (or `master` in older repositories) typically represents the stable, production-ready state of the project.

1.2.1 Internal Organizational Workflow

Teams can protect the `main` branch by requiring a pull request (PR) and review before merging changes. Developers create short-lived branches for each piece of work, then open a PR when ready.

Whatever branch naming convention a team adopts (tying the branch name to an issue number, as in `142-fix-denominator`, works well), record it where a new analyst will find it (Section 23.4). A practice repository, where someone can branch, break something, and open a PR without touching a production pipeline, is an effective way to teach this workflow (Section 23.6).



Step 1: Create a branch from `main`

```
git checkout main
git pull
git checkout -b feature/my-analysis
```

Step 2: Make changes, stage, and commit

```
git add analysis.R writeup.qmd
git commit -m "Add initial exploratory analysis for Q1 data"
```

Step 3: Push the branch to GitHub

```
git push -u origin feature/my-analysis
```

The `-u` flag sets the upstream tracking branch so future `git push` and `git pull` commands work without specifying the remote and branch name.

Step 4: Open a pull request

On GitHub, navigate to the repository. A banner will appear prompting you to open a PR from your recently pushed branch. Click **Compare & pull request**, add a description, assign reviewers, and submit.

Step 5: Review, merge, and delete

A reviewer approves the PR and merges it into `main` via the GitHub UI. After merging, delete the branch on GitHub (there is a button on the merged PR page). Locally, clean up with:

i Note

Code review is easier when diffs contain only meaningful changes. If one contributor reformats a file by hand and another does not, a PR's diff may be dominated by whitespace changes that obscure the actual logic. Using a formatter like Air (see Section 20.2) on every save means diffs are easier to review for changes in logic.

```
git checkout main
git pull
git branch -d feature/my-analysis
```

💡 Tip

Branch naming conventions help the team understand what a branch is for at a glance. Common prefixes:

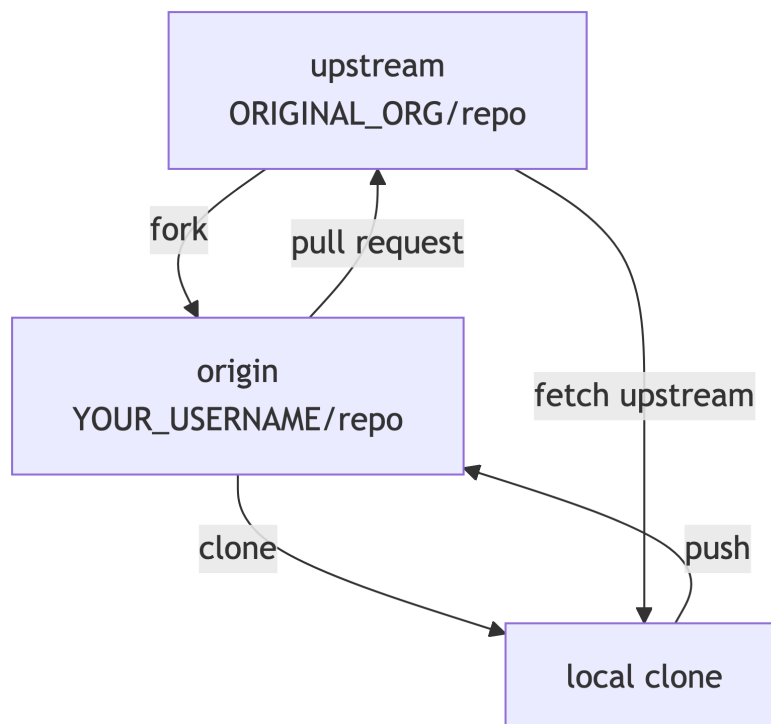
- **feature/**: new functionality (e.g., `feature/survival-analysis`)
- **fix/**: bug fixes (e.g., `fix/date-parsing-error`)
- **analysis/**: exploratory or one-off analyses (e.g., `analysis/q2-cohort`)
- **docs/**: documentation updates

i Note

Keep branches short-lived, ideally merged within a few days. Long-running branches diverge significantly from `main`, making merges painful and increasing the likelihood of conflicts.

1.2.2 Contributing to External Repositories

Use a fork when contributing to a repository you do not have write access to, such as an open-source tool or another team's public project. A fork is a personal copy of the repository under your GitHub account.



Step 1: Fork the repository

On the GitHub repository page, click **Fork** (top right) and choose your account as the destination.

Step 2: Clone your fork

```
git clone git@github.com:YOUR_USERNAME/repo.git
cd repo
```

Step 3: Add the original repository as **upstream**

```
git remote add upstream git@github.com:ORIGINAL_ORG/repo.git
```

Step 4: Branch, work, commit, and push to your fork

```
git checkout -b fix/typo-in-readme
# ... make changes ...
git add README.md
git commit -m "Fix typo in installation section"
git push -u origin fix/typo-in-readme
```

Step 5: Open a pull request to the upstream repository

On your fork's GitHub page, click **Contribute > Open pull request**. This creates a PR from your fork's branch into the original repository's `main` branch.

Step 6: Keep your fork in sync with upstream

As the original repository receives new commits, your fork will fall behind. Sync it with:

```
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

i Note

In the fork workflow, `origin` refers to **your fork** and `upstream` refers to the **original repository**. Keeping these straight avoids accidentally pushing to or pulling from the wrong remote.

1.3 Git in Positron

Positron includes a built-in Source Control panel (the branching icon in the left sidebar, or `Ctrl+Shift+G` / `Cmd+Shift+G`) that provides a graphical interface for the most common Git operations.

Viewing changes

The Source Control panel lists all files with uncommitted changes. Files are grouped into **Staged Changes** and **Changes** (unstaged). Click any file to open a diff view showing exactly what was added or removed.

Staging files

Hover over a file and click the `+` icon to stage it, or click the `+` next to the **Changes** heading to stage all modified files at once. To unstage, click the `-` icon next to a staged file.

Committing

Type a commit message in the text box at the top of the Source Control panel and click the **Commit** button (or press `Ctrl+Enter` / `Cmd+Enter`). This is equivalent to `git commit -m "your message"`.

Branching

The current branch name appears in the status bar at the bottom of the window. Click it to open the branch picker, where you can switch to an existing branch or create a new one. This is equivalent to `git checkout` or `git checkout -b`.

Pushing and pulling

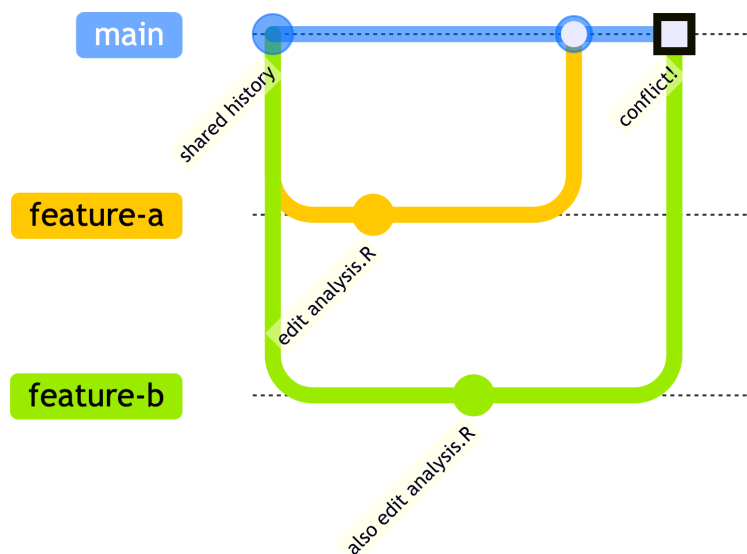
The Source Control panel has **Push** and **Pull** buttons in its toolbar (the `...` menu or the sync icon in the status bar). The sync button performs a pull followed by a push in one action.

💡 Tip

The integrated terminal in Positron (**Terminal > New Terminal**) is always available for Git operations that the UI doesn't expose, such as adding a remote, rebasing, cherry-picking, or any command requiring flags. The UI and terminal work on the same repository state, so you can mix both freely.

1.4 Merge Conflicts

A merge conflict occurs when two branches have made changes to the same lines of the same file, and Git cannot automatically determine which version to keep. Conflicts most commonly arise during `git merge`, `git rebase`, or when accepting a pull request that conflicts with recent changes to `main`.



1.4.1 What conflict markers look like

When a conflict occurs, Git edits the affected file to mark the conflicting regions:

```
<<<<<< HEAD
flu_clean <- flu |> filter(cases > 0, !is.na(county))
=====
flu_clean <- flu |> filter(!is.na(county), cases >= 1)
>>>>>> feature/update-filter-logic
```

- Everything between <<<<<< HEAD and ===== is the version from your current branch.
- Everything between ===== and >>>>>> is the version from the branch being merged in.

1.4.2 Resolving conflicts

1. Run `git merge <branch>` (or let a PR trigger the conflict). Git will report which files have conflicts.
2. Open each conflicted file. Positron highlights conflict regions with inline buttons: Accept Current Change, Accept Incoming Change, Accept Both Changes, or Compare Changes. Click the appropriate option or edit the file manually to produce the correct final version.
3. After resolving all conflicts in a file, save it and stage it:

```
git add path/to/resolved-file.R
```

4. Once all conflicted files are resolved and staged, complete the merge:

```
git commit
```

Git will pre-populate a commit message describing the merge; you can accept it as-is.

⚠ Warning

Never leave conflict markers (<<<<<<, =====, >>>>>>) in committed code. The file will be syntactically broken and the code will not run. Always verify the conflict is fully resolved before staging.

 Tip

Use `git status` during a conflict. It lists which files are in conflict (both `modified`), which are already resolved and staged, and what step to take next (commit or continue a rebase).

```
git status
```

1.5 .gitignore

A `.gitignore` file tells Git which files and directories to leave untracked. Matching untracked files are omitted from ordinary `git status` output and `git add ..` Already tracked files remain tracked, and `git add -f` can override ignore rules. Every project should have one.

1.5.1 Why it matters for data science

Data science projects routinely contain files that should never go into version control:

- Raw and processed data. Large files bloat a repository and slow every operation. Sensitive data files may have legal or compliance restrictions on where they can be stored. The project directory structure in Section 2.3 places a `.gitignore` directly inside `data/` and `output/` to block these files at the source.
- Credentials and secrets. `.env` files, config files with API keys or database passwords, and personal config files with paths to secure shared drives (see Section 2.3.2) should never be committed. Deleting a secret from the working file does not remove it from history. Revoke or rotate an exposed credential.
- Derived outputs. Figures, rendered reports, and cached results can be regenerated from code. Committing them creates noise in diffs and false impressions that something changed.
- Environment and editor artifacts. Files like `.DS_Store` (macOS), `.Rhistory`, `.RData`, and IDE-specific folders communicate nothing useful to collaborators.

1.5.2 Syntax

A `.gitignore` at the root of the repository applies to the whole project. The syntax is straightforward:

```
# Lines starting with # are comments

# Ignore a specific file
.env

# Ignore all files with this extension
*.csv

# Ignore generated outputs
output/

# Ignore a pattern anywhere in the tree
**/.DS_Store

# Exception: track this one file even though *.csv is ignored
!data/raw/small-example.csv
```

Patterns without a `/` match anywhere in the tree. A leading `/` anchors to the repository root. A trailing `/` matches directories only.

1.5.3 A starting point for R and Python projects

GitHub maintains a collection of language-specific templates at github.com/github/gitignore. When you create a new repository on GitHub, you can select one of these as a starting point. For most data science work in R and Python, a reasonable `.gitignore` covers:

```
# R
.Rhistory
.RData
.Rproj.user/
# Keep .Rproj files tracked so collaborators can open the project

# Python
__pycache__/_
*.py[cod]
.venv/
.env

# Data and outputs -- better handled with a .gitignore inside each folder
# (see project structure in the data organization chapter)
data/
output/

# OS artifacts
.DS_Store
Thumbs.db

# Credentials and personal config
.Renvirom
# Ignore personal configuration containing paths or secrets
config.yml
*.key
*.pem
```

⚠ Warning

A `.gitignore` only affects untracked files. If you accidentally committed a file before adding it to `.gitignore`, it will continue to be tracked. To stop tracking it without deleting it locally:

```
git rm --cached path/to/file
```

Then commit the removal and add the file to `.gitignore`.

💡 Tip

Per-directory `.gitignore` files work alongside the root one. The project structure in Section 2.3 places a `.gitignore` inside `data/` and another inside `output/`. This is a good pattern: the intent is visible right where the data lives, and it protects those folders even if the root `.gitignore` is later modified.

2. Data Organization

Organize data so that colleagues can find, read, and analyze it without manual cleanup. Consistent spreadsheets, file names, directories, and file formats make that work easier to repeat.

2.1 Data Organization in Spreadsheets

Broman and Woo's "Data Organization in Spreadsheets" [1] offers twelve practical recommendations for anyone who stores data in Excel or similar tools. Structure your data so that it can be read directly into R or Python without manual cleanup.

2.1.1 Be Consistent

Consistency is the single most important principle. Pick conventions and stick to them within a project:

- Use the same codes for categorical variables (e.g., always "M" and "F", not sometimes "Male")
- Use the same missing value code throughout (e.g., NA)
- Use the same variable names across files and time points
- Use the same date format everywhere

2.1.2 Make It a Rectangle

Raw data should be a single, clean rectangle:

- Rows are observations (subjects, samples, time points)
- Columns are variables
- One header row at the top with short, meaningful column names
- No merged cells, no blank rows used for visual separation, no summary rows mixed in

This is consistent with the principles of **tidy data** [2], where each variable forms a column, each observation forms a row, and each type of observational unit forms a table.

2.1.3 One Thing Per Cell

Each cell should contain exactly one value. Don't cram multiple pieces of information into a single cell (e.g., "120/80" for blood pressure; use two columns instead). Don't use a cell to store a note alongside a value.

2.1.4 Fill in All Cells

Don't leave cells blank to imply "same as above." Every cell should contain an explicit value. Use a consistent missing data code (e.g., NA) and document its meaning. Blank cells can leave readers unsure whether a value was missing, not collected, or zero.

2.1.5 Write Dates as YYYY-MM-DD

Use [ISO 8601](#) format for all dates: 2024-03-15. This format:

- Sorts correctly alphabetically
- Is unambiguous across locales (03/04/2024 means different things in the US vs. Europe)
- Is easy to recognize; verify how Excel stores and exports it

 Warning

Excel silently converts many text strings to dates (e.g., the human gene name `SEPT1` becomes a date). Consider storing year, month, and day in separate columns if you're collecting data in Excel, or prefix dates with an apostrophe to force text storage.

2.1.6 Choose Good Names for Things

Apply consistent naming rules to columns, sheets, and files:

- No spaces; use underscores (`_`) or hyphens (`-`)
- No special characters (`@`, `#`, `%`, `(`, `)`)
- Short but meaningful (`participant_id`, not `p` or `the_participant_identifier_number`)
- All lowercase is a safe default
- Avoid starting names with numbers

2.1.7 No Calculations in Raw Data Files

Keep raw data raw. Don't add sum rows, averages, or derived columns to the data file; do that in your analysis script. The raw data file is a permanent record of what was collected; calculations belong in code that can be inspected, corrected, and rerun.

2.1.8 Don't Use Formatting as Data

Color-coding, bold text, or cell highlighting to indicate something (e.g., red = outlier, bold = reviewed) is invisible to code. Add an explicit column instead: `is_outlier` (`TRUE/FALSE`) or `review_status` ("reviewed", "pending").

2.1.9 Create a Data Dictionary

Every dataset should be accompanied by a data dictionary (also called a codebook) that documents:

- Variable name
- Description of what it measures
- Units
- Allowable values or range
- How missing values are coded

A simple spreadsheet or CSV with one row per variable works fine.

2.1.10 Save Data in Plain Text

Save the analysis-ready version of your data as CSV (comma-separated values), not as `.xlsx`. CSV files:

- Can be opened by any software
- Are readable in version control diffs
- Don't have hidden formatting or macros
- Are stable long-term

Keep the original Excel file if needed, but always export a clean CSV as the file you hand off to analysis.

2.1.11 Make Backups

Raw data should be treated as read-only and backed up in at least two places (e.g., local drive + cloud storage). Consider these tiers:

- Primary: Your working copy
- Local backup: External drive or NAS
- Off-site/cloud: Institutional storage, OneDrive, or similar

Version control (see Chapter 1.) is excellent for code and small text files but is not ideal for large binary data files.

2.2 File Naming

Jenny Bryan’s “How to Name Files” [3] lays out three principles that make file names work well for both humans and computers. See also her talk from NormConf 2022:

<https://www.youtube.com/watch?v=ES1LTlnpLMk>

2.2.1 Three Principles

1. Machine readable

Use file names that are easy to parse in code:

- No spaces (use `_` or `-` instead)
- No special characters (`&`, `*`, `#`, `(`, `)`, accented letters)
- Avoid case sensitivity issues; stick to lowercase
- Use delimiters consistently so you can split on them: underscores between metadata fields, hyphens within a field

2. Human readable

The name should tell you what’s in the file without opening it:

- Include a descriptive “slug” that summarizes the content
- `2024-03-15-enrollment-summary.csv` is better than `data_v3_FINAL.csv`

3. Plays well with default ordering

Files should sort into a useful order automatically:

- Use ISO 8601 dates (`YYYY-MM-DD`) at the start of the name so files sort chronologically
- Left-pad numbers with zeros: `fig-01.png`, `fig-02.png`, ..., `fig-10.png` (not `fig-1.png`, `fig-2.png`, `fig-10.png`)

2.2.2 Recommended Pattern: YYYY-MM-DD-slug

For dated outputs (reports, snapshots, exports), the pattern `YYYY-MM-DD-descriptive-slug.ext` works well:

```
2024-03-15-enrollment-summary.csv
2024-06-01-adverse-events-cleaned.csv
2024-09-30-q3-interim-analysis.html
```

For numbered sequences (figures, scripts with a natural order), use zero-padded numbers:

```
01-import-data.R
02-clean-data.R
03-fit-models.R
04-make-figures.R
```

For files that don’t change often and aren’t dated, a simple descriptive slug is fine:

```
protocol.pdf
data-dictionary.xlsx
consent-form-v2.pdf
```

Tip

Consistent file names let you select files by pattern and extract dates, conditions, or sample IDs with code.

2.3 Project Directory Structure

2.3.1 Simple project-oriented workflow

A consistent folder structure makes projects navigable by collaborators (and your future self). One convention is shown below:

```

project-name/
├── data/
│   ├── .gitignore    # Don't commit data to version control.
│   ├── raw/          # original data -- never edited directly
│   └── processed/     # cleaned, analysis-ready files
├── R/                 # scripts and functions
├── output/
│   ├── .gitignore    # Don't commit large output files.
│   ├── figures/
│   └── tables/
└── report.qmd

```

Key rules:

- Preserve raw data. The `data/raw/` folder is read-only. Never overwrite or edit raw data files. Also, you don't want to commit large files to version control, so be careful with your `.gitignore`.
- All data manipulation is scripted. Cleaned data in `data/processed/` is generated by a script, not by hand.
- One project = one directory. Keep all files related to a project together.

The [ProjectTemplate](#) (R) and [Cookiecutter Data Science](#) (Python) tools can scaffold these structures automatically.

If several analysts maintain separate extracts of the same source, consider shared, curated datasets with documented versions and refresh dates. Chapter 17. covers the storage and access options.

2.3.2 Version controlling code with data on a shared drive

Finally, it's not uncommon in public health settings to have a setup where you're working with collaborators version controlling code as described in Chapter 1., but the data lives on a secure shared drive that cannot be moved or copied into the repository. Keep the code portable by configuring paths to the approved data location. The governance and access implications of working with data on secure shared drives, including data use agreements and secure data environments, are covered in Chapter 25..

Imagine that you have a distributed research team working with sensitive data that:

- Lives on a secure shared drive that cannot be moved
- Is organized in nested folders, for example, by state and year (e.g., `Lab/StateA/2023/`, `Lab/StateB/2024/`, etc.)
- Requires collaborative code development across Windows, Mac, and Linux machines
- Needs version control for reproducibility and collaboration

Storing data in a *database* would be preferable to a shared drive, but many public health agencies don't have the resources or infrastructure to set up and maintain databases. Configurable paths support this arrangement.

2.3.2.1 The Conflict

These approaches have drawbacks:

Option 1: One code repository without proper path management

- Forces hardcoded paths: `setwd("Z:/Lab/StateA/2024")`
- Breaks on different operating systems (Z:/ vs /Volumes/)
- Breaks when different team members have different drive mappings
- Violates [project-oriented workflow](#) principles
- Code is not reproducible

Option 2: Code repositories inside each data folder

- Creates dozens of scattered git repositories
- Makes code reuse nearly impossible
- Difficulty locating the current version of a shared function
- Accidental data commits to version control
- Multiple people doing git operations in the same shared folder can cause conflicts. These include file locking and permission errors.

Option 3: Move data into code repository

- Violates data security requirements
- Data cannot leave the secure shared drive
- Makes git repository enormous and slow
- Not feasible

2.3.2.2 A Better Way: Centralized Code Repository with Configurable Paths

Use a **single code repository with configurable paths**, cloned to each analyst's machine:

- One repository for all analysis code (easy to find, maintain, collaborate)
- Reproducible across team members with different operating systems
- Secure: data never leaves the shared drive or enters version control
- Flexible: works with nested folder structures
- Collaborative: each analyst works in a separate local clone
- Project-oriented: no `setwd()`, paths are configurable not hardcoded

See [stephenturner/demo-project-path-config](#) for an example of one way to do this using configuration files. This can also be achieved with environment variables. **How it works:**

- Code lives in a git repository on each person's local machine
- Each team member has a personal `config.yml` file (not version controlled) with their specific paths
- Code uses helper functions to construct paths dynamically
- Data stays on the shared drive (read-only for most operations)
- Outputs go to an approved location appropriate for their sensitivity
- Everyone commits code changes, never data

2.4 File Formats

Format	Best for	Avoid when
<code>.csv</code>	Tabular data, analysis input/output	Binary data, very wide datasets
<code>.tsv</code>	Tabular data with commas in values	None specific
<code>.json</code>	Nested/hierarchical data, configs	Large flat tables
<code>.parquet</code>	Large tabular data, columnar access	Simple small datasets
<code>.xlsx</code>	Data entry, sharing with non-coders	Final analysis-ready data

Plain text formats (CSV, TSV, JSON) are preferred for long-term storage and reproducibility. Keep portable exports of application-specific formats such as `.sav` and `.sas7bdat`. Although `.xlsx` uses an open specification, workbook formulas and formatting may be difficult to reproduce in other software. If you inherit data in these formats, Section 19.3 covers reading them into R and converting them to open formats.

2.5 Additional Best Practices

Use version control for code, not data. Git is designed for text files. Large or binary data files should live in dedicated storage (cloud drives, data repositories, databases) and be referenced in your code, not committed to the repository. Use a good `.gitignore`.

Document your cleaning steps. Generate processed data with scripts that authorized colleagues can run. Comments in that script explaining *why* you made certain decisions are invaluable.

Plan for sharing. Before a project ends, ask: could someone else reproduce this analysis from the raw data? Good file organization, consistent naming, and a clear README make the answer yes.

Avoid “final” in file names. `analysis-final-FINAL-v3-USE-THIS-ONE.R` is a sign that version control was not used. Use Git instead, and name the file simply `analysis.R`.

3. Data Validation and QA

An analysis can run to completion on bad data. Negative case counts, duplicate records, and unexpected column types may affect tables and figures without stopping the code. Check for these problems before using the data.

The remedy is to validate data at the moment of ingestion, before any analysis runs. If the data does not meet your expectations, the code should fail immediately and loudly, with a message that explains why. This chapter covers two tools for doing that: base R assertions for simple, targeted checks, and the `pointblank` package for structured validation of an entire dataset.

Throughout this chapter, the examples use the following simulated weekly influenza surveillance dataset. It has four deliberate problems: a negative case count, a missing county, a missing rate, and a duplicate record.

```
flu <- data.frame(
  week = c(1, 2, 3, 4, 4),
  county = c("Fairfax", "Arlington", NA, "Loudoun", "Loudoun"),
  disease = c("Flu", "Flu", "Flu", "Flu", "Flu"),
  cases = c(23, 41, 18, -5, 12),
  rate = c(2.1, 3.8, 1.6, NA, 1.1)
)
```

flu

	week	county	disease	cases	rate
1	1	Fairfax	Flu	23	2.1
2	2	Arlington	Flu	41	3.8
3	3	<NA>	Flu	18	1.6
4	4	Loudoun	Flu	-5	NA
5	4	Loudoun	Flu	12	1.1

3.1 Failing Fast

The simplest validation is a condition check paired with `stop()`. If the condition fails, execution halts and the message you wrote appears in the console. With `error: true`, Quarto also includes the error in the rendered output.

```
if (any(flu$cases < 0, na.rm = TRUE)) {
  stop("Negative case counts detected. Inspect raw data before proceeding.")
}
```

Error:
! Negative case counts detected. Inspect raw data before proceeding.

The `error: true` chunk option lets the document continue rendering even after an error, as needed for this demonstration. In a real analysis, you would not set this option; you would want rendering to stop until the data problem is fixed.

For multiple checks, `stopifnot()` is more concise. Named arguments become the error message, so the output tells you which check failed:

```
stopifnot(
  "Negative case counts" = all(flu$cases >= 0, na.rm = TRUE),
  "Missing county values" = !anyNA(flu$county),
)
```

```
"Duplicate records" = !anyDuplicated(flu[, c("week", "county")])
)
```

```
Error:
! Negative case counts
```

The error stops at the first failure. If `cases` and `county` both have problems, you only learn about `cases` until you fix it and re-run. For a dataset with many potential issues, that feedback loop is slow.

3.2 Structured Validation with pointblank

`pointblank` replaces the one-at-a-time assertion model with a declarative pipeline: you describe every expectation up front, run them all at once, and get a structured report showing which passed and which failed, without stopping at the first problem.

```
install.packages("pointblank")
```

3.2.1 Basic Workflow

The core workflow has three steps: create an agent bound to the data, add validation steps, then interrogate. The checks below detect the negative count, missing county, and duplicate week/county key in `flu`.

Note

The pointblank outputs are only rendered in html format at <https://dstt.stephenturner.us/>. These interactive formats are not rendered in the PDF or EPUB versions of this guide.

```
library(pointblank)

agent <- create_agent(tbl = flu, label = "Weekly flu surveillance") |>
  col_vals_gte(
    columns = cases,
    value = 0,
    label = "Case counts must be non-negative"
  ) |>
  col_vals_not_null(
    columns = c(week, county),
    label = "Week and county cannot be missing"
  ) |>
  rows_distinct(
    columns = c(week, county),
    label = "No duplicate week/county records"
  ) |>
  interrogate()

agent
```

All three checks fail: row 4 has `cases = -5`, row 3 has `county = NA`, and rows 4 and 5 are both `week = 4`, `county = "Loudoun"`. Each failure row shows the count of offending records. All three checks ran, so you can investigate the failures together.

3.2.2 Adding More Checks

Real surveillance data has more constraints. A few more validations on the same dataset:

```
create_agent(tbl = flu, label = "Weekly flu surveillance: extended") |>
  col_is_numeric(
    columns = c(cases, rate),
    label = "Case count and rate must be numeric"
```

```

) |>
col_vals_in_set(
  columns = disease,
  set = c("Flu", "COVID-19", "RSV"),
  label = "Disease must be from the approved list"
) |>
col_vals_between(
  columns = week,
  left = 1,
  right = 53,
  label = "Week must be between 1 and 53"
) |>
col_vals_gte(
  columns = rate,
  value = 0,
  na_pass = TRUE,
  label = "Rate must be non-negative (NAs allowed)"
) |>
interrogate()

```

The `na_pass = TRUE` argument on the rate check lets missing values pass that particular step. For this rate variable, we allow missing values but reject negative ones. These two conditions are separate checks and should be stated separately.

3.2.3 Common Checks for Public Health Data

Validation	pointblank function	Use for
No negative counts	<code>col_vals_gte(value = 0)</code>	Case counts, deaths, population denominators
No missing required fields	<code>col_vals_not_null()</code>	County, date, disease code
No duplicate records	<code>rows_distinct()</code>	Surveillance records keyed by geography + time
Values from a known list	<code>col_vals_in_set()</code>	Disease codes, county names, FIPS codes
Dates within a valid range	<code>col_vals_between()</code>	Report dates, surveillance weeks
Correct column type	<code>col_is_numeric()</code> , <code>col_is_date()</code>	Catches silent type coercion on import
Counts do not exceed a known total	<code>col_vals_lte()</code>	Deaths $\leq$ total cases, OD deaths $\leq$ total deaths

3.2.4 Stopping on Failure

By default, `interrogate()` returns the full report without stopping execution. To stop an automated pipeline when validation fails, check the result with `all_passed()`:

```

if (!all_passed(agent)) {
  stop("Data validation failed. Review the agent report before proceeding.")
}

```

This gives you the structured report from pointblank for diagnosis while still failing loudly enough to stop an automated run.

3.3 Where to Put Validation

Validation belongs in the setup block, immediately after data is read in and before any transformation or analysis begins. Add further checks after transformations that could introduce errors, such as joins and aggregations.

```
library(readr)
library(pointblank)

flu <- read_csv("data/flu-2024.csv")

# Validate immediately after reading
agent <- create_agent(tbl = flu, label = "flu-2024 validation") |>
  col_vals_gte(columns = cases, value = 0, label = "No negative counts") |>
  col_vals_not_null(columns = c(week, county), label = "No missing keys") |>
  rows_distinct(columns = c(week, county), label = "No duplicate records") |>
  interrogate()

if (!all_passed(agent)) {
  stop("Validation failed: see agent report above.")
}

# Analysis only runs if validation passes
```

Putting validation here means the failure is traceable to a specific data file and a specific expectation. The message tells you what was wrong, not just that something broke three steps later in an `aes()` call.

Tip

Commit your validation code to version control alongside the analysis (see Chapter 1.). When a data supplier changes a format, adds a new disease code, or starts providing a column in a different type, your validation catches it on the next render, if you have checks for those changes. Data pulled from a web API deserves the same treatment, for reasons covered in Section 18.7. When validation failures trace back to a change in an upstream data system, Chapter 25. covers how to work with IT and data stewards to resolve it.

4. Testing Analysis Code

A test records what a piece of code should do and checks that behavior after changes. For a recurring analysis, prioritize decisions that affect published results: denominator selection, reporting periods, suppression, and joins. Chapter 3. covers checks on the incoming data.

These rules often appear in several reports and outlast the person who first wrote them. A colleague editing a suppression function needs to know whether zero should be retained, what happens to missing values, and whether the threshold itself is suppressed. Tests record those decisions as examples the team can run. They help reviewers assess a change and can detect a broken rule before it affects a report.

You can use the `testthat` package in an ordinary analysis project. Creating an R package is optional; if the functions later become a package, their tests can move with them (Chapter 14.). Install `testthat` once, then load it in the session where you run the examples:

```
install.packages("testthat")
```

```
library(testthat)
```

4.1 Start with Functions

A calculation embedded in a long reporting script can be difficult to test on its own. Extracting it into a function lets you supply small inputs and inspect the result without importing the full dataset or rendering the report. It also gives the team one definition to maintain when several parts of an analysis use the same rule.

Start with a rate calculation:

```
calc_rate <- function(cases, population, per = 100000) {  
  cases / population * per  
}
```

This version handles ordinary numeric inputs. It does not yet check input types or define what should happen with a zero denominator. We will add those requirements and tests below.

Write down what callers can expect before choosing tests. For this function, rates use per-100,000 units by default. A shared population should work with several case counts, and vectors of equal length should be calculated element by element. Missing counts and zero or missing populations should return numeric missing values. Negative values and incompatible vector lengths should fail. These are choices about the function's behavior; a different analysis may need different rules.

4.2 First Tests with `testthat`

A `test_that()` block names a behavior and checks it with one or more expectations. Use invented inputs whose answers you can calculate independently:

```
test_that("rates use per-100,000 units and accept a shared population", {  
  expect_equal(calc_rate(5, 50000), 10)  
  expect_equal(calc_rate(c(3, 4), 100000), c(3, 4))  
})
```

```
Test passed with 2 successes 🍌.
```

Five cases in a population of 50,000 gives a rate of 10 per 100,000. That expected answer stays fixed even if someone rewrites the calculation. Repeating the function's own formula inside the expectation could repeat the same error.

Suppose someone changes the default units to per 1,000. This deliberately incorrect version shows how the test reports the change:

```
calc_rate_changed <- function(cases, population, per = 1000) {
  cases / population * per
}

test_that("rates use per-100,000 units", {
  expect_equal(calc_rate_changed(5, 50000), 10)
})
```

```
— Failure: rates use per-100,000 units —————
Expected `calc_rate_changed(5, 50000)` to equal 10.
Differences:
1/1 mismatches
[1] 0.1 - 10 == -9.9
```

```
Error:
! Test failed with 1 failure and 0 successes.
```

The failure identifies the test and compares the actual value, 0.1, with the expected value, 10. That tells the maintainer which requirement no longer holds. A description such as “test calc_rate” would give much less information. Name the behavior so someone unfamiliar with the implementation can understand the failure.

4.3 Choosing Expectations

Choose the expectation to match the requirement:

What matters	Expectation	Example
Numeric values, allowing floating-point tolerance	<code>expect_equal()</code>	A calculated rate
Exact value and type	<code>expect_identical()</code>	Numeric missing values or an empty numeric vector
Rejection of invalid input	<code>expect_error()</code>	Negative counts or incompatible vector lengths
A logical condition	<code>expect_true()</code>	An invariant required by a transformation
Size of a result	<code>expect_length()</code>	One result for each input count

The difference between numeric equality and exact identity matters because floating-point calculations can introduce small rounding differences:

```
0.1 + 0.2 == 0.3
```

```
[1] FALSE
```

```
expect_equal(0.1 + 0.2, 0.3)
```

The direct comparison returns `FALSE`, but `expect_equal()` passes within its numeric tolerance. Use it for calculated quantities. Use `expect_identical()` when the type is part of the requirement, such as a helper that must return a double vector even when every value is missing.

`expect_error()` checks that an invalid call fails. It passes when the expected error occurs:

```
test_that("rates reject character case counts", {
  expect_error(calc_rate("3", 100000))
})
```

Test passed with 1 success 🐱.

This simple version passes because R rejects arithmetic on character input. The complete implementation below supplies a more specific message. Matching part of that message in a test helps distinguish the intended input check from an unrelated error.

4.4 Edge Cases

For an illustrative suppression rule, keep zero and mask positive counts below 5. The important boundaries are 0, 1, 4, and 5; missing values should stay missing. Confirm your agency's actual rule before adapting this example (Section 25.6). This helper does not implement complementary suppression or establish that a release is safe.

A short implementation looks plausible:

```
suppress_small_naive <- function(x, threshold = 5) {
  ifelse(!is.na(x) & x > 0 & x < threshold, NA, x)
}
```

But suppose callers require a double vector. Test an input where every count is suppressed:

```
test_that("suppressed counts remain numeric", {
  expect_identical(suppress_small_naive(3), NA_real_)
})
```

```
— Failure: suppressed counts remain numeric —————
Expected `suppress_small_naive(3)` to be identical to NA_real_.
Differences:
Modes: logical, numeric
target is logical, current is numeric
```

```
Error:
! Test failed with 1 failure and 0 successes.
```

The bare `NA` is logical. For this input, `ifelse()` returns a logical missing value, so the test fails. This can violate a caller's type requirements even though the printed result looks right. Replacing `NA` with `NA_real_` handles this case, but an empty input to `ifelse()` still returns `logical(0)`. To guarantee a double result, convert the input first and then replace the selected values.

The following implementation also rejects invalid counts. Both helpers in this chapter use `check_numeric()` to require plain numeric vectors with finite, nonnegative values or missing values. Counts must be whole numbers:

```
check_numeric <- function(x, name, whole = FALSE) {
  if (!is.numeric(x) || is.object(x) || !is.null(dim(x))) {
    stop(name, " must be a plain numeric vector", call. = FALSE)
  }
  present <- x[!is.na(x)]
  if (any(!is.finite(present)) || any(present < 0)) {
    stop(name, " must contain finite, nonnegative values or NA", call. = FALSE)
  }
  if (whole && any(present != floor(present))) {
    stop(name, " must contain whole counts or NA", call. = FALSE)
  }
}
```

```
suppress_small <- function(x, threshold = 5) {
  check_numeric(x, "x", whole = TRUE)
  check_numeric(threshold, "threshold", whole = TRUE)
  if (length(threshold) != 1L || is.na(threshold) || threshold < 1) {
    stop("threshold must be one positive whole number", call. = FALSE)
  }
}
```

```

}
out <- as.double(x)
out[is.na(out) | (!is.na(out) & out > 0 & out < threshold)] <- NA_real_
out
}

```

Check the policy boundaries and the type requirements together:

```

test_that("suppression preserves boundaries and the output type", {
  expect_identical(
    suppress_small(c(0, 1, 4, 5, NA)),
    c(0, NA_real_, NA_real_, 5, NA_real_)
  )
  expect_identical(suppress_small(3), NA_real_)
  expect_identical(suppress_small(numeric(0)), numeric(0))
})

```

Test passed with 3 successes 🍬.

Each input checks a separate decision. Zero is retained, counts 1 and 4 are masked, and 5 is retained because the threshold is exclusive. The last expectation checks an empty result, which may occur after filtering. Test cases like these explain the rule to a future maintainer without requiring them to infer it from the function.

4.5 Regression Tests

When debugging identifies an error (Chapter 5), first reproduce it in a test. For example, a new row in a population lookup table might contain zero. The initial rate function returns infinity:

```
calc_rate(3, 0)
```

```
[1] Inf
```

Record the intended behavior before changing the function:

```

test_that("zero or missing populations yield numeric missing rates", {
  expect_identical(calc_rate(3, 0), NA_real_)
  expect_identical(calc_rate(3, NA_real_), NA_real_)
})

```

```

— Failure: zero or missing populations yield numeric missing rates —————
Expected `calc_rate(3, 0)` to be identical to NA_real_.
Differences:
1/1 mismatches
[1] Inf - NA == NA

```

```

Error:
! Test failed with 1 failure and 1 success.

```

Seeing the test fail confirms that it detects the reported problem. After the fix, keep it in the suite so later changes are checked against the same case.

A tempting fix uses `ifelse()` with a condition based only on population. That creates another problem when the population is a single value and case counts form a vector:

```

calc_rate_ifelse <- function(cases, population, per = 100000) {
  ifelse(is.na(population) | population <= 0,
        NA_real_, cases / population * per)
}

test_that("one population can be used with several case counts", {

```

```
expect_equal(calc_rate_ifelse(c(3, 4), 100000), c(3, 4))
})
```

```
— Failure: one population can be used with several case counts —————
Expected `calc_rate_ifelse(c(3, 4), 1e+05)` to equal `c(3, 4)`.
Differences:
Lengths differ: 1 is not 2
```

```
Error:
! Test failed with 1 failure and 0 successes.
```

The condition has length one, so `ifelse()` returns only the first rate. A test of `calc_rate_ifelse(3, 0)` alone would miss this. Run earlier tests after a fix as well as the test for the newly reported bug.

Here is the complete corrected rate function. It uses `check_numeric()` from the previous section, checks input lengths, and expands a scalar to match the other vector before calculating. Two empty inputs, or an empty input paired with a scalar, return `numeric(0)`. Other unequal lengths fail:

```
calc_rate <- function(cases, population, per = 100000) {
  check_numeric(cases, "cases", whole = TRUE)
  check_numeric(population, "population")
  check_numeric(per, "per")
  if (length(per) != 1L || is.na(per) || per <= 0) {
    stop("'per' must be one finite positive number", call. = FALSE)
  }
  nc <- length(cases)
  np <- length(population)
  if (nc != np && nc != 1L && np != 1L) {
    stop("cases and population must have equal lengths or one must be scalar",
        call. = FALSE)
  }
  if (nc == 0L || np == 0L) return(numeric(0))
  n <- max(nc, np)
  cases <- rep_len(as.double(cases), n)
  population <- rep_len(as.double(population), n)
  rate <- rep(NA_real_, n)
  usable <- !is.na(cases) & !is.na(population) & population > 0
  rate[usable] <- cases[usable] / population[usable] * per
  if (any(!is.na(rate) & !is.finite(rate))) {
    stop("rate exceeds the finite numeric range", call. = FALSE)
  }
  rate
}
```

Now check the repaired behavior and the ordinary calculations:

```
test_that("zero and missing inputs have explicit results", {
  expect_identical(calc_rate(3, 0), NA_real_)
  expect_identical(calc_rate(3, NA_real_), NA_real_)
  expect_identical(calc_rate(NA_real_, 100), NA_real_)
  expect_identical(
    calc_rate(c(3, NA_real_, 0, 4), c(0, 100, 100, NA_real_)),
    c(NA_real_, NA_real_, 0, NA_real_)
  )
})
```

```
Test passed with 4 successes 🌈.
```

```
test_that("rates retain the stated units and vector behavior", {
  expect_equal(calc_rate(5, 50000), 10)
```

```
expect_equal(calc_rate(c(3, 4), 100000), c(3, 4))
expect_equal(calc_rate(3, c(100000, 50000)), c(3, 6))
expect_equal(calc_rate(c(3, 4), c(100000, 50000)), c(3, 8))
expect_equal(calc_rate(5, 50000, per = 1000), 0.1)
})
```

Test passed with 5 successes 🌈.

The remaining tests cover empty inputs, invalid values, and changes to the suppression threshold. These cases follow from the stated requirements:

```
test_that("empty vectors keep a numeric type", {
  expect_identical(calc_rate(numeric(0), numeric(0)), numeric(0))
  expect_identical(calc_rate(numeric(0), 100), numeric(0))
  expect_identical(calc_rate(3, numeric(0)), numeric(0))
  expect_identical(suppress_small(numeric(0)), numeric(0))
})
```

Test passed with 4 successes 🌈.

```
test_that("invalid inputs fail instead of being silently recycled or coerced", {
  expect_error(calc_rate(1:2, c(100, 200, 300)), "equal lengths")
  expect_error(calc_rate(numeric(0), c(100, 200)), "equal lengths")
  expect_error(calc_rate("3", 100), "numeric vector")
  expect_error(calc_rate(matrix(3), 100), "numeric vector")
  expect_error(calc_rate(-1, 100), "nonnegative")
  expect_error(calc_rate(0.5, 100), "whole counts")
  expect_error(calc_rate(1, -100), "nonnegative")
  expect_error(calc_rate(1, Inf), "finite")
  expect_error(calc_rate(1, 100, per = 0), "positive")
  expect_error(calc_rate(1, 100, per = NA_real_), "positive")
  expect_error(calc_rate(1, 100, per = c(100, 1000)), "one finite")
})
```

Test passed with 11 successes 🌈.

```
test_that("suppression implements the illustrative policy without changing type", {
  expect_identical(suppress_small(c(0, 1, 4, 5, 12, NA)),
    c(0, NA_real_, NA_real_, 5, 12, NA_real_))
  expect_identical(suppress_small(1:4), rep(NA_real_, 4))
  expect_identical(suppress_small(c(0L, 5L)), c(0, 5))
  expect_identical(suppress_small(c(5, 9, 10), 10), c(NA_real_, NA_real_, 10))
  expect_error(suppress_small(-1), "nonnegative")
  expect_error(suppress_small(1.5), "whole counts")
  expect_error(suppress_small(3, threshold = 0), "positive whole")
  expect_error(suppress_small(3, threshold = c(5, 10)), "one positive")
})
```

Test passed with 8 successes 🐛.

A passing suite establishes that these cases work. It does not prove correctness for every input. Add tests when requirements change or a new failure exposes a case the suite missed.

4.6 Organizing Tests in a Project

The examples above run inline so you can see each failure and fix. In an analysis project, keep the final functions and their tests in separate files:

```
flu-report/  
  R/  
    rates.R  
  tests/  
    test-rates.R  
  report.qmd
```

Put the final `check_numeric()`, `suppress_small()`, and `calc_rate()` definitions in `R/rates.R`. Start `tests/test-rates.R` with this line:

```
source(file.path("../R", "rates.R"))
```

Then copy the passing `test_that()` blocks into that test file. Omit the deliberately failing demonstrations and the functions named `calc_rate_changed`, `suppress_small_naive`, and `calc_rate_ifelse`. Test file-names must begin with `test` for `testthat` to find them. `test_dir()` runs these tests from the test directory, which is why the source path starts with `...`

Run the suite from the project root:

```
testthat::test_dir("tests")
```

Run relevant tests while changing a rule and the full suite before committing (Chapter 1). For shared projects, also run them in continuous integration (Section 11.3), where a clean session can reveal dependencies on your local workspace. Assign responsibility for investigating failures; ignored failures do not protect the analysis.

Shared functions used across projects may belong in a package (Chapter 14). Package tests have a standard location, `tests/testthat/`, and can be run with `devtools::test()`. The expectations carry over; the package handles loading its functions, so remove the `source()` line.

4.7 Tests and Validation Are Different Checks

Tests use controlled inputs to check code behavior. Validation checks the actual input and intermediate data for each run. A passing rate-function test cannot establish that this week's extract includes every county. A validation check for complete county coverage cannot establish that the rate function uses the correct units.

When a data problem exposes a code bug, add both the relevant validation rule and a regression test. If a zero population caused an infinite rate, the input validation should flag the unexpected denominator, and the function test should check how the calculation handles it. Include both checks in code review so reviewers can see how the incident was addressed (Chapter 27.).

4.8 Further Reading

The [testing chapters of *R Packages*](#) explain test design, fixtures, and organization. The [testthat documentation](#) describes individual expectations and snapshot testing.

5. Debugging and Getting Help

Code can stop with an error or run to completion and produce a wrong answer. Validation and tests help catch both kinds of problems (Chapter 3., Chapter 4.). Debugging identifies their causes.

Read the full error, find the line where the result differs from your expectation, and inspect the objects used there. Tools such as `traceback()`, `browser()`, and `reprex` help you locate the problem and give someone else enough information to help.

5.1 Reading the Error Message

Read the error message before changing code. It usually identifies the failing operation and may show the inputs or function calls involved.

A tidyverse error may identify the outer operation and then a more specific cause. This example uses invented case counts. The column is named `cases`, but the summary refers to `case`:

```
library(dplyr)

flu <- data.frame(
  week = c(1, 1, 2, 2),
  county = c("Fairfax", "Arlington", "Fairfax ", "Arlington"),
  cases = c(23, 41, 30, 38)
)
```

```
flu |>
  group_by(county) |>
  summarise(total = sum(case))
```

```
Error in `summarise()`:
! In argument: `total = sum(case)`.
! In group 1: `county = "Arlington"`.
Caused by error:
! object 'case' not found
```

The line after `Caused by error:` identifies the missing object: `case`. The surrounding message tells you which expression failed (`total = sum(case)`) and which group was being processed. Read both parts, then check `names(flu)` to confirm the spelling. Changing `case` to `cases` fixes this error. The extra space in one county name is a separate problem that we will investigate in Section 5.4.

Base R errors are terser, and a few of them are famous for saying something true but unhelpful. These are worth recognizing on sight, because each cryptic message maps to a small set of likely causes:

Message	What it usually means
object of type 'closure' is not subsettable	You subsetted a function. Usually your data frame was never created, and its name collides with a function (<code>df</code> , <code>data</code>)
could not find function "functionname"	The package providing <code>functionname()</code> is not loaded; add the <code>library()</code> call
object 'x' not found	A typo, or code run out of order so the object does not exist yet
non-numeric argument to binary operator	Arithmetic on a character column, often one that was silently imported as text

Message	What it usually means
<code>undefined columns selected</code>	A column name in <code>df[, "name"]</code> does not exist, often a typo or a renamed column
<code>replacement has 4 rows, data has 5</code>	Assigning a vector of the wrong length into a data frame column

The “closure” error is easier to understand with an example. R’s stats package provides a function named `df`, which computes the density of the F distribution. If a line meant to create a data frame called `df` never ran, that name can still resolve to the function:

```
df$cases
```

```
Error in `df$cases`:
! object of type 'closure' is not subsettable
```

R cannot extract a column from a function. Check `typeof(df)` to confirm what the name refers to, then inspect the line that should have created the data frame. An earlier error or code run out of order may explain why the object is missing.

When a message means nothing to you, search for it. Strip out the parts specific to your code (your variable names, your file paths) and search the generic remainder in quotes. An error that has existed for years has been asked about publicly many times, and the same stripped message pasted to an AI assistant with a bit of surrounding code can help you develop a diagnosis to test (Chapter 8.).

5.2 Locating the Failure with `traceback()`

When functions call other functions, the error may occur several calls away from the line you ran. Here, `prep_report_data()` calls `add_rates()`, which calls a small rate function. The rate function checks whether its population argument is numeric:

```
rate_demo <- function(cases, population, per = 100000) {
  if (!is.numeric(population)) stop("population must be numeric")
  cases / population * per
}

add_rates <- function(df) {
  df$rate <- rate_demo(df$cases, df$popn)
  df
}

prep_report_data <- function(df) {
  add_rates(df)
}
```

```
flu2 <- data.frame(
  county = c("Fairfax", "Arlington"),
  cases = c(23, 41),
  population = c(1147532, 238643)
)

prep_report_data(flu2)
```

```
Error in `rate_demo()`:
! population must be numeric
```

Run these definitions and the failing call in your R console, then immediately run `traceback()`. It shows the calls leading to the error:

```
> traceback()
4: stop("population must be numeric")
```

```
3: rate_demo(df$cases, df$popn)
2: add_rates(df)
1: prep_report_data(flu2)
```

Read from your top-level call at 1 toward the failure at 4. Call 3 shows the arguments supplied by `add_rates()`: `df$cases` and `df$popn`. The table has a column named `population`, so `df$popn` returns `NULL`. The numeric check in `rate_demo()` rejects that value. The error occurs inside the rate calculation, but the incorrect column name is in its caller.

Without the numeric check, dividing by `NULL` would produce a zero-length result, and assigning it to this nonempty data frame would still fail. The check gives a more specific message and stops the calculation earlier. This demonstration checks only the population's type; a production rate function also needs checks for values such as zero or missing denominators (Chapter 4).

For tidyverse errors, `rlang::last_trace()` provides additional context. Run it in the console after the failing `summarise()` example above to see where that expression was evaluated.

5.3 Pausing Inside a Function with `browser()`

The traceback shows the calls, but you may also need to inspect the objects inside a function. To investigate the `add_rates()` failure, run these two lines in the console:

```
debugonce(add_rates)
prep_report_data(flu2)
```

`debugonce()` pauses the next call to `add_rates()` before its body runs. At the `Browse[1]>` prompt, expressions are evaluated inside the paused function. Here, `df` is the data frame passed to `add_rates()`, so you can inspect its names and the value being passed to `rate_demo()`:

```
Browse[1]> names(df)
[1] "county"      "cases"      "population"
Browse[1]> df$popn
NULL
Browse[1]> df$population
[1] 1147532 238643
```

This confirms the column-name error. You can also inspect dimensions, print a subset of rows, or evaluate individual expressions before allowing the function to continue. These commands control execution:

Command	Effect
<code>n</code>	Run the next line, staying in this function
<code>s</code>	Step into the next function call
<code>c</code>	Continue running
<code>Q</code>	Quit the debugger and abandon the call
<code>where</code>	Print the current call stack

For a longer function, inserting `browser()` lets you pause at a specific point. For example, this version pauses just before calculating the rates:

```
add_rates <- function(df) {
  browser()
  df$rate <- rate_demo(df$cases, df$popn)
  df
}
```

The same inspection commands work with either approach. Remove any inserted `browser()` call before committing. [Positron's debugger documentation](#) explains the corresponding editor controls.

Quit the debugger with `Q`, correct the column name, and run the function again:

```
add_rates <- function(df) {
  df$rate <- rate_demo(df$cases, df$population)
```

```
df
}

prep_report_data(flu2)
```

	county	cases	population	rate
1	Fairfax	23	1147532	2.004301
2	Arlington	41	238643	17.180475

Check the returned rates against an independent calculation. Keep a test using these inputs and expected rates so a later edit cannot reintroduce this failure unnoticed (Section 4.5).

5.4 When the Code Runs but the Answer Is Wrong

A pipeline can finish while returning an incorrect count or dropping a county. Compare the result with a known expectation, then inspect intermediate results to locate the first unexpected change.

The `flu` data created earlier includes a week 2 record for Fairfax. This filter returns no rows:

```
flu |>
  filter(county == "Fairfax", week == 2)
```

```
[1] week  county cases
<0 rows> (or 0-length row.names)
```

In a longer pipeline, run successive steps and compare row counts and values with your expectations. Once you find the first unexpected result, inspect that step's inputs. Here, there is only one step, so inspect the values being compared:

```
flu |> count(county)
```

```
  county n
1 Arlington 2
2  Fairfax 1
3  Fairfax 1
```

```
dput(unique(flu$county))
```

```
c("Fairfax", "Arlington", "Fairfax ")
```

`count()` reports separate groups for the two spellings of Fairfax. The quoted output from `dput()` makes the difference visible: "Fairfax " has a trailing space. It does not equal "Fairfax", so the comparison excludes the week 2 record. Similar problems occur with inconsistent capitalization or lookalike characters in manually entered data (Section 19.7).

For these county names, removing surrounding whitespace is appropriate. Confirm that rule before applying it to other fields; some identifiers require exact matching.

```
flu |>
  mutate(county = trimws(county)) |>
  filter(county == "Fairfax", week == 2)
```

```
  week county cases
1     2 Fairfax    30
```

The result now contains the expected record, with 30 cases.

A few habits make this whole class of bug surface quickly:

- After any `filter()` or `join`, check `nrow()` against a rough expectation. If a left join expected to be one-to-one adds rows, check for duplicate keys. If columns from the right-hand table contain unexpected NA values, check for unmatched keys with `anti_join()`.

- `count()` on any grouping column before grouping by it. This reveals near-duplicate categories like the one above.
- Print `summary()` of a numeric column before and after a transformation. Impossible values (negative counts, percentages above 100) are visible in the min and max.

Keep useful diagnostic checks as validation rules. The trailing-space incident should end with a `col_vals_in_set()` check on `county` in your pointblank agent (Section 3.2), so validation detects unexpected county names at import. If the wrong answer traced to a function you wrote, it should also end with a regression test (Section 4.5).

5.5 Shrinking the Problem: Minimal Reproducible Examples

If the problem remains unclear, reduce it to the smallest dataset and shortest code that still reproduce it. This minimal reproducible example, or *reprex*, helps you investigate the failure and ask for help.

Shrinking is diagnostic in its own right. Start deleting things: pipeline steps, columns, rows, packages. If the bug disappears when you remove a step, that step is implicated. If it persists with a 4-row toy data frame, you can investigate using the toy data. You may find the cause while reducing the example.

A reprex must include enough code and data for someone else to reproduce the failure in a fresh session. The filter problem above needs just one row and the failing comparison:

```
library(dplyr)

example_cases <- data.frame(county = "Fairfax ", cases = 30)
example_cases |> filter(county == "Fairfax")
```

```
[1] county cases
<0 rows> (or 0-length row.names)
```

Someone can copy this block alone and reproduce the empty result. It includes the package, the data, and the operation, with no dependence on objects created earlier in the chapter. An accompanying question could state: “I expected the Fairfax row with 30 cases, but the filter returns zero rows. Why doesn’t this comparison match?”

When an object is difficult to recreate by hand, `dput()` writes the R code needed to reconstruct it:

```
dput(example_cases)
```

```
structure(list(county = "Fairfax ", cases = 30), class = "data.frame", row.names = c(NA, -1L))
```

Paste that output after an assignment such as `example_cases <-` in the example you share. It preserves details such as the trailing space. Inspect the output before sharing it, and use invented values for surveillance examples (Section 25.6).

The *reprex* package runs the example in a fresh session and formats its code and output for sharing. Copy the complete example to your clipboard, including the `library()` call and data creation, then run this in the console:

```
reprex::reprex()
```

Paste the resulting formatted example into your question. Check that it reproduces the same failure. If it instead reports that an object or function is missing, add the code needed to create that object or load the package.

If the failure disappears after restarting R, investigate session state: objects created interactively, packages loaded earlier, or startup settings. Include relevant versions and environment details with the example.

5.6 Asking for Help

You have read the message, traced the failure, and built a reprex that still misbehaves. The reprex gives the person answering a concrete failure to investigate. A good request has three parts: what you expected, what happened instead (the complete error, not a paraphrase), and the reprex that demonstrates it.

Where to take it depends on the problem:

- A colleague, first. Explaining what you expected can help you spot an incorrect assumption. Teams work better when asking early is normal; a common team norm is to timebox solo struggling to something like 30 minutes, then ask, leading with what you already tried (Chapter 22.).
- An AI assistant. Paste the complete error message and the reprex, and ask for a diagnosis before a fix, so you can judge whether the model actually understood the problem. The assistants covered in Chapter 8. are strong at exactly the errors in this chapter’s table, and agentic tools like Claude Code can run your reprex and iterate on it (Section 8.3). Verify the proposed explanation and fix.
- The package’s documentation and issue tracker, when the problem is package-specific. Read the relevant vignette, then search the GitHub issues, including closed ones. If you have found a genuine bug, include the reprex in your bug report.
- [Posit Community](#) or [Stack Overflow](#), for questions with no obvious owner. Both communities respond well to a clear reprex and poorly to screenshots of code and “it doesn’t work.”

Before posting to a public forum or issue tracker, check the example for sensitive information. Between the reprex discipline (fake data, no internal paths or credentials) and your agency’s disclosure rules, nothing about a real person or a real record should appear in a posted question.

5.7 Further Reading

Jenny Bryan’s `rstudio::conf(2020)` talk [Object of type ‘closure’ is not subsettable](#) demonstrates practical debugging in R, covering restart-and-rerun hygiene, `traceback()`, `browser()`, and the reprex mindset. The [debugging chapter of *Advanced R*](#) (Wickham) goes deeper on the tools, including `options(error = recover)` and debugging inside RMarkdown/Quarto documents. The [reprex package documentation](#) includes a “reprex do’s and don’ts” article worth reading once before your first public question.

6. Reproducible Reporting

Copying analysis results into a Word document creates extra work whenever the data or code changes. Each number must be found and updated, and missed edits can leave the executive summary inconsistent with the tables.

A reproducible report combines prose with code that generates tables, figures, and values in the text. After changing the data or analysis, render the document to update those results.

Quarto is the tool that makes this practical. A Quarto document (`.qmd`) is a plain-text file that combines a YAML header describing the output format, prose, and code blocks that execute and embed their results. Render it once and get an HTML page. Change one line and render again to get a PDF or a Word document from the same source.

 [Quarto documentation](#)

The [Quarto documentation](#) is the definitive reference for document options, output formats, and formatting syntax. This chapter covers the concepts and patterns that matter most for reproducible reporting; the Quarto docs are the place to go for comprehensive reference material.

6.1 Python and Other Languages

Quarto supports R, Python, Julia, and Observable JavaScript; the execution engine determines which languages can share a document. The rest of this chapter uses R in most examples because R is the primary language in many public health teams, but Python code blocks work exactly the same way: they execute when the document renders and their output appears in the report.

A Python block that requires no external packages:

```
import statistics

weekly_cases = [127, 143, 98, 112, 156, 134, 89]
mean_cases = statistics.mean(weekly_cases)
peak_week = max(weekly_cases)

print(f"Mean weekly cases: {mean_cases:.1f}")
print(f"Peak week: {peak_week}")
```

Output:

```
Mean weekly cases: 122.7
Peak week: 156
```

The `statistics` module is part of the Python standard library. In your `.qmd` file, use an executable cell starting with ````{python}` to run this code at render time.

 Note

R and Python can coexist in the same `.qmd` file. R objects and Python objects live in separate sessions, but the `reticulate` package allows data to flow between them: `py$my_python_object` is useful when a team's data cleaning lives in Python and their visualization in R.

6.2 Document Anatomy

A Quarto document has three parts:

YAML front matter: a header block delimited by `---` that controls the title, author, date, and output format.

Prose: written in Markdown. The [Quarto authoring guide](#) covers formatting. The short version: `**bold**`, `*italic*`, `# Heading 1`, `## Heading 2`, and `- list item` cover the majority of what you need.

Code blocks: fenced blocks with braces around the language (``{r}` or ``{python}`). When the document renders, each block executes in sequence and its output (printed values, tables, plots) is embedded in the output.

A minimal Quarto document:

```
---
title: "Influenza Surveillance Summary"
author: "Epidemiology Team"
date: today
format: html
---

## Overview

This report summarizes influenza activity for the current surveillance week.

```{r}
#| label: fig-trend
#| fig-cap: "Weekly influenza case counts"

library(ggplot2)
flu_data <- data.frame(week = 1:4, cases = c(23, 41, 30, 38))
ggplot(flu_data, aes(x = week, y = cases)) +
 geom_line()
```
```

Render from the terminal with `quarto render report.qmd`, or from within Positron or RStudio using the Render button. The output file (here, `report.html`) appears in the same directory.

Code block options like `#| label:` and `#| fig-cap:` are set with the `#|` prefix on lines at the top of a block. Common options include:

| Option | Effect |
|-----------------------------|--|
| <code>echo: false</code> | Run the code but hide the source in the output |
| <code>include: false</code> | Run the code but hide both source and output |
| <code>message: false</code> | Suppress package loading messages |
| <code>warning: false</code> | Suppress warnings |
| <code>fig-cap: "..."</code> | Add a caption below a figure |

A setup block at the top of the document, typically labeled `#| label: setup` with `#| include: false`, is the right place to load packages and read data so the rest of the document has access to them without displaying that machinery to readers.

Tip

R code cells inside `.qmd` files can be formatted with the Air formatter (see Section 20.2). Place your cursor in a cell and press `Cmd+K Cmd+F` (Mac) or `Ctrl+K Ctrl+F` (Windows/Linux) to format that cell. With format-on-save enabled for both `[r]` and `[quarto]` in your workspace settings, the active cell formats automatically when you save the document.

6.3 One Input, Many Output Formats

The same `.qmd` file can produce different output formats by changing the `format` key in the YAML header. This means you can share an HTML version with colleagues for day-to-day review, render a PDF for

formal submission, and produce a Word document for stakeholders who need to annotate, all from one source file.

```
format: html    # interactive HTML page
format: pdf     # PDF via LaTeX
format: docx    # Microsoft Word
format: typst   # PDF via Typst (no TeX installation required)
```

To render multiple formats at once, list them:

```
format:
  html: default
  docx: default
  pdf: default
```

Running `quarto render report.qmd --to all` produces all three.

The practical benefit of format independence is durability. A report that exists only as a finished Word document is locked to the moment it was created; any change requires re-opening it and editing manually. The `.qmd` file is the permanent, authoritative version. The Word document is just one of its outputs, and it can always be regenerated. For guidance on choosing which format best serves a given audience, see Section 28.3.

The same principle extends past Quarto's own formats. A single prepared dataset can feed a Quarto narrative report and a Tableau or Power BI dashboard at the same time, with one definition of what each metric means (Section 10.7).

Format choice also determines what accessibility work is required. HTML is the most accommodating target and the easiest to check; PDF is the most demanding. Chapter 7. covers both, along with the alt text and color choices that apply regardless of format.

Note

Some content behaves differently across formats. Interactive elements like `plotly` charts and `DT` tables work in HTML but fall back to static output in PDF and Word. If Word or PDF is a primary deliverable, test your render early so you aren't surprised by what doesn't translate.

6.4 Code Is the Report

Inline code inserts computed values directly into prose.

In a Quarto document, inline R expressions surrounded by backticks evaluate and inject their result directly into text. Given a data frame called `cases` and an object `pct_change` computed in a prior code block:

```
A total of `r nrow(cases)` cases were reported in `r params$county` County
during `r params$year`, a `r pct_change`% change from the prior year.
```

When rendered, this becomes prose like:

A total of 1,847 cases were reported in Fairfax County during 2024, a 12.3% change from the prior year.

Every number came from code. If the underlying data is revised and the document is re-rendered, every figure that depends on it updates everywhere it appears, including the executive summary, body, and footnotes. Reuse the same computed objects to keep these values consistent.

Tip

Use `format()` or `scales::comma()` to control how numbers look inline. A raw expression produces 1847; `scales::comma(nrow(cases))` produces 1,847. For percentages, `scales::percent(0.123, accuracy = 0.1)` gives 12.3%; this function expects a proportion, so divide an already-computed percentage by 100. This formatting belongs in the inline expression, not in a separate variable, so the display logic stays close to where it's used.

This approach also makes the analytical decision trail explicit. A number that appears in a report had to come from somewhere in the code. That traceability is exactly what public health accountability requires: when a number is questioned, you can show exactly how it was produced.

6.5 Paths and Project Structure

An absolute file path can prevent a colleague from running a report. A path like `/Users/maria/Documents/Projects/flu-report/data/flu-2024.csv` depends on Maria's directory structure.

The solution is to use paths relative to the project root and to structure the project so that the `.qmd` file and its data live in the same directory tree. As described in Chapter 2., a well-organized project directory has a clear, stable structure:

```
flu-report/
├── flu-report.qmd
├── data/
│   └── flu-2024.csv
└── output/
```

From `flu-report.qmd`, the data file is simply `"data/flu-2024.csv"`.

If `.qmd` files are nested in subdirectories, the `here` package resolves paths relative to the project root regardless of where the file sits:

```
library(here)
flu_data <- read_csv(here::here("data", "flu-2024.csv"))
```

`here::here()` finds the project root by looking for a `.Rproj`, `.git`, or similar marker file. It works whether code runs interactively from the console or from within a render.

⚠ Warning

Never use `setwd()` inside a `.qmd` file. It changes the working directory mid-execution and will almost certainly break for anyone else who renders the document. Relative paths and `here()` are the right tools.

6.6 Parameterized Reports

Parameterized reports take the reproducibility model one step further: instead of hardcoding a county, disease, or time period into the analysis, you declare those values as parameters and pass them in at render time. One template becomes many reports.

6.6.1 Declaring Parameters

Parameters are declared in the YAML front matter under the `params` key. Each parameter gets a name and a default value:

```
---
title: "County Disease Surveillance Report"
subtitle: "`r params$county` County -- `r params$year`"
date: today
format: html

params:
  county: "Fairfax"
  year: 2024
  disease: "Influenza"
---
```

The default values are what render when you run `quarto render` with no additional arguments. They also define the parameter type: a quoted string is a character, a bare number is numeric.

6.6.2 Using Parameters

Inside the document, parameters are available as `params$name` in code blocks and inline expressions:

```
#| label: setup
#| include: false

library(dplyr)
library(ggplot2)

# Filter to the selected county, year, and disease
county_data <- surveillance |>
  filter(
    county == params$county,
    year == params$year,
    disease == params$disease
  )

total_cases <- nrow(county_data)
pct_change <- round((total_cases / prior_year_cases - 1) * 100, 1)
```

Then in prose:

```
This report summarizes `r params$disease` surveillance data for
`r params$county` County during `r params$year`.

A total of `r scales::comma(total_cases)` cases were reported,
a `r pct_change`% change from the prior year.
```

Nothing in the body of the document is hardcoded to a specific county or year. The entire report adapts to whatever parameters are supplied at render time.

6.6.3 Rendering with Different Parameters

To render the report for a different county without editing the file, pass parameters on the command line:

```
quarto render report.qmd \
  -P county:"Arlington" \
  -P year:2024 \
  --output "arlington-2024.html"
```

To generate one report per county automatically, use `quarto_render()` from the `quarto` R package:

```
library(quarto)
library(purrr)

counties <- c("Fairfax", "Arlington", "Alexandria", "Loudoun")

walk(counties, function(county) {
  quarto_render(
    "report.qmd",
    execute_params = list(county = county, year = 2024),
    output_file = paste0(tolower(gsub(" ", "-", county)), "-2024.html")
  )
})
```

This loop uses the same template to render a separate HTML report for each county. For recurring reports (weekly surveillance summaries, monthly dashboards), this loop can be driven by a script that pulls the current list of counties or time periods from the data itself.

💡 Quarto parameters documentation

The [Quarto documentation on parameterized reports](#) covers additional options including Knitr and Jupyter parameter handling, and how to use parameters with Quarto Projects to generate many outputs in a single command.

6.7 Consistent Branding with `brand.yml`

When a team produces reports across multiple formats (HTML, Word, PDF), maintaining consistent colors, fonts, and logos is tedious if each format is configured separately. `brand.yml` is a specification for defining visual identity once and applying it across Quarto outputs and other Posit tools.

A `_brand.yml` file defines the elements of an organization's visual identity:

```
meta:
  name: "County Health Department"
  link: "https://health.example.gov"

color:
  palette:
    navy: "#1B4F72"
    teal: "#0E6655"
    light-gray: "#F2F3F4"
  foreground: navy
  background: white
  primary: navy

typography:
  fonts:
    - family: Source Sans Pro
      source: google
  base: Source Sans Pro
  headings: Source Sans Pro

logo:
  small: "assets/logo-small.png"
  medium: "assets/logo-medium.png"
```

Save this as `_brand.yml` in the project root. Quarto picks it up automatically at render time for supported formats.

Brand support varies by output format and template. HTML and Typst support many `_brand.yml` settings; check the [format support table](#) before relying on a particular setting. A shared brand file reduces repeated configuration for recurring reports.

i Documentation

- `brand.yml` specification: posit-dev.github.io/brand-yml: the full reference for all supported fields, including color, typography, logo, and defaults.
- Quarto + `brand.yml`: quarto.org/docs/authoring/brand.html: how Quarto consumes a `brand.yml` file and which output formats support it.

7. Accessibility

A report that cannot be read by a screen reader, a chart that disappears for someone with color blindness, a PDF with no usable reading order: each can prevent someone from using your work. Approximately 8% of men have some form of color vision deficiency. Screen reader users include blind and low-vision users but also people with dyslexia, motor impairments, and situational limitations. For most public health organizations, accessible outputs are a legal requirement.

Quarto handles much of the structural work automatically: semantic HTML headings, document metadata flowing into PDF fields, proper reading order. What it cannot do automatically is supply meaning that only you have: descriptive alt text for figures, color choices that work without color, and table structure a screen reader can navigate. Quarto 1.8 added HTML accessibility checks, and Quarto 1.9 added PDF standards support, so there are now tools to verify your work.

This chapter assumes familiarity with Quarto documents and code chunks. For background on Quarto's output formats and rendering workflow, see Chapter 6..

7.1 Accessibility Requirements

Digital accessibility is not optional for most public health organizations. The legal and policy framework has tightened considerably in recent years.

Section 508 of the Rehabilitation Act requires all U.S. federal agencies to make their electronic and information technology accessible to people with disabilities. The 2017 refresh (with compliance required from January 2018) aligned federal requirements with the Web Content Accessibility Guidelines (WCAG) 2.0 Level AA, which remains the formal legal floor. WCAG 2.1 and 2.2 are backwards compatible with 2.0, and agencies are encouraged to go beyond the 2.0 floor, but the Access Board's standards formally incorporate WCAG 2.0 by reference. CDC applies Section 508 as the baseline standard for all agency-developed and procured technologies, including reports and dashboards. CSTE updated its Special Emphasis Reports in December 2022 to adhere to federal 508 compliance regulations.

OMB Memorandum M-24-08 (December 2023) was the first major Section 508 guidance update in a decade. It requires federal agencies to designate a Section 508 program manager accountable for digital accessibility oversight, establish processes for tracking and resolving accessibility issues, and incorporate accessibility into procurement. If your team develops tools or reports for federal partners, M-24-08 shapes what they can accept.

ADA Title II extends accessibility requirements to state and local government entities. A 2024 DOJ final rule set WCAG 2.1 Level AA as the compliance standard. DOJ extended the compliance dates in an [April 2026 interim final rule](#): April 26, 2027 for entities with a total population of 50,000 or more, and April 26, 2028 for those with populations under 50,000 or any special district government. State and local health agencies are covered directly by this rule. Note that these deadlines are specific to Title II public entities; private institutions receiving federal funding have general accessibility obligations under Section 504, but the specific WCAG conformance deadlines from the 2024 rule do not apply to them.

WCAG 2.2 organizes accessibility requirements around four principles, often abbreviated **POUR**:

- **Perceivable**: Information must be presentable in ways users can perceive, including text alternatives for non-text content
- **Operable**: Interface components and navigation must be operable, including keyboard navigation and sufficient time to interact
- **Understandable**: Information and operation must be understandable, including readable text and predictable behavior
- **Robust**: Content must work with current and future assistive technologies

WCAG organizes success criteria into three conformance levels: A (minimum), AA (the standard required by Section 508's WCAG 2.0 floor and ADA Title II's WCAG 2.1 rule), and AAA (enhanced). Level AA covers the requirements most organizations face.

WCAG versions and backwards compatibility

WCAG 2.0, 2.1, and 2.2 are backwards compatible: meeting 2.2 automatically satisfies 2.1 and 2.0. The most practically relevant additions in 2.2 include minimum touch target sizes (2.5.8) and constraints on cognitive demands during login (3.3.8). For static reports and dashboards, the changes in 2.1 and 2.2 often concern interactive features.

7.2 Alt Text for Figures

Every figure in a published report needs a text alternative (alt text) that conveys the same information to someone who cannot see the image. Screen readers announce alt text when they encounter a figure; without it, users hear only “image” or nothing at all.

In Quarto, the `fig-alt` chunk option sets alt text separately from the figure caption (`fig-cap`). They serve different purposes:

- `fig-cap` is the visible caption beneath the figure, typically a brief label like “Weekly influenza case counts, 2024.”
- `fig-alt` is the invisible text alternative. It should describe what the figure shows: the trend, the comparison, the key value, for someone who cannot see it.

```
library(ggplot2)

ggplot(ili_data, aes(x = week, y = cases, color = region)) +
  geom_line(linewidth = 1) +
  labs(
    x = "Surveillance week",
    y = "ILI cases",
    color = "Region"
  ) +
  theme_minimal()
```

Describe the information the figure conveys, including relevant trends and comparisons. “A line chart” tells a screen reader user almost nothing. “Weekly ILI case counts peak in the Northeast in week 5 at 2,400 cases, then decline through summer” tells them what the chart is about.

A few guidelines:

- If the figure caption already conveys the key message, alt text can reinforce or expand on it, but they should not be identical
- If the underlying data is in a table elsewhere in the document, you can note that in the alt text: “See `@tbl-flu-counts` for the underlying data”
- Decorative images (logos, dividers) can use `fig-alt: ""`: an empty string signals to screen readers that the image carries no information

Generating alt text with Claude Code

Writing good alt text for many figures is tedious. Emil Hvitfeldt developed a Claude Code skill that automates it: given a `.qmd` file, it reads the code and drafts `fig-alt` descriptions. Check each description against the rendered figure and underlying data. See [this post on adding alt text to Quarto documents with a Claude Code skill](#) for installation instructions and examples.

Once installed, the skill is available as a slash command:

```
/write-alt-text @my-report.qmd
```

💡 Alt text travels to PDF

When you enable PDF accessibility standards (see Section 7.6), alt text set via `fig-alt` carries through to the tagged PDF. Writing it once in your code chunk covers both HTML and PDF output.

7.3 Color and Contrast

Approximately 8% of men and 0.5% of women have color vision deficiency, most commonly difficulty distinguishing red from green. Using color as the only way to distinguish categories means a meaningful share of your audience may not be able to read the chart.

Color should never be the *sole* differentiator. Combine it with shape, line type, or direct labels. When color is used, choose palettes designed for color vision deficiency.

The viridis palettes (available in the `viridis` package and built into `ggplot2` since version 3.0) are perceptually uniform and distinguishable under all common forms of color vision deficiency, including in grayscale. They work for both continuous and discrete data.

```
library(ggplot2)

ggplot(overdose_data, aes(x = year, y = deaths, fill = substance)) +
  geom_col(position = "dodge") +
  scale_fill_viridis_d(option = "D") +
  labs(
    x = "Year",
    y = "Deaths",
    fill = "Substance"
  ) +
  theme_minimal()
```

ColorBrewer palettes (available in the `RColorBrewer` package) were designed with accessibility in mind. The "Dark2" and "Set2" palettes are colorblind-safe for up to eight categories. For sequential data, "YlOrRd" and "Blues" are accessible and widely understood in public health contexts.

```
library(ggplot2)
library(RColorBrewer)

ggplot(vax_data, aes(x = coverage, y = county, fill = age_group)) +
  geom_col() +
  scale_fill_brewer(palette = "Dark2") +
  facet_wrap(~age_group) +
  labs(
    x = "Coverage (%)",
    y = NULL,
    fill = "Age group"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
```

Text contrast is also a consideration. WCAG 2.2 Level AA requires a contrast ratio of at least **4.5:1** for normal-weight text and **3:1** for large text (18pt or 14pt bold). Light gray text on a white background, common in default `ggplot2` axis labels and annotations, often fails this threshold. The `axe-core` checks described in Section 7.5 will flag contrast violations in HTML output.

⚠️ Test your palette

To check whether a chart is readable without color, print it in grayscale or run it through a color vision simulator. The `colorblindr` package provides `cvd_grid()`, which renders a `ggplot` in several color-deficiency simulations side by side. If categories are still distinguishable, the palette works.

7.4 Accessible Tables

Screen readers navigate HTML tables by announcing row and column headers. A table without proper headers, or with merged cells that break the row/column relationship, can be confusing or unusable. The same structural requirements apply to tagged PDFs.

- Every table should have a caption that identifies what it contains
- Column headers should be descriptive, not cryptic abbreviations
- Avoid spanning cells (merged rows or columns): they complicate the header/cell relationship that screen readers rely on
- Use tables only for tabular data, not for layout

The `gt` package produces well-structured HTML tables with semantic markup. `knitr::kable()` with `kableExtra` is also widely used. Both produce accessible output when used with captions and clear headers.

Table 7.1: Reported disease cases, deaths, and case-fatality rates by condition, 2023.

```
library(gt)
library(dplyr)

disease_summary |>
  gt() |>
  tab_header(
    title = "Reportable Disease Summary, 2023",
    subtitle = "Selected conditions, all jurisdictions"
  ) |>
  cols_label(
    condition = "Condition",
    cases = "Cases",
    deaths = "Deaths",
    cfr = "Case-fatality rate (%)"
  ) |>
  fmt_number(columns = c(cases, deaths), decimals = 0) |>
  fmt_number(columns = cfr, decimals = 1)
```

The `tab_header()` and `cols_label()` calls provide the semantic title and full column labels that screen readers announce. A table without these is harder to navigate for users relying on assistive technology.

7.5 HTML Accessibility Checks

Quarto 1.8 introduced integrated support for `axe-core`, an industry-standard accessibility testing engine. Enabling it causes Quarto to inject the `axe-core` script into rendered HTML, which runs when the page loads in a browser.

To enable axe checks, add the `axe` option to your document's HTML format:

```
format:
  html:
    axe: true
```

With this basic configuration, violations are reported as messages in the browser developer console (open with F12 or Cmd+Option+I). To see a visible report embedded on the page itself, use the `document` output mode:

```
format:
  html:
    axe:
      output: document
```

This is useful during development, but you would not want to publish it. The recommended pattern is a debug `project profile` that enables the document report only locally:

```
# _quarto-debug.yml (activated with: quarto render --profile debug)
format:
  html:
    axe:
      output: document
```

A third mode, `json`, produces machine-readable output compatible with browser automation tools like Puppeteer or Playwright:

```
format:
  html:
    axe:
      output: json
```

What axe-core checks

axe-core tests for a broad set of WCAG 2.x violations: missing alt text, insufficient color contrast, missing form labels, improper heading hierarchy, and missing landmark regions. It catches problems that are mechanically detectable. It cannot evaluate whether alt text is *meaningful*, only whether it is present. Human review is still necessary.

A minimal document that demonstrates a contrast violation:

```
---
title: "Accessibility check demo"
format:
  html:
    axe:
      output: document
---

The following text fails contrast requirements:
[This text is nearly invisible.]{style="color: #eee"}
```

When previewed in a browser, the axe report flags the contrast failure and identifies the affected element. Resolve violations before publishing, then remove the `axe` option (or restrict it to the debug profile).

7.6 PDF Accessibility

Quarto 1.9 introduced experimental support for PDF accessibility standards, building on new tagging capabilities in LaTeX and Typst. Accessible PDFs embed semantic structure (document title, heading hierarchy, reading order, image alt text) in a form that screen readers and other assistive technologies can use.

Two standards are supported:

- PDF/UA-1 (Universal Accessibility, first edition): the widely-used baseline standard, supported via Typst
- PDF/UA-2 (second edition): a newer standard built on PDF 2.0, supported via LaTeX

7.6.1 Enabling PDF Standards

Typst (UA-1):

```
format:
  typst:
    pdf-standard: ua-1
```

LaTeX (UA-2):

```
format:
  pdf:
    pdf-standard: ua-2
```

Two things are always required, regardless of renderer:

1. A `title` in the YAML front matter
2. `fig-alt` on every figure (see Section 7.2)

Quarto’s Markdown-based workflow handles most other requirements automatically: document metadata flows into the correct PDF fields, and heading structure from Markdown satisfies tagging requirements for reading order and document outline.

7.6.2 Validating with veraPDF

Quarto uses [veraPDF](#), an open-source PDF/A and PDF/UA validator, to check that output meets the specified standard. Install it with:

```
quarto install verapdf
```

Once installed, Quarto runs veraPDF automatically after rendering when `pdf-standard` is set. Validation results appear in the terminal output.

7.6.3 Known Limitations

LaTeX: Content placed in the margin (using `.column-margin` divs, `cap-location: margin`, or margin-positioned references) prevents UA-2 compliance. The underlying LaTeX packages involved in margin placement do not cooperate with PDF tagging. Avoid margin content if UA-2 compliance is required.

Typst: Book-format documents are not yet fully compliant due to structural issues in the underlying orange-book template. For standalone documents, Typst validates cleanly and catches UA-1 violations automatically.

💡 Typst vs. LaTeX for accessibility

Typst currently offers better practical PDF accessibility than LaTeX. It validates UA-1 cleanly on typical documents, requires no TeX installation, and is not affected by the margin-content restriction that blocks LaTeX’s UA-2 compliance. If accessible PDF output is a priority and you are starting a new document, Typst is worth considering. See Chapter 6. for guidance on Quarto output formats.

7.7 Authoring Workflow

Accessibility is easier to build in during authoring than to retrofit after the fact. A practical workflow for a Quarto report:

1. Write alt text for every figure using `fig-alt`. Do this when you write the code chunk, not at the end. Consider using the [Claude Code alt text skill](#) to generate first drafts.
2. Use an accessible palette: viridis or a colorblind-safe ColorBrewer palette. Verify by simulating color vision deficiency or viewing in grayscale.
3. Structure tables with captions and descriptive column labels via `gt` or `kable`.
4. Check HTML output with `axe: true` during local preview; resolve flagged violations before publishing.
5. Enable `pdf-standard` for PDF deliverables (`ua-1` for Typst, `ua-2` for LaTeX) and install veraPDF to validate.
6. Avoid margin content in LaTeX if UA-2 compliance is required.

i Accessibility benefits everyone

Accessible outputs are not only for users with disabilities. Alt text improves search engine indexing. Captions help non-native English speakers and people reading in noisy environments. High-contrast text is easier to read on mobile screens in bright light. Well-structured tables are easier for all users to navigate.

8. Coding with AI

AI tools for coding have matured rapidly, covering a spectrum from simple inline completions to fully autonomous agents that can read, write, and execute code across an entire project. This chapter provides a brief orientation to the tools most relevant to data scientists working in Positron.

It covers the tools only. What your agency permits, which data may go where, who reviews an AI-assisted analysis before it informs a decision, and what you have to record so the result can be defended later are all covered in Chapter 26. Read that chapter before pointing any of these tools at agency data.

8.1 Code Completion with GitHub Copilot

[GitHub Copilot](#) is a code completion service that suggests code as you type. Install the **GitHub Copilot** extension in Positron, and suggestions will appear inline in the editor and in notebooks. Press Tab to accept a suggestion or Escape to dismiss it.

Copilot draws on the surrounding code and comments to infer what you're trying to write. It works well for boilerplate, common patterns, and filling in function arguments you'd otherwise have to look up.

A GitHub Copilot subscription is required; a free tier is available for individual developers.

8.2 Posit Assistant

[Posit Assistant](#) is the default AI experience in Positron beginning with version 2026.07. It can use files and session context, including loaded data, plots, and console history, to help write and run analysis code. The context available to it depends on the connected session, tools, and permissions.

Start with a specific task and inspect the proposed changes. For a larger change, use `/plan` to work through the approach before implementation. Review tool permissions before allowing commands or access to external services. The [current documentation](#) describes planning, permissions, and project instructions.

To connect an approved model provider, open the Command Palette and run **Authentication: Configure Language Model Providers**. Authentication depends on the provider: some use an API key, others use browser sign-in or cloud credentials. Posit AI Pass is Posit's hosted service. Consult the [provider setup guide](#) for the account and authentication requirements.

Data handling also depends on that connection. With a third-party provider, prompts and selected context may be sent to that provider; with Posit AI Pass, Posit's service terms apply. Review [Posit's privacy and terms guidance](#) and the specific provider agreement. Do not infer privacy guarantees from the editor's name.

8.2.1 Older Positron Assistant and Databot instructions

Posit Assistant replaced both Positron Assistant and Databot in Positron 2026.07. Older tutorials describing Ask/Edit/Agent modes or a separate Databot installation apply to earlier releases. Use the current setup guide for new installations. The [archived Databot page](#) identifies the transition.

8.3 Claude Code

Claude Code is a terminal-based AI coding agent from Anthropic. Claude Code operates directly in a shell session with access governed by its permissions and configuration: it can read and write files, search the codebase, run commands, and coordinate changes across many files in a single session.

Install it (instructions below for MacOS, Linux, WSL, see [docs](#) for Windows).

```
curl -fsSL https://claude.ai/install.sh | bash
```

Then launch it from Positron's integrated terminal (**Terminal** > **New Terminal**):

claude

Claude Code is particularly effective for multi-file refactors, complex debugging sessions, and tasks that are tedious to coordinate manually. It is also well suited to translating and explaining legacy SAS programs, a workflow covered in Section 19.4.

8.4 Cost, Privacy, and Model Choice

8.4.1 Capability

Frontier models from OpenAI and Anthropic are substantially more capable than open-weight alternatives for complex reasoning and coding tasks. For most everyday coding work (fixing errors, writing functions, refactoring) a mid-tier model like Anthropic's Claude Sonnet is fast, inexpensive, and more than capable enough. Save the most powerful (and expensive) frontier models for genuinely hard problems.

Evaluate models on representative tasks: explaining an error, changing a function, or revising a report. Check correctness, the amount of review required, and response time. Model quality changes with releases and differs across tasks. Section 26.5 covers deployment decisions.

8.4.2 Cost

Compare the account's subscription, included usage, and overage rules. API services commonly meter input and output tokens; a token is a unit of text, not a fixed number of words. Agent sessions can make many calls and repeatedly send context. Measure a few realistic sessions before estimating a monthly budget, and set spending alerts where available.

8.4.3 Privacy

A common concern is whether proprietary code sent to a cloud AI service will be used to train future models. The short answer is: **not if you're paying for API access**.

OpenAI and Anthropic state that they do not use API inputs and outputs for model training by default. Opt-in sharing, feedback, retention, and product-specific terms require separate attention. Confirm which account and service the assistant uses; a paid consumer subscription is not the same as API access.

Note

See the authoritative statements directly:

- [Anthropic's commercial-product training policy](#)
- [OpenAI API data controls](#)

Open-weight models (such as Meta's Llama or Google's Gemma) can be run entirely on local hardware, which can support data-residency requirements if the entire deployment, including logs and connected tools, is configured accordingly. Self-hosting requires hardware and staff support; test the chosen model on the work it will actually perform. Evaluate the specific service, agreement, data, and configuration under your agency's policy (Chapter 26.).

9. Dashboards

Quarto dashboards use `format: dashboard` to create multi-panel HTML data displays that can be hosted as static files. They're a middle ground between a static report (a single linear document) and a Shiny app (interactive but requires a running R process). Dashboards are ideal for sharing summaries, KPIs, and plots that need to look polished without the overhead of maintaining a server. Chapter 28. covers how to decide whether a dashboard is the right output format for a given audience and finding.

This chapter covers the Quarto-native approach, where the whole dashboard is a text file under version control. Many public health teams publish through Tableau or Power BI instead, because the license is already paid and the stakeholders already know the interface. Chapter 10. covers that case: how to keep the analysis reproducible in R when using a BI platform.

The examples in this chapter are drawn from a real dashboard tracking U.S. drug overdose mortality data from the National Vital Statistics System (2015–2023).

Live demo and code

- Live demo: stephenturner.github.io/quarto-dashboard-demo
- Source code: github.com/stephenturner/quarto-dashboard-demo

Quarto dashboard documentation

The [Quarto documentation](#) is the definitive resource for dashboard features, options, and examples. This chapter provides a practical walkthrough of building a dashboard, but the [Quarto docs](#) are the place to go for comprehensive reference material.

9.1 Dashboard Anatomy

9.1.1 YAML Front Matter

A minimal Quarto dashboard needs only two things in the front matter: a title and `format: dashboard`.

```
---  
title: "My Dashboard"  
format: dashboard  
---
```

Additional options control the theme, layout behavior, and navigation elements:

```
---  
title: "Drug Overdose Mortality"  
subtitle: "National Vital Statistics System, 2015–2023"  
format:  
  dashboard:  
    scrolling: false  
    theme: cosmo  
date: today  
---
```

Setting `scrolling: false` (the default) sizes each row to fill the viewport height, producing a fixed-height layout. Setting `scrolling: true` allows the page to scroll freely, which is useful when the content is too tall to fit on one screen.

9.1.2 Rows and Columns

Quarto dashboards lay out content in rows by default. Level-2 headings (##) define rows, and each code cell within a row becomes a card that fills its share of the available width.

```
## Row {height=25%}

[cards here: share the row equally]

## Row {height=75%}

[cards here: taller row for plots]
```

The `{height=}` attribute (as a percentage or pixel value) controls how much vertical space each row takes. To switch to a column-based layout, use `orientation: columns` in the YAML and `{width=}` attributes on headings instead.

9.1.3 Cards

Every executable code cell in a dashboard automatically becomes a card. The cell's output (a plot, a table, text) fills the card. Use the `#| title:` cell option to give the card a header:

```
#| title: "Monthly Admissions"

ggplot(admissions, aes(x = month, y = n)) +
  geom_col()
```

9.1.4 Value Boxes

Value boxes are compact summary cards ideal for KPIs and headline numbers. They are created with the `#| content: valuebox` cell option. The cell should return a named list with `icon`, `color`, and `value` keys.

```
#| content: valuebox
#| title: "Total Patients"

list(
  icon = "person-fill",
  color = "primary",
  value = nrow(patients)
)
```

Icons are [Bootstrap Icons](#) names (without the `bi-` prefix). Colors can be Bootstrap theme colors ("primary", "success", "danger", "warning", "info") or any CSS color string.

9.1.5 Tabsets

Adding `.tabset` to a row makes each card within that row a tab instead of a side-by-side panel. Each cell's `#| title:` becomes the tab label.

```
## Row {.tabset}

```${r}
#| title: "Plot"

plot code
```

```${r}
#| title: "Data"

table code
```
```

Tabsets are useful when you have multiple views of the same data (e.g., a plot and the underlying table) and don't want them to compete for screen space.

9.2 Building a Dashboard

This section walks through building the overdose mortality dashboard step by step. All code blocks here are display-only; the executable versions live in the demo repository.

9.2.1 Project Structure

A Quarto dashboard lives in its own project directory with a `_quarto.yml` file and an `index.qmd`:

```
quarto-dashboard-demo/
├─ _quarto.yml
├─ index.qmd
└─ od-data-clean.csv
```

The `_quarto.yml` sets the project type and default format options:

```
project:
  type: default

format:
  dashboard:
    theme: cosmo
    scrolling: false
```

9.2.2 Loading Data

The setup chunk loads packages and reads the data. Compute any summary values you'll reuse across multiple cells here so they're available throughout the document.

```
#| label: setup
#| message: false
#| warning: false

library(readr)
library(dplyr)
library(ggplot2)
library(plotly)
library(DT)

od <- read_csv("od-data-clean.csv")

# Computed once, reused in value boxes and plots
total_state_years <- nrow(od)
year_min          <- min(od$year)
year_max          <- max(od$year)

top_state_recent <- od |>
  filter(year == max(year)) |>
  arrange(desc(pod)) |>
  slice(1)
```

The `pod` column is the proportion of all deaths attributable to overdose, the primary metric throughout the dashboard.

9.2.3 Value Boxes

Three value boxes occupy the first row. Each is a separate code cell with `#| content: valuebox` and a `#| title:.` Place them all under the same `## Row` heading so they share the row equally.

```
## Row {height=20%}
```

```
#| content: valuebox
#| title: "State-Year Observations"

list(
  icon = "table",
  color = "primary",
  value = scales::comma(total_state_years)
)
```

```
#| content: valuebox
#| title: "Years Covered"

list(
  icon = "calendar-range",
  color = "info",
  value = paste0(year_min, "-", year_max)
)
```

```
#| content: valuebox
#| title: "Highest OD % (Most Recent Year)"

list(
  icon = "graph-up-arrow",
  color = "danger",
  value = paste0(
    top_state_recent$state, ": ",
    scales::percent(top_state_recent$pod, accuracy = 0.1)
  )
)
```

9.2.4 Static plots (ggplot2)

A static ggplot2 plot renders as a PNG image inside a card. It works well for PDF exports and environments without JavaScript, but users can't hover for values or zoom in.

```
#| title: "Overdose % Over Time: Selected States"

highlight_states <- c("WV", "DC", "OH", "KY")

od |>
  filter(state %in% highlight_states) |>
  ggplot(aes(x = year, y = pod, color = state, group = state)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 0.1)) +
  scale_x_continuous(breaks = year_min:year_max) +
  labs(
    x = NULL,
    y = "Overdose deaths (% of all deaths)",
    color = "State"
  ) +
  theme_minimal(base_size = 13)
```

9.2.5 Interactive plots (plotly)

Wrapping a ggplot in `plotly::ggplotly()` converts it to an interactive plot with hover tooltips, zoom, and pan without changing the dashboard layout.

```
#| title: "Overdose % Over Time: Selected States"

p <- od |>
  filter(state %in% highlight_states) |>
  ggplot(aes(x = year, y = pod, color = state, group = state)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 0.1)) +
  scale_x_continuous(breaks = year_min:year_max) +
  labs(
    x = NULL,
    y = "Overdose deaths (% of all deaths)",
    color = "State"
  ) +
  theme_minimal(base_size = 13)

ggplotly(p)
```

Tip

Swapping a static ggplot2 plot for `ggplotly()` is a one-line change that requires no modifications to the dashboard layout. Add interactivity where it helps users explore data; keep static plots where simplicity or export quality matters more.

For finer control over tooltips, use the `text` aesthetic and pass `tooltip = "text"` to `ggplotly()`:

```
p2 <- bar_data |>
  ggplot(aes(
    x = reorder(state, total_od),
    y = total_od,
    fill = highlighted,
    text = paste0(state, ": ", scales::comma(total_od))
  )) +
  geom_col() +
  coord_flip()

ggplotly(p2, tooltip = "text")
```

9.2.6 Tables (DT)

`DT::datatable()` produces an interactive HTML table with built-in search, sorting, and pagination.

```
#| title: "Data"

od |>
  mutate(pod = scales::percent(pod, accuracy = 0.01)) |>
  rename(
    State = state,
    Year = year,
    `Total Deaths` = ndeath,
    `OD Deaths` = nod,
    `OD %` = pod
  ) |>
  DT::datatable(
    filter = "top",
```

```
options = list(pageLength = 10, scrollX = TRUE),
rownames = FALSE
)
```

`filter = "top"` adds a per-column search box above each column header, allowing users to filter by state or year without any server-side code.

9.3 Previewing and Rendering

From inside the project directory:

```
quarto preview index.qmd # live-reload preview in browser
quarto render index.qmd  # render to index.html
```

Note

`quarto preview` on a dashboard opens a standalone browser window showing the dashboard itself. If you also have the book open in another preview, they run independently; the dashboard preview is not embedded in the book.

9.4 Hosting on GitHub Pages

Because `format: dashboard` is incompatible with `format: html` (the book format used here), a dashboard cannot be a page in a Quarto book. The cleanest solution is to keep the dashboard in its own GitHub repository and deploy it separately.

9.4.1 Repository Setup

Create a new GitHub repository (e.g., `quarto-dashboard-demo`) and push the project files:

```
git init -b main
git add .
git commit -m "Initial dashboard"
git remote add origin git@github.com:stephenturner/quarto-dashboard-demo.git
git push -u origin main
```

9.4.2 Publishing

`quarto publish gh-pages` renders the project and pushes the output to the `gh-pages` branch. Configure GitHub Pages to serve the `gh-pages` branch in the repository settings.

```
quarto publish gh-pages
```

After the first publish, GitHub Pages will be available at <https://stephenturner.github.io/quarto-dashboard-demo>. Subsequent publishes update the live site.

Tip

Automate with GitHub Actions. Instead of running `quarto publish` manually on your local machine, you can set up a GitHub Actions workflow that re-renders and deploys on every push to `main`. The [Quarto documentation](#) provides a ready-to-use workflow YAML; copy it into `.github/workflows/publish.yml` in your repository and GitHub will handle the rest.

10. BI Platforms

Chapter 9. covers dashboards built in Quarto, where the entire product is a text file under version control. Many public health teams publish through Tableau or Power BI instead. The license is already paid, often at the enterprise level; program staff already know the interface; and in agencies where the BI platform is the sanctioned publishing route, a Quarto dashboard is not an accepted substitute regardless of its technical merits.

This chapter covers how to keep an analysis reproducible when the presentation layer is a proprietary application. The organizing principle is that shared analytical rules belong in version-controlled preparation code, and the BI tool reads the prepared data. Calculations that depend on interactive selections may remain in the BI model, with their definitions and checks under version control where the format supports it.

There is a longer-term argument for the same arrangement. R skills and reusable analysis code transfer across projects, including those using different BI platforms. The same holds for the work itself: an R pipeline survives an agency replacing Tableau with Power BI. Keep the analysis portable when deciding where to implement it.

10.1 Dividing the Work

Keep the R preparation code in version control, test it, and verify that it reruns from raw data. Use the BI application to display the prepared extract.

It can be convenient to recode categories, exclude records, or define rates inside a workbook. Those rules then need their own documentation, tests, and review. If another report uses the same rules, implement them in shared preparation code to avoid maintaining two versions.

For a specified set of filters, check that an independent calculation reproduces the dashboard's numbers. Include any BI measures in that reconciliation, with the same denominator, grouping, and suppression rules.

Tip

Choose the prepared table's level of detail to support the intended views. Keep the components needed for recalculation, including compatible rate numerators and denominators. Document each column's meaning.

10.2 Pre-Aggregating Before Import

BI dashboards rarely display individual records. They display counts by county and week, rates by age group, trends by month. Importing line-level data into a dashboard that only shows aggregates means the BI engine recomputes the same aggregation on every interaction, over a table orders of magnitude larger than the result.

Performance degrades as the source data grows, and the usual remedy is to aggregate in R before import. Consider a simulated line-level influenza case file:

```
set.seed(42)

counties <- c(
  "Fairfax",
  "Arlington",
  "Loudoun",
  "Prince William",
  "Alexandria",
  "Chesterfield",
```

```

"Henrico",
"Richmond",
"Roanoke",
"Virginia Beach"
)

n <- 250000

cases <- data.frame(
  case_id = seq_len(n),
  county = sample(counties, n, replace = TRUE),
  onset_date = as.Date("2021-01-03") + sample(0:1090, n, replace = TRUE),
  age_group = sample(
    c("0-17", "18-49", "50-64", "65+"),
    n,
    replace = TRUE
  ),
  hospitalized = sample(c(TRUE, FALSE), n, replace = TRUE, prob = c(0.08, 0.92))
)

nrow(cases)

```

```
[1] 250000
```

```
format(object.size(cases), units = "MB")
```

```
[1] "7.6 Mb"
```

If the dashboard displays weekly counts by county and age group, that is the aggregation to perform in R:

```

weekly <- cases |>
  mutate(week_start = as.Date(cut(onset_date, breaks = "week")) |>
    summarize(
      cases = n(),
      hospitalizations = sum(hospitalized),
      .by = c(county, week_start, age_group)
    ) |>
  arrange(county, week_start, age_group)

nrow(weekly)

```

```
[1] 6280
```

```
format(object.size(weekly), units = "MB")
```

```
[1] "0.2 Mb"
```

Two hundred fifty thousand rows become about six thousand, and the imported file shrinks accordingly. Nothing the dashboard displays is lost.

The cost is that the aggregation grain fixes what the dashboard can show. Drilling down to individual cases or slicing by an unretained variable is no longer possible. For a surveillance dashboard this constraint is usually appropriate, since individual records should not be exposed, but choose the aggregation level before building the dashboard.

Warning

Aggregation grain interacts with disclosure rules. A view that permits filtering to a single county, a single month, and a single age group can produce cells below the suppression threshold. Quarterly aggregation may reduce the number of small cells, but still requires disclosure checks. Choose the grain with the thresholds in Section 25.6 in mind, and check suppression before publishing and across all permitted filter combinations.

10.2.1 Default Aggregation

BI tools sum measures by default. Check whether that default makes sense for each measure.

Two cases recur. A measure that should be broken out by a dimension is totaled across it, so a view intended to show the current year's burden silently reports a multi-year total. And a rate or percentage is averaged when it should be recomputed from summed numerators and denominators; the mean of ten county rates is not the state rate.

For an interactive crude rate, retain compatible numerators and denominators and calculate the ratio for the selected group. Ten cases among 1,000 people and 45 among 9,000 yield different rates; a dashboard combining them should calculate $(10 + 45) / (1000 + 9000) * 100000$, or 550 per 100,000. Do not sum the displayed rates. Summing denominators is valid only when their units, periods, and populations support that operation; repeated population values across weeks, for example, cannot be added to produce an annual population.

Define and review this calculation in the BI model, or limit the dashboard to precomputed groups. Distinct-person counts across overlapping groups and age-adjusted rates also need their own aggregation rules. Reconcile representative filter combinations against independent calculations, including empty selections and zero denominators.

Suppression must apply to the released data and all supported views. Hiding a mark while leaving its count in a downloadable extract does not protect it. Check whether totals, adjacent periods, or combinations of filters reveal a suppressed count by subtraction. Apply complementary suppression, restrict the available views, or release a separately reviewed aggregate when needed (Section 25.6).

10.3 Publishing Programmatically

The manual publishing loop runs the R script, writes a file, opens the workbook, refreshes the data source, saves, and publishes. A missed refresh leaves the published dashboard out of date.

Both major platforms expose REST APIs that close the loop from code, using the `httr2` patterns covered in Chapter 18. Tableau Server and Tableau Cloud authenticate with a personal access token and return a credentials token for subsequent calls:

```
library(httr2)

server <- "https://tableau.health.state.gov"
api <- "3.19"

auth <- request(paste0(server, "/api/", api, "/auth/signin")) |>
  req_headers(Accept = "application/json") |>
  req_body_json(list(
    credentials = list(
      personalAccessTokenName = Sys.getenv("TABLEAU_PAT_NAME"),
      personalAccessTokenSecret = Sys.getenv("TABLEAU_PAT_SECRET"),
      site = list(contentUrl = "surveillance")
    )
  )) |>
  req_perform() |>
  resp_body_json()

token <- auth$credentials$token
site_id <- auth$credentials$site$id
```

For an existing published data source, set `datasource_id` to its Tableau ID and request a refresh. The server must be able to reach the source data; this request does not upload a CSV created on your laptop. A successful request starts an asynchronous job, whose completion must also be checked.

```
request(paste0(
  server,
  "/api/",
  api,
  "/sites/",
  site_id,
  "/datasources/",
  datasource_id,
  "/refresh"
)) |>
req_headers(`X-Tableau-Auth` = token, Accept = "application/json") |>
req_body_raw("<tsRequest></tsRequest>", type = "text/xml") |>
req_perform()
```

Power BI offers a [dataset refresh endpoint in its REST API](#), with authentication through Microsoft Entra ID. The specific endpoints change on the vendor's schedule, so the vendor documentation is the only reliable reference; the pattern is stable.

i Note

Before adopting a Tableau API wrapper such as `vvtableau`, check its current maintenance status and API coverage. Direct `httr2` calls are another option, but your team must maintain them when the vendor changes the API. Chapter 13. covers how to weigh this class of decision.

10.3.1 Obtaining Permissions

Access approval can take longer than implementation. Issuing a personal access token or provisioning a service account requires a permission change, and requests of this kind routinely stall.

Ask early and specifically (Section 24.4). What the pipeline needs is a token or service account scoped to publish to a single project or workspace, not administrator rights, and saying so plainly makes the request substantially easier to approve. Name the specific datasource or dataset. State that the alternative is a staff member manually refreshing an extract on a schedule, which carries both a labor cost and a reliability risk. If the agency requires a service account, request one: a token tied to an individual becomes a broken pipeline when that individual leaves (Section 23.8).

Read credentials from environment variables, following the pattern in Section 16.5.

10.4 Power BI and SharePoint

In agencies standardized on Microsoft, a common arrangement has the R pipeline write a prepared file to a SharePoint document library, a Power BI report read from it, and a scheduled refresh run nightly. Control who can modify the file.

A file in a SharePoint library is a shared, mutable object. Anyone with write access can open it, sort a column, and save. The scheduled refresh then runs against a file that no longer matches what the pipeline produced, and nothing announces the change.

Restrict write access. Write pipeline output to a location where only the pipeline writes, and set permissions accordingly. Never have the pipeline overwrite a file that a person is expected to edit. Include a generated-at timestamp in the extract, or a small metadata file, so readers can see when the pipeline last produced the data. Monitor the scheduled refresh (Section 11.6).

10.5 Version Control for BI Artifacts

Git can store binary files, but its line-based review tools work best with text. A `.pbix` file is a compressed archive containing the data model, the queries, the visuals, and a cached copy of the data. Committing it yields a history of opaque snapshots: you can restore a prior version, but you cannot see what changed between two of them or merge concurrent edits.

Power BI Desktop projects (.pbip) save report and semantic model definitions as text files suitable for Git. Check availability and limitations in your deployment before adopting this format.

Tableau supports both packaged and plain-text workbooks. A .twbx is a packaged workbook including the data and is opaque for the same reasons. A .twb is XML and will produce a diff, though the generated XML can make changes difficult to review. Preferring .twb with a separately managed data source is still worthwhile, because the diff is occasionally readable and the repository stays smaller.

For binary workbooks, keep a readable change history alongside the file. Version control the R preparation code properly, with meaningful commits, branches, and review, and document any measures implemented in the BI model so they can also be audited. Keep the workbook in the repository or a controlled location but record its changes separately, marking it binary in .gitattributes so Git stops attempting to diff it. Maintain a written change log for the workbook, in the repository, updated in the same commit that updates the workbook: two lines per change recording what changed in the view and why. The log is manual, and should accompany the platform's version history where available.

💡 Tip

Large BI files do not belong in the main repository history. A workbook committed weekly for two years produces a repository that is slow to clone. Git LFS addresses this, as does the simpler approach of storing workbooks in a versioned shared location and committing only the change log (Section 1.5).

10.6 Concurrent Editing

Because the artifact does not merge, two people editing the same binary workbook can overwrite each other's work. Agree on how to coordinate edits. Name one owner per workbook and record it in the team handbook (Section 23.2). Anyone else requiring a change requests it from the owner or takes explicit temporary ownership. Where the platform supports checkout or locking, use it.

Keeping preparation in the pipeline reduces the number of changes that require editing the workbook. Two analysts can then work simultaneously on the R side, with branches and review, and neither opens the binary file. For text-based project formats, use branches and review where supported, then open and test the merged project in the BI application; a clean text merge does not establish that the dashboard works.

10.7 Metric Definitions

A dashboard tooltip, a data dictionary in the repository, and a footnote in the quarterly report can diverge when they are maintained by different people at different times.

Define each metric once in a machine-readable file that the pipeline reads, and generate the rest from it. A small YAML or CSV in the repository with one row per metric, recording the name, definition, numerator, denominator, units, and any suppression rule, is sufficient:

```
dict <- readr::read_csv("metadata/metrics.csv")

# The pipeline uses it to label and annotate the extract
weekly_labeled <- weekly |>
  tidyr::pivot_longer(
    cols = c(cases, hospitalizations),
    names_to = "metric",
    values_to = "value"
  ) |>
  left_join(dict, by = "metric")

# And writes the same definitions out for the BI tool to read
readr::write_csv(dict, "extracts/metric_definitions.csv")
```

The BI tool imports the definitions file alongside the data and uses it for tooltips and axis labels. A changed definition changes in one place, in version control, with a commit message recording the reason. Section 21.4 covers the data dictionary as a documentation artifact; the addition here is using that dictionary as an input to the pipeline.

This also removes the apparent competition between BI and reproducible reporting. The same prepared extract and the same definitions can feed a Power BI report for interactive exploration and a Quarto

document for the quarterly narrative (Section 6.3): one input, several outputs, one definition of what a case is.

10.8 Checks Before Publishing

Confirm that the prepared data supports the permitted views, that rates and totals reconcile, and that disclosure rules hold across filters and downloads. Review both preparation code and BI measures. Coordinate edits according to the file format, and verify the refreshed published result after deployment.

11. Automation and Scheduling

Every Monday morning the respiratory surveillance report goes out. Someone opens the project, pulls the latest line list, renders the Quarto document, uploads the HTML, and emails the link to the program team. It takes twenty minutes if nothing goes wrong, longer if the data moved or a package updated over the weekend. It happens fifty-two times a year, and it depends entirely on a person remembering to do it.

The reporting chapter (Chapter 6.) showed how to build a report whose numbers update when the data changes, and the dashboards chapter (Chapter 9.) showed how to publish one to the web. This chapter is about the next step: making those things happen on a schedule, or in response to an event, without a person in the loop. The recurring weekly report becomes a job that runs at 6am whether or not anyone remembers, and the team finds out only if something breaks.

11.1 What to Automate (and What Not To)

Automation pays off when the same work happens the same way, repeatedly. A weekly surveillance render, a nightly data refresh from a public API (Chapter 18.), a validation check that should run every time a file lands: all of these are worth the up-front effort to set up, because the effort is amortized over dozens or hundreds of runs.

The same logic applies to publishing steps that sit downstream of the analysis. A dashboard extract that gets refreshed by hand every Monday is a scheduled job waiting to be written, and both Tableau and Power BI expose APIs for exactly this (Section 10.3).

It pays off poorly, or not at all, for one-off analyses, for work that requires a human decision partway through, and for exploratory work where the whole point is that you do not yet know what you are going to do. A useful test: would you run this code, in this same way, more than a handful of times? If yes, automate it. If no, the time spent automating is time not spent on the analysis.

An automated pipeline can repeat errors across many outputs. Automation presumes the underlying work is already validated (Chapter 3.) and reproducible (Chapter 15.). Test the pipeline before scheduling it and keep validation in each run.

11.2 Scheduling on Your Own Machine

The simplest form of automation is a saved script that the operating system runs on a schedule. The first requirement is that the script run without anyone clicking anything. Instead of opening the project and rendering interactively, put the render in an R script that can be run headlessly:

```
# render_report.R
quarto::quarto_render(
  input = "respiratory_report.qmd",
  execute_params = list(week = Sys.Date())
)
```

This is the same parameterized render from Section 6.6, just invoked from a script instead of by hand. From a terminal, `Rscript render_report.R` runs it start to finish with no interaction.

Once the work is a single command, the operating system can run it on a schedule. On macOS and Linux that scheduler is `cron`. A crontab entry is a schedule followed by a command; this one runs the render every Monday at 6am:

```
0 6 * * 1 cd /Users/you/projects/surveillance && /usr/local/bin/Rscript render_report.R
```

The five fields before the command are minute, hour, day-of-month, month, and day-of-week. The `cd` sets the working directory before R starts, so the relative report path and project startup files resolve correctly. Replace both paths for your machine; `command -v Rscript` in a terminal shows the R executable you normally use. Quote paths containing spaces. The `cronR` package can create and manage these entries

from R if you would rather not edit a crontab by hand. On Windows the equivalent is Task Scheduler, which the `taskscheduleR` package wraps in the same way.

Machine scheduling is easy to set up and good enough for plenty of internal work, but it has real limits. The machine has to be powered on and awake at 6am Monday, which rules out a laptop that goes home in a bag on Friday. Paths must be absolute, because a scheduled job does not start in your project directory or with your usual environment. If nobody receives an alert when the job fails, the report simply does not appear, and you find out when the program team asks where it is. The next two sections address each of these in turn.

11.3 Continuous Automation with GitHub Actions

For work whose code already lives on GitHub (Chapter 1.), GitHub Actions runs jobs on GitHub’s machines instead of yours. A workflow is a YAML file in the `.github/workflows/` directory of the repository. It describes when to run (a trigger) and what to do (a sequence of steps), and GitHub spins up a fresh virtual machine each time to carry it out. Because the machine is GitHub’s, it does not matter whether your laptop is open.

A workflow can be triggered by a push, by a manual button, or by a schedule. This one renders a Quarto report every Monday at 6am UTC, can also be run on demand, and publishes the result to GitHub Pages (Section 9.4):

```
# .github/workflows/render.yml
name: Render surveillance report

on:
  schedule:
    - cron: "0 6 * * 1"      # every Monday at 06:00 UTC
  workflow_dispatch:        # also allow manual trigger

jobs:
  render:
    runs-on: ubuntu-latest
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4

      - uses: quarto-dev/quarto-actions/setup@v2

      - uses: r-lib/actions/setup-r@v2

      - uses: r-lib/actions/setup-renv@v2

      - uses: quarto-dev/quarto-actions/publish@v2
        with:
          target: gh-pages
    env:
      GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

The steps read top to bottom: check out the repository, install Quarto and R, restore the exact package versions recorded by `renv` (Section 15.1), then render and publish. The maintained building blocks here are `r-lib/actions` for R setup and `quarto-actions` for rendering and publishing; the Quarto documentation on GitHub Actions, linked in Appendix A., is the reference for the publishing options.

i Note

The `cron` schedule in a GitHub Actions workflow runs on UTC, not your local time zone, and does not adjust for daylight saving. A “6am Monday” job will land at a different local hour depending on the season. For a surveillance report this rarely matters; when it does, schedule against UTC deliberately.

When a job needs a credential (an API token as in Section 18.5, a database password), that secret never goes in the workflow file or anywhere else in the repository. GitHub stores it under the repository’s

Settings > Secrets and variables, and the workflow reads it as an environment variable at run time (the `secrets.GITHUB_TOKEN` reference above is the built-in example). This is the same discipline as the `.Renv` pattern in Chapter 16.: credentials live outside version control, and code reads them from the environment.

11.4 Multi-Step Pipelines with targets

A single render is one step. Many real projects are a chain: read the raw extract, clean it, validate it, summarize, then render a report from the summary. When any one input changes, you want to rerun the steps that depend on it and skip the ones that do not. Rerunning everything every time wastes compute on a large dataset; rerunning the wrong subset can leave results based on stale intermediate files.

The `targets` package tracks dependencies between pipeline steps. Files must be declared as file targets, helper functions must be loaded, and report dependencies must be visible to the pipeline.

This small project reads invented case counts, checks and cleans them, and produces a weekly report. Create an empty project directory with the following files and two subdirectories, `R/` and `data/`. Each code block below gives the complete contents of one file:

```
surveillance/
  _quarto.yml
  _targets.R
  R/
    functions.R
  data/
    line_list.csv
    report.qmd
```

Install the Quarto command-line application and the R packages `targets`, `tarchetypes`, `quarto`, `knitr`, and `rmarkdown`. This example runs locally and does not publish anything.

Save the input as `data/line_list.csv`:

```
week,county,cases
1,Fairfax,23
1,Arlington,41
2,Fairfax,30
2,Arlington,38
```

In `R/functions.R`, define the helpers. These checks require the expected columns, at least one row, no missing values in those columns, and nonnegative numeric counts. A production pipeline should add checks for its reporting periods, allowed county names, and other input requirements (Chapter 3).

```
clean_line_list <- function(x) {
  needed <- c("week", "county", "cases")
  stopifnot(all(needed %in% names(x)), nrow(x) > 0L,
    !anyNA(x[needed]), is.numeric(x$cases), all(x$cases >= 0))
  x$county <- trimws(x$county)
  x
}

summarize_by_week <- function(x) {
  stats::aggregate(cases ~ week, data = x, FUN = sum)
}
```

The pipeline definition goes in `_targets.R`. Declaring the CSV with `format = "file"` lets `targets` detect changes to its contents. Loading the helper functions makes their code available for dependency tracking:

```
library(targets)
library(tarchetypes)
source("R/functions.R")

list(
  tar_target(raw_file, "data/line_list.csv", format = "file"),
  tar_target(raw, read.csv(raw_file)),
```

```
tar_target(clean, clean_line_list(raw)),
tar_target(summary, summarize_by_week(clean)),
tar_quarto(report, path = "report.qmd", quiet = FALSE)
)
```

The report reads the summary by name in an active code cell. Save this as `report.qmd`:

```
---
title: "Simulated weekly case summary"
format: html
---

This report uses invented counts for a pipeline demonstration.

```${r}
#| echo: false
summary <- targets::tar_read(summary)
knitr::kable(summary)
```
```

`tarchetypes::tar_quarto()` detects that target read and tracks the report's input and output files. Keep reads explicit in the report: dynamically constructed target names or reads hidden in helper functions may not be detected. Declare additional file dependencies when automatic detection does not cover them.

Save the following configuration as `_quarto.yml`. Keep Quarto freezing and caching disabled so they do not prevent execution when the pipeline requests a render:

```
project:
  type: default
format: html
execute:
  freeze: false
  cache: false
```

From the project root, run:

```
targets::tar_make()
```

Open `report.html`. It should show 64 cases for week 1 and 68 for week 2. Running the pipeline again without changing anything should skip all targets.

Before scheduling the pipeline, check which steps rerun after each kind of change. Changing a count in the CSV should rebuild the data preparation and report. Changing `summarize_by_week()` should rebuild the summary, and the report if the summary's value changed. Editing only the report text should rerender the report without repeating data preparation. `targets::tar_outdated()` lists targets that need updating, and `targets::tar_visnetwork()` displays the dependency graph. A dependency the pipeline does not know about cannot trigger a rebuild.

11.5 Running in a Reproducible Environment

A scheduled job that relies on whatever packages happen to be installed on the machine can fail after an unrecorded package update. Eventually a package updates, a function's default changes, and the Monday report renders differently or fails outright. And because nobody changed the code, the cause is hard to find.

The fix is to pin the environment so that the scheduled run matches the one you tested. Recording dependencies with `renv` (Section 15.1) lets the job restore the exact package versions it was built against; the GitHub Actions example above did this with the `setup-renv` step. A Docker image can also record the system libraries and tools the job needs (Section 15.2). Section 15.4 covers when each approach is worth the overhead. Record the environment used by the scheduled job.

11.6 Knowing When It Breaks

An automated job can fail without anyone noticing. A dashboard may continue displaying old results for weeks after a refresh fails. Alert the person responsible so they can investigate.

Build in failure signals from the start. Configure GitHub Actions notifications for the people responsible for the workflow and verify that they receive failure alerts. For higher-stakes pipelines, add a step that posts to a Slack or Teams channel on failure so the alert lands where the team actually looks.

Validation belongs inside the automated job in addition to interactive work. A `pointblank` stop-on-fail check (Section 3.2) placed before publishing can halt the run and trigger an alert when the data fails a required check.

11.7 Sensitive Data and Automation

Most of this chapter's cloud examples assume the data is safe to send to GitHub's machines. Much public health data is not. Protected health information, restricted-use files, and data governed by a use agreement generally must not run on GitHub-hosted runners or any other shared cloud infrastructure, because doing so moves the data outside the boundary where its handling is authorized.

This does not rule out automation; it changes where the automation runs. The options are to use self-hosted runners that sit inside the agency's secure network (while preventing protected data from appearing in uploaded logs, caches, or artifacts), to schedule the job on an approved on-premises server with cron or Task Scheduler, or to use whatever managed analytics platform the agency already operates. The mechanics from earlier in this chapter still apply; only the location changes.

The decision of where a scheduled job may run is a governance decision before it is a technical one. The terms of the relevant data use agreement, the HIPAA considerations, and the secure-environment requirements in Chapter 25. determine what is permissible, and provisioning a scheduled job on a shared server is an IT request that should start early (Section 24.4). Work that out before building the pipeline, not after, because a pipeline built for the wrong environment cannot simply be moved into the right one.

11.8 Further Reading

- The [targets user manual](#) [4] is the definitive guide to building reproducible pipelines in R, covering branching, cloud storage, and high-performance computing well beyond the minimal example here.
- The `r-lib/actions` and `quarto-actions` repositories document the GitHub Actions building blocks for R and Quarto, including ready-made example workflows.
- *Building Reproducible Analytical Pipelines with R*, linked in Appendix A., connects scheduling, pipelines, and reproducible environments into a single framework and goes substantially deeper on each.

12. Package Loading

Package load order affects which function R calls. Loading MASS after dplyr gives `MASS::select()` precedence over `dplyr::select()`. R prints a masking message, but it is easy to miss among many `library()` calls. A later call may then fail or use an unintended function.

This chapter is about the packages you have already decided to use: how they interact once loaded, and how to keep that interaction predictable. Chapter 13. covers the prior question of whether to take on a package at all, and how to assess its maintenance and requirements.

12.1 How R's search path works

When you call `library()`, R attaches the package to the *search path*, an ordered list of environments R searches when resolving a name. You can inspect it at any time:

```
search()
```

```
[1] ".GlobalEnv"      "package:stats"    "package:graphics"
[4] "package:grDevices" "package:utils"    "package:datasets"
[7] ".env"            "package:methods"  "Autoloads"
[10] "package:base"
```

After loading a few packages, the path grows:

```
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':
```

```
filter, lag
```

```
The following objects are masked from 'package:base':
```

```
intersect, setdiff, setequal, union
```

```
library(MASS)
```

```
Attaching package: 'MASS'
```

```
The following object is masked from 'package:dplyr':
```

```
select
```

`package:MASS` now sits above `package:dplyr` in the search path. When R encounters the name `select`, it finds `MASS::select` first and stops. R prints a message when this happens:

```
# Attaching package: 'MASS'
# The following object is masked from 'package:dplyr':
#   select
```

In a script that loads fifteen packages, these messages scroll past. The script runs, and there is no error to flag that `select()` is now going to the wrong package.

⚠ R's masking message is easy to miss

R prints a note at load time when a package masks a function from an earlier-loaded one, but in a long block of `library()` calls the messages are easily overlooked. The earlier function remains available through its package namespace, but the later one takes precedence for an unqualified call. The `conflicted` package, covered in Section 12.4, changes this: unresolved function-name conflicts produce an error.

12.2 Load only what you need

Every `library()` call extends the search path. Remove unused package calls to reduce opportunities for masking bugs.

Load a package with `library()` only if you call its functions throughout the script using bare names. If a package appears once or twice, use `::` instead (see Section 12.3).

Loaded by habit:

```
library(tidyverse)
library(lubridate)
library(janitor)
library(readxl)
library(openxlsx)
library(scales)
library(zoo)
library(forecast)
library(MASS)
library(car)
```

Trimmed to what the script uses:

```
library(dplyr)
library(ggplot2)
library(lubridate)
library(readr)
```

The shorter list documents dependencies. A reader can see what the script uses and trust the list is intentional.

💡 Loading tidyverse

`library(tidyverse)` loads `ggplot2`, `dplyr`, `tidyr`, `readr`, `purrr`, `tibble`, `stringr`, `forcats`, and `lubridate` in a single call. That is a reasonable choice when you use several of those packages. For a script that only needs `dplyr` and `ggplot2`, loading individual packages is more explicit about what you depend on and puts fewer packages on the search path.

You'll get the following message when you load tidyverse:

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.0      ✓ readr      2.1.6
✓ forcats    1.0.1      ✓ stringr    1.6.0
✓ ggplot2    4.0.1      ✓ tibble     3.3.0
✓ lubridate  1.9.4      ✓ tidyr      1.3.1
✓ purrr      1.2.0
— Conflicts ————— tidyverse_conflicts() —
* dplyr::filter() masks stats::filter()
* dplyr::lag()     masks stats::lag()
i Use the conflicted package to force all conflicts to become errors
```

12.3 Use `::` for one-off functions

The `::` operator calls a function directly from a specific package without attaching it to the search path, so no masking occurs.

```
# Reading a single Excel file
data <- readxl::read_excel("surveillance_data.xlsx")

# Reading a large CSV with data.table's fast parser
raw <- data.table::fread("large_file.csv")

# Fitting a distribution without loading MASS
fit <- MASS::fitdistr(x, "normal")
```

`::` also documents function origin at the call site. `readxl::read_excel()` names its source; `read_excel()` does not.

When to use `::` versus `library()`:

- Use `library()` when you call functions from a package throughout the script. Loading `dplyr` makes sense if you are writing a dozen `filter()` and `mutate()` calls.
- Use `::` when you need one or two functions from a package and do not use it elsewhere.

i `::` works even without `library()`

You do not need to call `library()` before using `::`. As long as a package is installed, `readxl::read_excel()` will find and call it. You get the function without attaching the package or affecting the search path.

12.4 The conflicted package

The `conflicted` package [5] changes the default behavior: unresolved function-name conflicts produce an error so you can choose the intended function.

Install from CRAN:

```
install.packages("conflicted")
```

Without `conflicted`, load order silently determines which `select()` runs:

```
library(dplyr)
library(MASS)

# Silently uses MASS::select(), not dplyr::select()
result <- mtcars |> select(mpg, cyl)
```

```
Error in `select()` :
! unused arguments (mpg, cyl)
```

With `conflicted` loaded first, the same call throws an informative error:

```
library(conflicted)
library(dplyr)
library(MASS)

result <- mtcars |> select(mpg, cyl)
```

```
Error:
! [conflicted] select found in 2 packages.
Either pick the one you want with `::`:
• MASS::select
• dplyr::select
```

```
Or declare a preference with `conflicts_prefer()`:  
• `conflicts_prefer(MASS::select)`  
• `conflicts_prefer(dplyr::select)`
```

`conflicted` asks you to choose which `select()` to use.

Declare your preferences with `conflicts_prefer()`, once at the top of the script alongside your `library()` calls:

```
library(conflicted)  
library(dplyr)  
library(MASS)  
  
conflicts_prefer(dplyr::select)
```

```
[conflicted] Will prefer dplyr::select over any other package.
```

```
conflicts_prefer(dplyr::filter)
```

```
[conflicted] Will prefer dplyr::filter over any other package.
```

```
# Now select() and filter() unambiguously mean dplyr  
result <- mtcars |> select(mpg, cyl)
```

Use `conflict_scout()` to see every conflict in your session before any function calls:

```
conflict_scout()
```

```
3 conflicts
```

```
• `filter()`: dplyr
```

```
• `lag()`: dplyr and stats
```

```
• `select()`: dplyr
```

Run it after loading your packages to see what needs a preference declared.

💡 conflicted works well alongside tidyverse

Load `conflicted` before `tidyverse`, then declare preferences for the most common conflicts:

```
library(conflicted)
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ forcats 1.0.1      ✓ readr  2.2.0
✓ ggplot2 4.0.3      ✓ stringr 1.6.0
✓ lubridate 1.9.5    ✓ tibble  3.3.1
✓ purrr 1.2.2        ✓ tidyr   1.3.2
```

```
conflicts_prefer(dplyr::filter)
```

```
[conflicted] Removing existing preference.
[conflicted] Will prefer dplyr::filter over any other package.
```

```
conflicts_prefer(dplyr::select)
```

```
[conflicted] Removing existing preference.
[conflicted] Will prefer dplyr::select over any other package.
```

```
conflicts_prefer(dplyr::lag)
```

```
[conflicted] Will prefer dplyr::lag over any other package.
```

After these declarations, `filter()`, `select()`, and `lag()` unambiguously refer to `dplyr`. Other unresolved conflicts produce errors.

12.5 Team implications

Package load order affects reproducibility across the team. The same script can produce different results on different machines when analysts have different packages installed or load them in a different order. `dplyr::select()` always means `dplyr::select()`, but `select()` depends on who loaded what and when.

Code review (see Chapter 27.) is a natural checkpoint for package hygiene. Reviewers should flag unnecessary `library()` calls and bare function names that commonly conflict. Both are easier to catch in review than when writing.

`renv` (see Section 15.1) ensures everyone is using the same package versions, but it does not determine which `select()` runs when both `dplyr` and `MASS` are loaded. `renv` and `conflicted` are complementary: one locks down versions, the other makes conflict resolution explicit.

The package block at the top of a shared script documents what the script depends on. A bloated list loaded by habit, combined with silent masking, means different analysts running the same script may not be running the same code.

13. Managing Package Dependencies

Chapter 12. covers what happens when you call `library()`: the search path, masking, and how to keep a script's package list accurate. Chapter 15. covers pinning versions with `renv` and freezing an entire environment into a container. Before loading or pinning a package, decide whether the project needs it.

Each dependency adds code maintained outside your project. Most of those commitments are worth making; R packages provide tested implementations of common tasks such as reading CSV files. The ones that go badly tend to fail at inconvenient times, in an automated job, on a Linux server, months after the person who introduced the dependency has moved on.

13.1 The Cost of a Dependency

Consider installation, system requirements, compatibility, image size, and maintenance.

Install time. Packages install from binaries in seconds on macOS and Windows. On Linux servers and inside Docker builds they usually compile from source. A dependency tree that installs in ten seconds on a laptop can take several minutes in a container, and uncached rebuilds repeat that work.

System libraries. Some R packages wrap C or C++ libraries that must be present on the machine. Those libraries must also be installed, which may require help from IT. Missing system libraries can prevent installation on another machine.

Compatibility. Changes in indirect dependencies can affect your results, even if the package you chose has not changed.

Container size. Images grow with the dependency tree. A 400 MB image pulls faster and costs less to store than a 4 GB image, and in a scheduled pipeline that pull recurs (Section 11.5).

Maintenance attention. Each dependency is something to update, something whose release notes may matter, and something to explain to the next analyst.

13.2 Measuring Dependency Weight

The recursive tree can be counted before committing to anything. `tools::package_dependencies()` ships with base R:

```
db <- available.packages()

deps <- tools::package_dependencies(
  "dplyr",
  db = db,
  recursive = TRUE,
  which = c("Depends", "Imports", "LinkingTo")
)

base_recommended <- rownames(installed.packages(
  priority = c("base", "recommended")
))
length(setdiff(unique(unlist(deps)), base_recommended))
```

`pak::pkg_deps_tree()` presents the same information as a tree, which identifies *which* package contributes the weight:

```
pak::pkg_deps_tree("httr2")
```

Record the package version and repository snapshot or retrieval date with a dependency count. The code above explicitly excludes installed base and recommended packages. Counts alone do not establish whether a dependency is appropriate: consider what it provides and what maintaining an alternative would require.

Package counts omit system requirements. Installing `sf` also requires geospatial libraries such as GDAL, GEOS, and PROJ, whose versions and compatibility affect a container build. Measure installation time in the intended environment.

 Tip

Check the `SystemRequirements` field on a package’s CRAN page before adding anything geospatial, anything that renders images, anything that connects to a database, or anything built on Rcpp. It lists dependencies that an R package manager may not install for you.

Dependency weight matters most where builds are automated and least where a person installs once. Convenience may justify more dependencies during exploration. For a containerized job, check installation time and system requirements before adding a package.

13.3 Meta-Packages

`tidyverse` is a meta-package: installing it brings in a broad dependency tree, and `library(tidyverse)` attaches its core packages. Section 12.2 presents the search-path argument for naming packages individually; the dependency argument points the same direction.

Meta-packages are appropriate for interactive work, teaching, exploratory analysis, and any script that genuinely uses most of what they load. The convenience is real and the cost is not noticeable in those settings.

Naming individual packages is appropriate for production pipelines, containers, packages under development, and anything another team must install. A header reading `library(dplyr)`, `library(readr)`, `library(ggplot2)` states its requirements precisely. `library(tidyverse)` does not show which attached packages the script uses. The same reasoning applies to `tidymodels`: if a modeling script uses `parsnip` and `recipes`, name them.

13.4 Vetting a Package

Before adopting a package, review its maintenance and compatibility.

 Tip

For a deep dive on this, see my recent paper, “Ten quick tips to SNIFF out sustainable and secure scientific software” [6].

Date of last release. A long interval since the last release can reflect stability or inactivity. Read recent issues and maintainer responses for context.

Issue response. Sort the issue tracker by most recent. Maintainer replies on the last several issues indicate an active package. Issues open for eighteen months without comment indicate otherwise.

Reverse dependencies. A package that a hundred other CRAN packages depend on will not disappear without many people noticing. CRAN lists these on each package page.

CRAN check status. The package page reports check results across platforms. Persistent failures on a platform you use are a warning.

Dependency version constraints. Check for exact-version requirements or upper bounds that conflict with other packages you need. A minimum version requirement alone does not pin a package to that version.

Maintainer. Many R packages have a single maintainer. Consider who could maintain the code if that person becomes unavailable. A single-maintainer utility package is unremarkable; a single-maintainer package at the center of a surveillance pipeline warrants a contingency plan.

Independent upstream changes. A package wrapping a vendor’s REST API depends on both the maintainer’s attention and the vendor’s release schedule, and the vendor has no obligation to the package.

⚠ Warning

The last two criteria compound. A wrapper package that is on CRAN, currently functional, singly maintained, not recently updated, and pinned to an older dependency has no present defect. Its failure mode is that the vendor ships an API change and no one updates the wrapper, at which point a pipeline that ran reliably for a year stops. Using `httr2` directly (Section 10.3) gives your team responsibility for maintaining the API calls.

None of these findings is automatically disqualifying. Use the review to identify maintenance needs before adopting the package.

13.5 Auditing an Existing Project

You may inherit a project with an existing dependency list. Before deciding what to add, establish what is already present.

`renv::dependencies()` scans a project directory for every `library()`, `require()`, and `::` call and reports which packages the project uses and which file each reference came from:

```
renv::dependencies()
```

Look for these cases. Packages appearing exactly once are candidates for `::` in place of `library()`, or for removal. Investigate unfamiliar packages before removing them; they may still be needed. Packages appearing in a script but not in the environment documentation are how a pipeline that runs today fails on a new machine.

A first pass over an inherited pipeline commonly removes a substantial fraction of the `library()` calls without changing any result. Doing this before containerizing is worthwhile, since removing unused dependencies can shorten uncached builds.

13.6 Lifecycle Stages

Tidyverse packages label functions with lifecycle stages: stable, experimental, superseded, and deprecated. The badges appear in the documentation and carry specific meanings.

Stable functions are safe to build on. Superseded functions continue to work indefinitely and receive no new features, so you can plan migration around the project's needs. Experimental functions may still change their interface; they are reasonable in analysis code you control and worth avoiding in a package other people install. Deprecated functions will eventually error, so plan migration before the function is removed.

`tidyr::separate()` is superseded, not deprecated. It remains available and receives critical bug fixes. If you adopt `separate_wider_delim()` or a related replacement, ensure everyone has a compatible version of `tidyr`.

```
packageVersion("dplyr")
```

```
[1] '1.2.1'
```

Printing the version in a setup chunk helps diagnose differences between installations. It does not enforce a version requirement; use an explicit check or restore the project lockfile for that.

13.7 Version Consistency Across a Team

After choosing dependencies, record and test updates in the project environment. Section 15.1 is the main reference for lockfiles, restoration, and coordinated updates. Function-name conflicts are a separate issue covered in Section 12.4.

13.8 Writing Your Own Instead

Occasionally the appropriate choice is no dependency.

A twenty-line function you control, understand, and test (Chapter 4.), living in your own repository, is preferable to a package with forty dependencies that provides the same capability plus ninety others. This applies most clearly to small utilities: a parser for an agency’s file naming convention, a function implementing local suppression rules, a wrapper around a single API endpoint.

Use established implementations for tasks with complex correctness requirements. Date and time zone arithmetic, statistical methods, character encoding, and HTTP all reward using an established implementation, because writing your own can introduce errors that are difficult to detect.

A rough test: code you could write correctly in an afternoon and explain to a colleague in five minutes is often better written than depended upon. Code requiring you to consult a specification first is better taken as a dependency.

Vendoring is the intermediate option, meaning copying a specific function into your own codebase with attribution and a comment recording its origin and date. This suits the case where a package offers one needed function and forty unneeded dependencies, and where the license permits it. Check the license, retain the attribution, and recognize that you have assumed maintenance of the copy.

13.9 Organizing Your Own Packages

Once a team accumulates shared helper functions, the appropriate container is a package (Chapter 14.), which provides namespacing, documentation, tests, and a single versioned copy of the shared functions.

The question that follows is how many packages, and the organizing principle matters more than the count. Consider organizing shared utilities by function, such as plotting or standardization. Separate packages for plotting conventions and for standardization routines will each be used across many projects and maintained coherently. Packages split by originating data source will each accumulate a little plotting, a little cleaning, and a little reporting, duplicated across all of them. The tidyverse is organized on this principle: `dplyr` transforms, `ggplot2` plots, `readr` reads.

Separating independent functions lets developers maintain plotting conventions in one place and gives each package a clear scope. Getting it wrong produces a set of packages that depend on one another and must be updated together, which is a single package with additional overhead.

A new data source should mean a new function in an existing package, or at most a thin ingest package depending on the shared ones, without duplicating the shared utilities.

13.10 Further Reading

- The [dependencies chapter of *R Packages*](#) discusses how to evaluate the costs and benefits of a dependency.
- The [lifecycle documentation](#) explains each stage badge and what it commits the maintainer to.
- *R Packages* [7] covers the `DESCRIPTION` file, the distinction between `Imports` and `Suggests`, and keeping a package’s own dependency footprint small.
- Teams migrating from SAS need to learn how R manages dependencies, because the two systems distribute extensions differently. The habit that transfers poorly is loading everything by default; the habit to build is naming what you use and qualifying with `pkg::fn` where conflicts exist (Section 12.3, Section 19.8).

14. Package Development

Most R users start writing functions to avoid repeating themselves. The same data cleaning steps appear in three scripts, so they get pulled into a shared helper. That helper gets sourced into a fourth script. A colleague asks for it and it gets emailed. Then it breaks on their machine because they're missing some dependency. A package provides a standard way to distribute these shared functions.

My general rule is: If you do something more than once, write a function. If you write more than one function, make it a package.

An R package is the right unit of sharing for reusable code. It combines your functions with documentation, a test suite, versioning, and a clear installation path.

This chapter is a brief introduction to R package development. It is not a comprehensive guide, but it will give you the basics and point you to resources for learning more.

14.1 Why build a package?

Copied helper functions are easy to lose track of. If three reports contain their own copy of a county-cleaning function, a correction in one report may never reach the others. With a shared package, the team maintains one implementation, reviews the correction once, and releases an update that each project can adopt.

Writing function documentation makes you decide what callers can expect. For a rate calculation, that includes the denominator units, the treatment of missing values, and the result when the population is zero. Those decisions help a colleague use the function without reading its implementation or asking its author. Package help pages make the answers available through `?your_function`, and R CMD check warns about undocumented exported objects. Section 21.3 covers writing that documentation alongside the code.

Declaring dependencies in `DESCRIPTION` records which packages your code needs. Installation tools use those declarations to install required R packages from the configured sources. A colleague no longer has to reconstruct the set of packages you happened to have loaded when you wrote the helper. The declarations also give reviewers a place to inspect the package's dependencies and assess the cost of adding another one (Chapter 13.).

A package gives the team a repeatable installation procedure. You can distribute it through an internal repository or a Git repository that colleagues can access; publishing on CRAN is optional. Installation puts the functions and their help pages in the expected locations, so each analysis can load the package without setting paths to a shared helper script. New team members can follow the same setup instructions as everyone else.

Shared functions need tests because a change can affect several analyses. A suppression helper, for example, should have tests for counts just below and at the threshold, as well as zero and missing values. Keeping those tests with the package lets you run them before releasing a change. A colleague fixing one case can check whether the other documented cases still work. Chapter 4. covers choosing useful tests and adding regression tests after a bug.

Versioned releases let projects adopt changes deliberately. A report already under review can continue using the version it was checked with while another project tests an update. Record the package version in each project's `renv` lockfile and retain access to its source so the environment can be restored (Section 15.1). Declaring dependencies tells users what the package needs; the lockfile records the versions a particular analysis used.

A package still needs someone responsible for reviewing changes, maintaining documentation, and releasing updates. Start with functions that several projects already share, such as common validation checks or reporting helpers. Section 13.9 discusses how to divide that code into packages with a manageable scope.

14.2 The package development workflow

The core loop of package development (edit code, load and run it, run tests, update documentation, check) is well captured by the Posit package development cheat sheet at <https://rstudio.github.io/cheatsheets/html/package-development.html>.

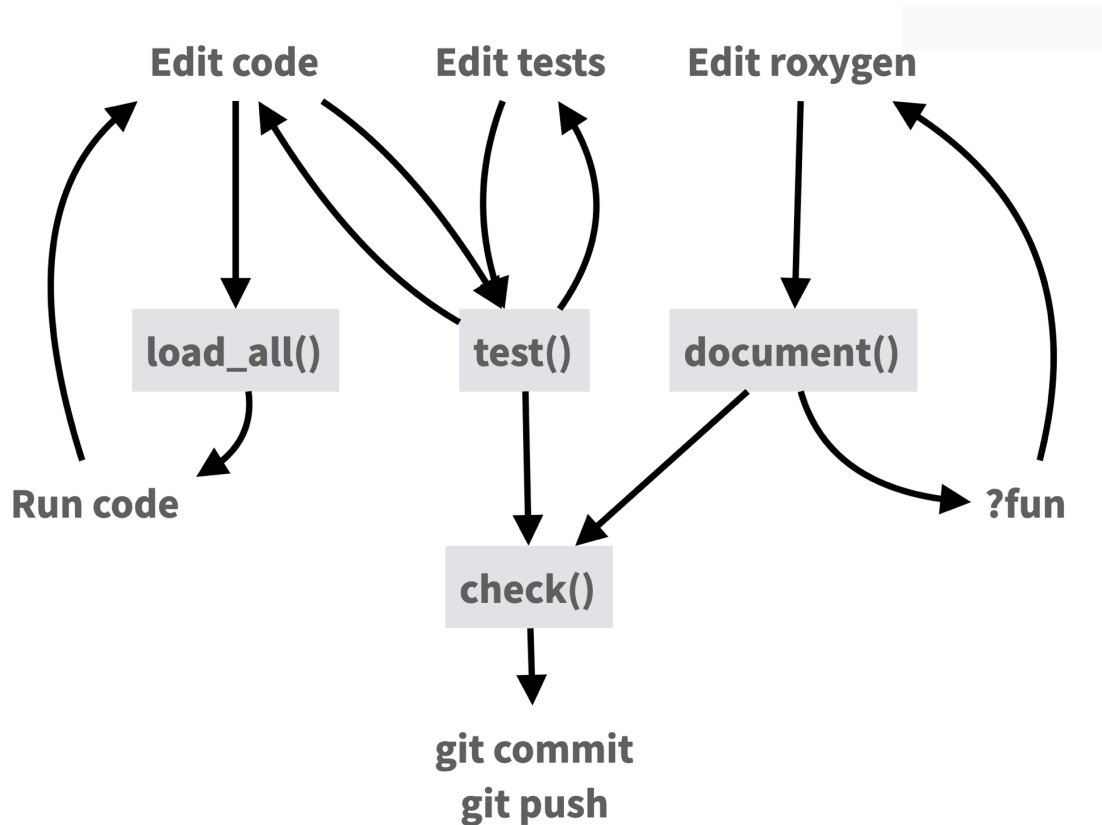


Figure 14.1: The package development workflow. Source: [Posit Package Development Cheat Sheet](#), CC BY 4.0.

Use the resources below for a complete walkthrough.

14.3 Resources

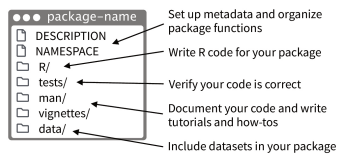
The definitive reference is [R Packages \(2nd ed.\)](#) by Hadley Wickham and Jennifer Bryan [7], available free online. It covers package structure, function documentation, testing, data, and the publication process.

The [Posit Package Development cheat sheet](#) is a quick reference for the devtools and usethis commands used throughout development.

Package Development : : CHEATSHEET

Package Structure

A package is a convention for organizing files into directories. This cheat sheet shows how to work with the 7 most common parts of an R package:



There are multiple packages useful to package development, including **usethis** which handily automates many of the more repetitive tasks. Install and load **devtools**, which wraps together several of these packages to access everything in one step.

Getting Started

Once per machine:

- Get set up with **use_devtools()** so **devtools** is always loaded in interactive R sessions

```
if (interactive()) {
  require("devtools", quietly = TRUE)
  # automatically attaches usethis
}
```

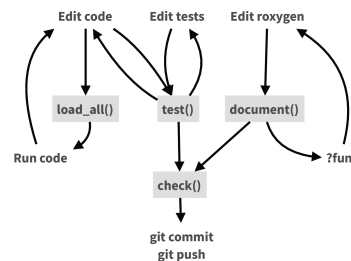
- **create_github_token()** — Set up GitHub credentials
- **git_vaccinate()** — Ignore common special files

Once per package:

- **create_package()** — Create a project with package scaffolding
- **use_git()** — Activate git
- **use_github()** — Connect to GitHub
- **use_github_action()** — Set up automated package checks

Having problems with git? Get a situation report with **git_sitrep()**.

Workflow



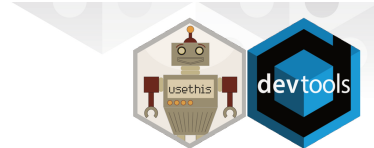
- **load_all()** (Ctrl/Cmd + Shift + L) — Load code
- **document()** (Ctrl/Cmd + Shift + D) — Rebuild docs and NAMESPACE
- **test()** (Ctrl/Cmd + Shift + T) — Run tests
- **check()** (Ctrl/Cmd + Shift + E) — Check complete package

R/

All of the R code in your package goes in **R/**. A package with just an **R/** directory is still a very useful package.

- ✓ Create a new package project with **create_package("path/to/name")**.
- ✓ Create R files with **use_r("file-name")**.

- Follow the tidyverse style guide at style.tidyverse.org
- Click on a function and press **F2** to go to its definition
- Find a function or file with **Ctrl + .**



DESCRIPTION

The **DESCRIPTION** file describes your work, sets up how your package will work with other packages, and applies a license.

- ✓ Pick a license with **use_mit_license()**, **use_gpl3_license()**, **use_proprietary_license()**.
- ✓ Add packages that you need with **use_package()**.

Import packages that your package requires to work, R will install them when it installs your package.

use_package(x, type = "imports")

Suggest packages that developers of your package need. Users can install or not, as they like.

use_package(x, type = "suggests")

NAMESPACE

The **NAMESPACE** file helps you make your package self-contained: it won't interfere with other packages, and other packages won't interfere with it.

- ✓ Export functions for users by placing **@export** in their roxygen comments.
- ✓ Use objects from other packages with **package:object** or **@importFrom package object** (recommended) or **@import package** (use with caution).
- ✓ Call **document()** to generate **NAMESPACE** and **load_all()** to reload.

| DESCRIPTION | NAMESPACE |
|---------------------------------|--------------------------------------|
| Makes packages available | Makes function available |
| Mandatory | Optional (can use :: instead) |
| use_package() | use_import_from() |



CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at r-pkgs.org • HTML cheatsheets at pos.it/cheatsheets • devtools 2.4.5 • usethis 3.1.0 • testthat 3.2.3 • roxygen2 7.3.2 • Updated: 2025-08

man/

The documentation will become the help pages in your package.

- ✓ Document each function with a roxygen block above its definition in **R/**. In RStudio, Code > Insert Roxygen Skeleton helps (Ctrl/Cmd + Alt + Shift + R).
- ✓ Document each dataset with roxygen block above the name of the dataset in quotes.
- ✓ Document the package with **use_package_doc()**.
- ✓ Build documentation in **man/** from Roxygen blocks with **document()**.

vignettes/

- ✓ Create a vignette that is included with your package with **use_vignette()**.
- ✓ Create an article that only appears on the website with **use_article()**.
- ✓ Write the body of your vignettes in R Markdown.

Websites with pkgdown

- ✓ Use GitHub and **use_pkgdown_github_pages()** to set up pkgdown and configures an automated workflow using GitHub Actions and Pages.
- ✓ If you're not using GitHub, call **use_pkgdown()** to configure pkgdown. Then build locally with **pkgdown::build_site()**.

tests/

- ✓ Set up test infrastructure with **use_testthat()**.
- ✓ Create a test file with **use_test()**.
- ✓ Write tests with **test_that()** and **expect_()**.
- ✓ Run all tests with **test()** and run tests for current file with **test_active_file()**.
- ✓ See coverage of all files with **test_coverage()** and see coverage of current file with **test_coverage_active_file()**.

ROXYGEN2

The **roxygen2** package lets you write documentation inline in your **R** files with shorthand syntax.

- Add roxygen documentation as comments beginning with **#**.
- Place a roxygen **@** tag (right) after **#** to supply a specific section of documentation.
- Untagged paragraphs will be used to generate a title, description, and details section (in that order).

```
#' Add together two numbers
#'
#' @param x A number.
#' @param y A number.
#' @returns The sum of 'x' and 'y'.
#' @export
#' @examples
#' add(1, 1)
add <- function(x, y) {
  x + y
}
```

COMMON ROXYGEN TAGS

| | | |
|--------------|----------------|----------|
| @description | @family | @returns |
| @examples | @inheritParams | @seealso |
| @examplesif | @param | |
| @export | @rdname | |

README.Rmd + NEWS.md

- ✓ Create a README and NEWS markdown files with **use_readme_rmd()** and **use_news_md()**.

| Expect statement | Tests |
|--------------------------|--|
| expect_equal() | Is equal? (within numerical tolerance) |
| expect_error() | Throws specified error? |
| expect_snapshot() | Output is unchanged? |

```
test_that("Math works", {
  expect_equal(1 + 1, 2)
  expect_equal(1 + 2, 3)
  expect_equal(1 + 3, 4)
})
```

data/

- ✓ Record how a data set was prepared as an R script and save that script to **data-raw/** with **use_data_raw()**.
- ✓ Save a prepared data object to **data/** with **use_data()**.

Package States

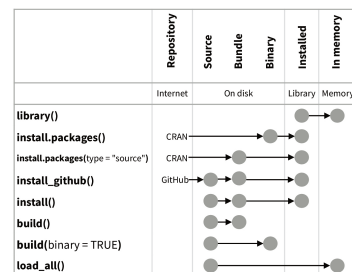
The contents of a package can be stored on disk as a:

- **source** - a directory with sub-directories (as shown in Package structure)
- **bundle** - a single compressed file (.tar.gz)
- **binary** - a single compressed file optimized for a specific OS

Packages exist in those states locally or remotely, e.g. on CRAN or on GitHub.

From those states, a package can be installed into an R library and then loaded into memory for use during an R session.

Use the functions below to move between these states.



Visit r-pkgs.org to learn much more about writing and publishing packages for R.



CC BY SA Posit Software, PBC • info@posit.co • posit.co • Learn more at r-pkgs.org • HTML cheatsheets at pos.it/cheatsheets • devtools 2.4.5 • usethis 3.1.0 • testthat 3.2.3 • roxygen2 7.3.2 • Updated: 2025-08

Figure 14.2: The Posit Package Development cheat sheet. Source: [Posit](https://posit.co), CC BY 4.0.

In January 2026, I gave a two-hour workshop on R package development in Positron covering this workflow end to end: package structure, writing and documenting functions, exported package data, programming with dplyr, unit tests with testthat, code coverage with covr, continuous integration with GitHub Actions, and a pkgdown documentation site. I also demonstrated using Positron Assistant with Claude in a few places.

All materials are available:

- Package documentation site: <https://stephenturner.github.io/rpkgdemo/>
- Full tutorial: <https://stephenturner.github.io/rpkgdemo/articles/rpkgdemo>
- Source code: <https://github.com/stephenturner/rpkgdemo>
- Recording: https://www.youtube.com/watch?v=hc_RwZx3wNE

https://www.youtube.com/watch?v=hc_RwZx3wNE

Finally, the [Writing R Extensions manual](#) is the official reference for package structure and metadata. It is a bit dry, but it is the source of truth for how packages work internally. I'd recommend starting with the R Packages book and asking your favorite AI for help when you run into trouble, but the R Extensions manual is the detailed authoritative guide and is there when you need it.

15. Reproducible Environments

An analysis can fail after a package, R, or a system library changes, even when the code is unchanged. Recording software versions and preserving the environment helps colleagues rerun the analysis and diagnose differences.

This chapter covers two levels of environment management:

- Package-level with `renv`, which records and restores R package versions
- System-level with Docker and the Rocker project, which captures the entire computing environment including R itself, system libraries, and non-R tools
- Bridging both with `pracpac`, which containerizes R packages by combining `renv` dependency capture with Docker image generation

These tools exist on a spectrum of complexity and completeness. A good rule of thumb: start with `renv` for any analysis project involving collaborators, and use Docker when you need to share system dependencies as well as R packages.

Both tools pin whatever your project already depends on. Neither one asks whether a dependency was a good idea, and the answer matters most here, because system dependencies can make installation slower or harder on a server. Chapter 13. covers choosing and vetting packages before they reach the lockfile.

15.1 Package Environments with `renv`

The `renv` package gives each R project its own private library. Instead of every project sharing a single system-wide package library (where upgrading a package for one project might break another), `renv` isolates dependencies per project and records the exact versions in a lockfile.

15.1.1 How `renv` Works

`renv` maintains three things:

1. A project-private library at `renv/library/`: packages installed here don't affect other projects
2. A lockfile (`renv.lock`): a plain-text record of every package version used in the project
3. An activation script (`renv/activate.R`): auto-loaded by `.Rprofile` to activate the project library when R starts in that directory

The lockfile is what makes sharing and restoring the environment possible. It records the package name, version, source (CRAN, Bioconductor, GitHub), and a hash for verification:

```
{
  "R": {
    "Version": "4.4.1",
    "Repositories": [
      {
        "Name": "CRAN",
        "URL": "https://packagemanager.posit.co/cran/latest"
      }
    ]
  },
  "Packages": {
    "dplyr": {
      "Package": "dplyr",
      "Version": "1.1.4",
      "Source": "Repository",
      "Repository": "CRAN",
      "Hash": "fedd9d00c2944ff00a0e2696ccf048ec"
    }
  }
}
```

```
}
}
```

This file should be committed to version control (see Chapter 1.). It's how collaborators and future you restore the same environment.

15.1.2 Core Workflow

Starting a new project:

```
renv::init()
```

This initializes `renv` in the current project: creates the private library, scans your scripts for `library()` and `require()` calls to discover dependencies, installs those packages, and writes the initial `renv.lock`.

After installing or updating packages:

```
# Install packages as usual
install.packages("ggplot2")

# or use renv's wrapper -- preferred
renv::install("tidyr")

# Then snapshot to record the new state
renv::snapshot()
```

`renv::snapshot()` updates `renv.lock` to reflect the current state of your project library. Run it whenever you add or update packages, then commit the updated lockfile.

Restoring an environment (e.g., on a new machine or after cloning a repo):

```
renv::restore()
```

This reads `renv.lock` and installs exactly the recorded versions. Collaborators can use it to install the recorded R package versions, provided those versions and their system dependencies remain available.

Tip

When you open a project with an existing `renv.lock`, `renv` will notice if your local library is out of sync with the lockfile and suggest running `renv::restore()`. You can also check status explicitly:

```
renv::status()
```

This reports any discrepancies between your installed packages and `renv.lock`.

Checking for package updates:

```
renv::update()    # update all packages to latest versions
renv::snapshot()  # then re-record the new state
```

15.1.3 Working as a Team

When collaborating on a project with `renv`:

1. One person initializes `renv` and commits `renv.lock`, `renv/activate.R`, and `.Rprofile` to the repository
2. Everyone else clones the repo and runs `renv::restore()` to set up their local environment
3. When anyone installs or updates a package, they run `renv::snapshot()` and commit the updated `renv.lock`

Commit these files; do not commit the `renv/library/` folder itself (add it to `.gitignore`; `renv` does this automatically). The library can be rebuilt from the lockfile; committing it would add thousands of binary files to your repository.

i Note

`renv` uses a global package cache on each machine, so packages are only downloaded and installed once even if you use them in multiple projects. Restoring an environment the second time is nearly instant if the packages are already cached.

15.1.4 What `renv` Does Not Solve

`renv` restores R package versions, but you must also manage:

- The R installation itself (`renv` records its version but does not install it): if collaborator A runs R 4.3 and collaborator B runs R 4.4, results may differ even with identical packages
- System libraries: packages that depend on compiled system libraries (e.g., `sf` needs GDAL, `xml2` needs libxml2) can still break if system libraries differ
- Non-R software: any external tools called from R (Python, command-line tools, etc.)
- Namespace conflicts: when multiple packages export functions with the same name, `renv` does not determine which one your script uses; that depends on load order. See Chapter 12. for how to manage this.

For version and system-level concerns, Docker provides a more complete solution.

15.2 Containers with Docker

Docker is a tool for packaging an entire computing environment (the operating system, system libraries, R, R packages, and any other software) into a portable, self-contained unit called a **container**. Containers reduce differences between machines, but results can still depend on architecture, external inputs, and runtime settings.

15.2.1 Key Concepts

Image: A read-only template that defines the environment. It contains the files and settings used to start a container.

Container: A running instance of an image. You can run multiple containers from the same image.

Dockerfile: A text file containing the instructions for building an image. It specifies the base image, software to install, files to copy, and startup command.

Registry: A place to store and share images. Docker Hub is the public default; organizations may run private registries.

15.2.2 Why Docker for R?

The limitations of `renv` described above (R version, system libraries, non-R software) are exactly what Docker addresses. Preserving the image retains the R version and system libraries for later runs. Test it on the deployment platform and preserve required data and external dependencies too.

Docker is especially useful for:

- Deploying analyses to servers: the same image that works locally runs in production (see Chapter 25. for working with IT on institutional deployment)
- Sharing complete analyses: collaborators with a compatible container runtime can run the image
- Pipelines that mix R with other tools: Python, command-line tools, databases, and R all coexist in one image
- Shiny apps: containerized apps can be deployed anywhere without managing a server environment

15.2.3 The Rocker Project

Setting up Docker for R from scratch requires some Linux knowledge and installing R and its dependencies. The [Rocker project](#) solves this by providing a curated set of R Docker images that are well-maintained, regularly updated, and designed specifically for data science workflows.

15.2.3.1 The Versioned Stack

The Rocker versioned stack is built around pinned R versions, making it the right choice for reproducible work:

| Image | Built on | Contents |
|-------------------|------------|--|
| rocker/r-ver | Ubuntu LTS | Base R at a specific version |
| rocker/rstudio | r-ver | Adds RStudio Server |
| rocker/tidyverse | rstudio | Adds tidyverse, devtools, remotes |
| rocker/verse | tidyverse | Adds TeX, Pandoc, Quarto |
| rocker/geospatial | verse | Adds spatial libraries (sf, terra, GDAL) |

The versioned images accept an R version tag, which pins both R and the package repository snapshot to that time period:

```
# R 4.4.1 with RStudio Server
docker pull rocker/rstudio:4.4.1

# R 4.3.0 with tidyverse
docker pull rocker/tidyverse:4.3.0
```

Tip

Use a specific version tag in production work; use an image digest when you need to identify an exact build. `rocker/rstudio:latest` will point to different R versions as new releases come out; `rocker/rstudio:4.4.1` always means exactly that version.

15.2.3.2 Running Rocker Containers

To start an RStudio Server session in your browser with R 4.4.1:

```
docker run --rm \
  -p 8787:8787 \
  -e PASSWORD=yourpassword \
  -v "$(pwd)":/home/rstudio/project \
  rocker/rstudio:4.4.1
```

Then open `http://localhost:8787` in a browser and log in with username `rstudio` and the password you set. The `-v` flag mounts your current directory into the container so you can access your local files.

Key flags: `--rm`: remove the container when it exits (avoids accumulating stopped containers) `-p 8787:8787`: map host port 8787 to container port 8787 `-e PASSWORD=...`: set the RStudio login password `-v host_path:container_path`: mount a local directory into the container

15.2.3.3 Writing a Dockerfile

For projects that need packages beyond what the base Rocker images provide, you write a Dockerfile that extends a Rocker image:

```
FROM rocker/tidyverse:4.4.1

# Install system dependencies (if needed)
RUN apt-get update && apt-get install -y \
  libssl-dev \
  && rm -rf /var/lib/apt/lists/*

# Install additional R packages
RUN R -e "install.packages(c('janitor', 'gt', 'gtsummary'))"
```

```
# Copy project files
COPY . /home/rstudio/project
```

Build and run the image:

```
docker build -t my-analysis:latest .
docker run --rm -p 8787:8787 -e PASSWORD=pass my-analysis:latest
```

Note

Docker and renv together. For the most complete reproducibility, use both: a Dockerfile that pins the R version (via a versioned Rocker base) and `renv` inside the container to pin package versions. The `renv.lock` file is copied into the image at build time, and `renv::restore()` installs the exact package versions during the build. This combination captures the full stack: OS, R, and packages.

15.3 prapac: R Packaging with Docker

`prapac` (Practical R Packaging with Docker) [8] bridges the R package development workflow and Docker. It provides a `usethis`-style interface that automates the generation of Docker configuration for R packages under development, using `renv` to capture dependencies.

While `renv` and basic Rocker Dockerfiles cover many use cases, `prapac` is aimed at teams that have structured their analysis code as an R package and want to containerize that package, along with all its dependencies, for deployment or sharing.

15.3.1 What prapac Does

`prapac` automates a four-step process that would otherwise require manual effort:

1. Dependency capture: uses `renv` to snapshot all package dependencies matching the developer's current environment.
2. Package building: creates a source tarball (`.tar.gz`) of the R package.
3. Dockerfile generation: writes a Dockerfile that installs the captured dependencies and the package from source.
4. Image building: optionally builds the Docker image with version tags drawn from the package `DESCRIPTION` file.

The generated Dockerfile uses a Rocker base image, installs `renv` and `BiocManager`, restores dependencies from the `renv.lock`, and installs the package, mirroring the developer's environment inside the container.

15.3.2 Installation

```
install.packages("prapac")
```

15.3.3 Workflow

15.3.3.1 Step 1: Set up your R package with renv

`prapac` assumes your analysis is structured as an R package (with a `DESCRIPTION` file) and that `renv` is initialized:

```
renv::init()
# ... develop your package, install dependencies ...
renv::snapshot()
```

15.3.3.2 Step 2: Generate Docker configuration

From inside your R package project:

```
library(pracpac)
use_docker()
```

This creates a `docker/` subdirectory containing three artifacts:

```
your-package/
├─ DESCRIPTION
├─ R/
├─ renv.lock
└─ docker/
    ├─ Dockerfile
    ├─ renv.lock      # copy of the project lockfile
    └─ your-package_1.0.0.tar.gz  # built source package
```

15.3.3.3 Step 3: Build the Docker image

Either pass `build = TRUE` to `use_docker()` to do everything in one step, or build separately:

```
# One-step: generate and build
use_docker(build = TRUE)

# Two-step: generate first, inspect/customize, then build
use_docker()
# ... optionally edit docker/Dockerfile ...
build_image()
```

The two-step approach is useful when you want to customize the generated Dockerfile before building, for example to add system library dependencies or additional non-R tools.

15.3.3.4 Step 4: Run and share the image

```
# Run the container
docker run --rm -it your-package:1.0.0

# Share via Docker Hub or a private registry
docker push your-org/your-package:1.0.0
```

The image tag is derived from the package version in `DESCRIPTION`, so image versions stay in sync with package versions automatically.

Tip

`pracpac` is particularly useful for deploying R-based pipelines alongside non-R tools. After `use_docker()` generates the initial Dockerfile, you can extend it to install Python, command-line bioinformatics tools, or any other software your pipeline needs. The R dependencies are already handled by `renv`; you just add the additional layers on top.

15.3.4 Use Cases

Analysis pipelines. Structure your analysis as an R package, containerize with `pracpac`, and deploy to a server or cloud environment with the package dependencies recorded at build time.

Shiny applications. An R package that contains a Shiny app can be containerized with `pracpac`, then deployed to any Docker-capable hosting environment.

Reproducible publications. Sharing a Docker image alongside a paper or report means reviewers and readers can rerun the analysis in the supplied environment.

Mixed-language pipelines. If your R package calls Python scripts or command-line tools, customize the generated Dockerfile to add those dependencies. The result is a single container that runs the complete pipeline.

15.4 Choosing Your Approach

You can combine these tools. The right level of environment management depends on how much isolation and portability you need:

| Scenario | Recommended approach |
|---|--|
| Solo project, short duration | <code>renv</code> only |
| Team collaboration on analysis code | <code>renv</code> + commit <code>renv.lock</code> to git |
| Need consistent R and system dependencies across machines | <code>renv</code> + Docker (Rocker base) |
| Deploying an analysis or app to a server | Docker (Rocker base) |
| Analysis structured as an R package | <code>pracpac</code> |
| Pipeline mixing R with other tools | <code>pracpac</code> with customized Dockerfile |

The most common entry point for DSTT teams is `renv`: add it to any collaborative project, commit the lockfile, and let teammates restore the environment with `renv::restore()`. This alone eliminates most “it doesn’t work on my machine” problems. Docker and `pracpac` become relevant when you need to share a fully self-contained environment or move an analysis into production.

15.5 Further Reading

The tools covered in this chapter (`renv`, Docker/Rocker, and `pracpac`) are the most practical starting points for most teams. **Nix** and **rix** offer another way to manage reproducible environments.

Nix is a package manager built around the principle of fully reproducible, declarative builds. Unlike `renv` (which captures R package versions) or Docker (which captures a container image), Nix works at the level of the entire software supply chain: every package, system library, compiler, and tool is described by a precise specification and built from source in an isolated environment. The result is a level of reproducibility that is difficult to achieve any other way.

For R users, Nix’s power is most accessible through **rix**, an rOpenSci package developed by Bruno Rodrigues and Philipp Baumann. **rix** provides an R-native interface for generating Nix environments: you specify the R version, packages, and system dependencies in R code, and **rix** writes the `default.nix` file that Nix uses to build the environment:

```
library(rix)

rix(
  r_ver = "4.4.1",
  r_pkgs = c("dplyr", "ggplot2", "tidyr"),
  system_pkgs = NULL,
  git_pkgs = NULL,
  ide = "rstudio",
  project_path = "."
)
```

This generates a `default.nix` file that, when built with Nix, produces an environment with exactly that R version and those packages, down to the system library level, without Docker, and with dependencies specified by the generated Nix configuration.

rix is a compelling option if your team is willing to install Nix and invest in learning its model. The payoff is reproducibility that is more rigorous than `renv` alone and less operationally complex than Docker for day-to-day development. Bruno Rodrigues’s book *Building Reproducible Analytical Pipelines with R* covers this approach in depth.

16. Databases from R

Spreadsheets and CSV files work well for small datasets, but many public health agencies store their surveillance data in relational databases: SQL Server, Oracle, PostgreSQL, or similar systems. These databases are managed by IT, shared across teams, and updated continuously as new data arrives. Analysts query the data on the server and retrieve the rows and columns they need.

This chapter covers how to work with databases from R. The examples use [DuckDB](#), an in-process analytical database that requires no server setup and installs as an R package. DuckDB is practical for local analysis of large files, but the same patterns (DBI, dbplyr) apply to institutional databases like SQL Server and PostgreSQL, though SQL support and type handling vary by backend. The chapter ends with connection patterns for those remote systems. Chapter 18. covers public web APIs, including CDC's open data portal and the Census Bureau.

This chapter assumes the database already exists and that someone else chose it. If your team is deciding what to stand up, what it will cost, and which budget line pays for it, Chapter 17. covers that decision.

Note

Getting access to an institutional database involves IT, data stewards, and sometimes a data use agreement. That process is covered in Chapter 25.. This chapter covers what to do once access is in hand.

16.1 DBI: The Common Interface

The [DBI](#) package defines a consistent interface for interacting with databases from R. Every database backend (DuckDB, SQL Server, PostgreSQL, SQLite) provides its own driver package, but the functions you call are the same regardless of which system you are connected to.

```
install.packages(c("DBI", "duckdb", "duckplyr", "dbplyr", "odbc"))
```

The four core DBI functions:

| Function | What it does |
|-----------------------------|--|
| <code>dbConnect()</code> | Open a connection to a database |
| <code>dbGetQuery()</code> | Run a SQL <code>SELECT</code> and return results as a data frame |
| <code>dbExecute()</code> | Run a statement and return the number of affected rows |
| <code>dbDisconnect()</code> | Close the connection |

A connection always needs to be closed when you are done. Inside a function, use `on.exit()` to close the connection even if the function errors:

```
query_example <- function() {  
  con <- DBI::dbConnect(duckdb::duckdb())  
  on.exit(DBI::dbDisconnect(con), add = TRUE)  
  DBI::dbGetQuery(con, "SELECT 1 AS example")  
}
```

16.2 DuckDB

DuckDB is an in-process analytical database designed for fast queries over large datasets. It runs inside your R session with no separate server, installs as a single package, and can read CSV and Parquet files directly without loading them into R first. It is a good fit for local analysis of data that is too large to work with comfortably as an R data frame, and its DBI interface is identical to remote databases, so many connection and query patterns transfer to SQL Server or PostgreSQL. Test SQL expressions and data types on the target backend.

16.2.1 Setting up

Connect with `dbConnect()`. An in-memory database (which disappears when the connection closes) is the default:

```
library(DBI)
library(duckdb)

con <- dbConnect(duckdb())
```

To keep the database between sessions, pass a file path:

```
con <- dbConnect(duckdb("surveillance.duckdb"))
```

16.2.2 Loading data

Use `dbWriteTable()` to load an R data frame into a DuckDB table. The examples in this chapter use a simulated weekly influenza surveillance dataset for Virginia counties, 2021–2023.

```
set.seed(42)
counties <- c(
  "Fairfax",
  "Arlington",
  "Loudoun",
  "Prince William",
  "Alexandria",
  "Chesterfield",
  "Henrico",
  "Richmond",
  "Roanoke",
  "Virginia Beach"
)
pop <- c(
  1150000,
  238000,
  420000,
  460000,
  155000,
  340000,
  330000,
  230000,
  100000,
  460000
)

flu <- expand.grid(
  week = 1:52,
  year = 2021:2023,
  county = counties,
  stringsAsFactors = FALSE
)
flu$cases <- pmax(0L, round(rnorm(nrow(flu), mean = 15, sd = 10)))
```

```
flu$population <- rep(pop, each = 156)

nrow(flu)
```

```
[1] 1560
```

```
dbWriteTable(con, "flu", flu, overwrite = TRUE)
```

To see which tables are in the database:

```
dbListTables(con)
```

```
[1] "flu"
```

16.2.3 Querying with SQL

`dbGetQuery()` runs a SQL `SELECT` and returns the result as a data frame:

```
dbGetQuery(
  con,
  "
  SELECT year, SUM(cases) AS total_cases
  FROM flu
  GROUP BY year
  ORDER BY year
"
)
```

```
  year total_cases
1 2021         7852
2 2022         7802
3 2023         7732
```

DuckDB can also read CSV and Parquet files directly using SQL, without loading them into R first:

```
# Read a CSV directly in a query
dbGetQuery(con, "SELECT * FROM read_csv('data/flu-2024.csv') LIMIT 5")

# Read a folder of Parquet files
dbGetQuery(con, "SELECT COUNT(*) FROM read_parquet('data/flu-*.parquet')")
```

16.3 dbplyr: dplyr Syntax on Any Database

Writing SQL by hand is workable for simple queries but awkward for anything that requires multiple joins, filters, and aggregations. The `dbplyr` package lets you write standard `dplyr` code against a database connection. It translates your code to SQL automatically.

16.3.1 Lazy tables

`tbl()` creates a reference to a database table. The result looks like a data frame but nothing has been pulled into R yet:

```
library(dplyr)
library(dbplyr)

flu_tbl <- tbl(con, "flu")
flu_tbl
```

```
# A query: ?? x 5
# Database: DuckDB 1.5.5 [root@Darwin 25.6.0:R 4.5.2/:memory:]
  week  year county  cases population
  <int> <int> <chr>   <dbl>      <dbl>
1     1   2021 Fairfax    29    1150000
2     2   2021 Fairfax     9    1150000
3     3   2021 Fairfax    19    1150000
4     4   2021 Fairfax    21    1150000
5     5   2021 Fairfax    19    1150000
6     6   2021 Fairfax    14    1150000
7     7   2021 Fairfax    30    1150000
8     8   2021 Fairfax    14    1150000
9     9   2021 Fairfax    35    1150000
10    10   2021 Fairfax    14    1150000
# i more rows
```

16.3.2 Writing queries with dplyr

dplyr verbs work on database tables the same way they work on data frames. The key difference is that operations are lazy: they accumulate until you ask for the results with `collect()`.

```
q <- flu_tbl |>
  filter(year == 2023) |>
  group_by(county) |>
  summarize(
    total_cases = sum(cases, na.rm = TRUE),
    rate_per_100k = round(
      sum(cases, na.rm = TRUE) / first(population) * 100000,
      1
    )
  ) |>
  arrange(desc(rate_per_100k))

q
```

```
# A query: ?? x 3
# Database: DuckDB 1.5.5 [root@Darwin 25.6.0:R 4.5.2/:memory:]
# Ordered by: desc(rate_per_100k)
  county      total_cases rate_per_100k
  <chr>          <dbl>      <dbl>
1 Roanoke           709         709
2 Alexandria        690        445.
3 Richmond          851        370
4 Arlington         836        351.
5 Chesterfield      887        261.
6 Henrico           674        204.
7 Loudoun           845        201.
8 Prince William    816        177.
9 Virginia Beach    768        167
10 Fairfax          656         57
```

The # A query: header identifies a lazy query; ?? means dbplyr has not determined the total row count. Printing it fetches a preview; `collect()` retrieves the full result:

```
collect(q)
```

```
# A tibble: 10 × 3
  county      total_cases rate_per_100k
  <chr>          <dbl>      <dbl>
1 Roanoke           709         709
2 Alexandria        690        445.
3 Richmond          851        370
4 Arlington         836        351.
5 Chesterfield      887        261.
6 Henrico           674        204.
7 Loudoun           845        201.
8 Prince William    816        177.
9 Virginia Beach    768        167
10 Fairfax          656         57
```

| | | | |
|----|----------------|-----|------|
| 1 | Roanoke | 709 | 709 |
| 2 | Alexandria | 690 | 445. |
| 3 | Richmond | 851 | 370 |
| 4 | Arlington | 836 | 351. |
| 5 | Chesterfield | 887 | 261. |
| 6 | Henrico | 674 | 204. |
| 7 | Loudoun | 845 | 201. |
| 8 | Prince William | 816 | 177. |
| 9 | Virginia Beach | 768 | 167 |
| 10 | Fairfax | 656 | 57 |

16.3.3 Inspecting generated SQL

`show_query()` reveals the SQL that dbplyr would send to the database. This is useful for debugging, learning SQL, or handing a query off to a colleague who prefers SQL:

```
show_query(q)
```

```
<SQL>
SELECT
  county,
  SUM(cases) AS total_cases,
  ROUND_EVEN((SUM(cases) / FIRST(population)) * 100000.0, CAST(ROUND(1.0, 0) AS
INTEGER)) AS rate_per_100k
FROM flu
WHERE ("year" = 2023.0)
GROUP BY county
ORDER BY rate_per_100k DESC
```

Inspect generated SQL when debugging or checking performance. Translation support and query efficiency depend on the database backend.

Tip

Pull as little data as possible. Filter, aggregate, and join on the database side before calling `collect()`. Pulling a million rows to R and then filtering in R is far slower than filtering in SQL and pulling only the rows you need.

16.4 duckplyr: DuckDB as a dplyr Backend

`duckplyr` takes a different approach. Instead of requiring an explicit connection, it makes DuckDB the execution engine for ordinary dplyr operations on data frames. You convert a data frame once with `as_duckdb_tibble()`, and subsequent dplyr operations run through DuckDB when supported.

```
library(duckplyr)
```

```
flu_duck <- as_duckdb_tibble(flu)
```

The result looks and behaves like a tibble, but dplyr verbs execute via DuckDB:

```
flu_duck |>
  filter(year == 2023) |>
  group_by(county) |>
  summarize(
    total_cases = sum(cases),
    rate_per_100k = round(sum(cases) / first(population) * 100000, 1)
  ) |>
  arrange(desc(rate_per_100k))
```

```
# A tibble: 10 × 3
  county      total_cases rate_per_100k
* <chr>      <dbl>      <dbl>
1 Roanoke      709        709
2 Alexandria   690        445.
3 Richmond     851        370
4 Arlington    836        351.
5 Chesterfield  887        261.
6 Henrico      674        204.
7 Loudoun      845        201.
8 Prince William 816        177.
9 Virginia Beach 768        167
10 Fairfax     656         57
```

16.4.1 How duckplyr differs from dbplyr

	dbplyr	duckplyr
Connection	Explicit (<code>dbConnect</code>)	None needed
Data location	Database table	R data frame
Use case	Query a database	Speed up work on large data frames
Result	Lazy (need <code>collect()</code>)	Returned directly

duckplyr is most useful when you have a large data frame already in R and want faster aggregations without the overhead of a separate database connection. dbplyr is the right tool when data lives in a database and you want to query it without pulling it all into R.

i Note

duckplyr falls back to standard dplyr automatically when it encounters a verb or expression it cannot execute through DuckDB. Fallback preserves support for operations DuckDB cannot handle; you should still validate results. You can see fallback events with `duckplyr::fallback_review()`.

16.5 Connecting to Institutional Databases

In practice, surveillance data often lives on a managed SQL Server, Oracle, or PostgreSQL server that IT administers. The `odbc` package provides R drivers for these systems. The connection syntax differs from DuckDB, while the same DBI and dbplyr interfaces remain available.

16.5.1 SQL Server

The most common database at state and local health departments. Requires the ODBC Driver for SQL Server, which IT typically installs on managed machines:

```
library(odbc)

con <- dbConnect(
  odbc(),
  driver = "ODBC Driver 18 for SQL Server",
  server = "db.health.state.gov",
  database = "surveillance",
  uid = Sys.getenv("DB_USER"),
  pwd = Sys.getenv("DB_PASSWORD"),
  encrypt = "yes"
)
```

16.5.2 PostgreSQL

Common in cloud deployments and some state agencies:

```
con <- dbConnect(
  odbc(),
  driver = "PostgreSQL Unicode",
  server = "db.health.state.gov",
  database = "surveillance",
  uid = Sys.getenv("DB_USER"),
  pwd = Sys.getenv("DB_PASSWORD")
)
```

16.5.3 Storing credentials safely

Never put database credentials in code or commit them to version control. Use environment variables instead. Store them in a project-level `.Renviron` file (which your `.gitignore` should exclude):

```
# .Renviron: do not commit this file
DB_USER=yourname
DB_PASSWORD=yourpassword
```

Retrieve them in code with `Sys.getenv()`, as shown in the connection examples above. An `.Renviron` file in the project root is loaded automatically when R starts in that directory. This keeps credentials out of scripts and repositories while making them available to anyone with appropriate access to the file.

⚠ Warning

Add `.Renviron` to your `.gitignore` before adding credentials to it. A credential committed to a repository is a credential that may be exposed.

16.5.4 Reusing Query Patterns

After the connection is open, you can reuse common DBI and dbplyr patterns, checking backend support for the functions you use:

```
# Works on SQL Server, PostgreSQL, DuckDB, etc.
cases_tbl <- tbl(con, "flu_cases")

cases_tbl |>
  filter(year == 2023, county == "Fairfax") |>
  arrange(week) |>
  collect()
```

16.6 Practical Guidance

Always close connections. Inside a function, use `on.exit(dbDisconnect(con), add = TRUE)`. At script level, close the connection explicitly or use `tryCatch(..., finally = dbDisconnect(con))`.

Use dbplyr instead of writing SQL. Unless you need a SQL feature that dbplyr does not support, dbplyr verbs are easier to read, easier to debug, and work across database backends without modification.

Filter on the database side. Every row you pull into R costs time and memory. Apply `filter()`, `group_by()`, and `summarize()` before `collect()` so you bring back only the result, not the raw data.

Check the data types after collect(). Databases and R sometimes disagree on types (dates, integers, NULLs vs. NA). Validate with `str()` or `glimpse()` after the first `collect()` and add type coercions to your setup block if needed.

17. Choosing and Paying for Data Infrastructure

Chapter 16. covers connecting to a database from R and querying it, assuming the database exists and someone else chose it. This chapter covers that choice: which system, what it costs, which budget pays for it, and how to avoid a bill nobody planned for.

The decision reaches analytic teams more often than it once did. Agencies are consolidating surveillance data out of shared drives, vendors market warehouses directly to health departments, and analysts who have never provisioned infrastructure find themselves evaluating platforms. The technical comparison is usually the easy part. Projects stall on the question of which budget line absorbs the cost.

17.1 Identifying the Problem

“We need a data warehouse” describes a solution. Before evaluating platforms, state the problem in a sentence containing no product names.

The problems that drive these conversations are distinct from one another. Files may be slow, where a quarterly analysis reads a 2 GB CSV, takes ten minutes, and exhausts memory when another year is added. Files may be scattered, where six analysts hold six extracts of one source pulled on six different dates and two reports disagree. The authoritative version of a dataset may exist without being identifiable except by asking a particular person. Access may be manual, where obtaining data requires emailing someone who runs a query and returns a spreadsheet, adding days to every question. Or several people may need concurrent read access with permissions and an access record.

These have different remedies. Scattered files and unclear provenance are organizational problems, and a warehouse purchased without addressing them becomes a warehouse of scattered files. Changing the file format can improve slow reads: Parquet with DuckDB (Section 16.2) handles considerably more data than most public health teams hold, on a laptop, at no cost. Concurrent writes and centrally managed permissions may call for a server or managed service.

Tip

Record which of these problems apply, in order of severity, before the first vendor conversation. Use that list to evaluate vendor demonstrations against your requirements.

17.2 Sizing the Requirement

Obtain the actual numbers before shopping: the size of the largest file, the total volume of non-redundant data, and the growth rate. Distinguish measured usage from projected growth.

If a team holds one to two gigabytes of non-redundant data, with small individual files, a laptop may be sufficient. What makes such holdings feel large is their distribution across hundreds of files in dozens of folders, with redundant copies of the same extract, which is a different problem with a considerably cheaper solution.

As an initial trial, test a single-file analytical database for a dataset under roughly 10 GB with a single writer. Performance depends on the queries and available memory. Between 10 GB and a few hundred gigabytes, one machine with columnar storage remains viable, and a shared server becomes a question of who needs access and how. Above that, or with genuine concurrent write traffic, or with data arriving continuously from many sources, a managed platform begins to justify its cost.

Growth rate matters more than current volume. A dataset doubling annually will cross a threshold on a predictable schedule. One growing by 200 MB per year will not, and sizing it for a decade hence purchases capacity that will never be used.

17.3 Row Stores and Column Stores

One piece of technical background clarifies the rest of the decision.

Row-oriented storage keeps values from one record together and often suits retrieving or updating individual records. Column-oriented storage groups values by field and can reduce the work needed to scan a few columns across many records. These are useful distinctions between transactional and analytical workloads, often called OLTP and OLAP.

Products can support both designs. For example, [SQL Server supports rowstore and columnstore indexes](#). Before proposing a migration, ask whether the existing system supports a suitable storage or indexing arrangement.

Performance depends on the query, data types, indexes, memory, concurrency, and hardware. Benchmark representative queries on realistic data volumes. Record elapsed time and cost, including data loading and recurring updates; a speed ratio from another workload is not a capacity estimate for yours.

[DuckDB \[9\]](#) is a columnar analytical database that runs in-process, installs as an R package, requires no server, and reads Parquet and CSV files directly. Section 16.2 covers its use. For a team whose problem is slow files, it is generally the complete answer at no cost. [MotherDuck](#) provides DuckDB with a hosted shared component for cases where several people query the same data without each holding a copy, with usage-based pricing to evaluate against your expected workload.

Note

Columnar storage and file format are separable decisions. Writing prepared datasets as Parquet provides compression and column-selective reads immediately, works with DuckDB, Python, Snowflake, and BI tools with a suitable connector, and requires no platform decision at all. Try this before committing to a platform migration.

17.4 Managed Platforms

Snowflake, BigQuery, Databricks, and Azure Synapse are the platforms a health agency is most likely to be offered. Compare their managed services with the support and capabilities your team needs.

They provide elastic compute that scales for a heavy query and scales back down, concurrency for many simultaneous users without mutual interference, managed backup and disaster recovery, permissions and audit features for your security team to evaluate, and vendor staff on call when the system fails outside business hours.

They do not provide clean data, agreed metric definitions, an end to scattered extracts, or analysts who know SQL. Your team still needs to do that work.

The strongest practical argument for a managed platform in state and local public health is rarely performance. It is that the state health department already operates one, other programs are already on it, and shared tenancy brings shared datasets, shared access controls, and an existing support path. Include those support arrangements when comparing alternatives.

17.5 Consumption Pricing

Managed platforms use different billing models. Snowflake, for example, charges for warehouse compute in credits, with prices that depend on the edition, region, and contract. Storage bills separately and is generally inexpensive. Compute accrues whenever a warehouse is running, and compute is what costs money.

By default there is no ceiling. A query left running, a scheduled job that fails and retries, a dashboard configured to refresh every five minutes against a large table, or a warehouse that never auto-suspends will accumulate charges with nothing to stop them. This is the standard mechanism by which organizations are surprised by a cloud invoice.

The following controls use Snowflake terminology; check the equivalent controls and their coverage on your chosen platform.

Configure auto-suspend so the warehouse suspends after a short idle period, one to five minutes. Idle compute can cost more than the queries themselves; account for minimum billing periods when setting auto-suspend.

Configure a warehouse resource monitor with a credit quota, notification recipients, and suspension actions. For example, notify at 50 and 75 percent and suspend below the budget limit, leaving a buffer for usage while suspension takes effect. Test that notifications reach the responsible staff.

A monitor is not a cap on the total invoice. Snowflake resource monitors cover warehouses, exclude serverless and AI services, and may exceed a threshold before suspension completes. Cloud-services charges can also continue after a warehouse stops. Use the relevant budgets and alerts for other services, and include storage and contract charges in the overall budget. [Snowflake documents the monitor's coverage and limitations](#).

Set query runtime limits so a runaway scan cannot consume a month's budget overnight.

Right-size the compute. Larger warehouse sizes cost more per second. Most public health analytical queries run acceptably at the smallest size, which is where to start.

Review cost attribution weekly for the first several months. Every major vendor ships tooling that attributes spend to users, warehouses, and individual queries. The team lead needs access to it, not only IT.

Warning

Ask the vendor in writing what occurs when committed spend is exceeded. Contracts vary: some suspend service, some bill overage at a higher on-demand rate, and some continue billing without notification. A budget line with no ceiling is a governance problem even when the technology is functioning correctly.

17.6 Build Versus Buy

Self-hosting can appear cheaper and more flexible than a managed service.

Include maintenance work when comparing costs. Self-hosting means someone patches the server, manages backups, tests restores, resolves a full disk over a holiday weekend, and completes the security questionnaire. On a small analytic team, that person is an epidemiologist hired to do epidemiology.

Choose an approach that meets the measured workload and access requirements. For a small dataset, a DuckDB file with a documented refresh process may be sufficient; check the filesystem and concurrency requirements before sharing the file.

Self-hosting does win in specific circumstances, and one of them recurs in county government. Where procurement and infrastructure barriers make a commercial cloud purchase slower and harder than standing up open-source tooling internally, self-hosting may be practical. Budget staff time for its maintenance.

17.7 Funding the Cost

A categorical epidemiology budget may not cover warehouse costs. Confirm allowable expenses with the budget owner.

Consider these funding options.

Adding your program to an existing enterprise license usually works best. Where the state health department holds a platform contract, ask whether counties or programs can be added as tenants. The marginal cost to the state is small and the case is straightforward: shared platform, shared datasets, less duplicated effort.

Folding the cost into an informatics or IT cost center is the next option, since infrastructure is what those cost centers exist to fund. It requires IT to accept that your data is their responsibility, which is a relationship question more than a budget question (Section 24.4).

Some public health funding streams permit infrastructure explicitly. Data modernization funding exists for this purpose, and several federal cooperative agreements allow infrastructure costs that a disease-specific categorical grant does not.

Paying from the program budget is feasible only where the amount is small and predictable, which argues for the less expensive options in Section 17.3.

Two things improve any of these requests. Bring a cost estimate based on measured usage and a vendor quote, including uncertainty in future usage. And bring the cost of the status quo: analyst hours spent reconciling extracts, reporting delays, duplicated storage. This lets decision-makers compare the proposed expense with the cost of current work.

17.8 Protected Data on a Managed Platform

A vendor's compliance posture and your compliance obligations are distinct, and the gap between them is where problems arise.

A vendor will represent that a platform is HIPAA-eligible. That generally means the vendor will execute a business associate agreement, that the service offers the necessary controls, and that specific configurations are supported. It does not mean your deployment is compliant. Encryption, access controls, audit logging, and data residency are typically your configuration responsibility, and a HIPAA-eligible service configured carelessly is not compliant.

Settle five things before moving protected data.

Confirm that a signed business associate agreement is in place and covers the specific services in use. Eligibility is granted per service, and the analytics or AI feature you intend to use may fall outside the agreement covering storage.

Read the data use agreement (Section 25.2). Many specify where data may be stored, who may access it, and whether it may leave a jurisdiction. A cloud region in another state can violate a DUA while satisfying HIPAA.

Establish who administers the platform, including whether vendor support staff can access your data and under what circumstances.

Establish what is logged and for how long. Audit logging is what makes “who accessed this record” an answerable question.

Review the terms governing secondary services separately. Warehouse-hosted machine learning and AI features frequently fall under different terms than the warehouse itself. Chapter 26. covers that question in detail.

Note

If you plan to use a platform's AI features, include them in the procurement review. The features that make a modern warehouse attractive include model training and natural-language querying over agency data, so the platform decision now contains an AI policy decision. Review those features and their terms before signing.

17.9 Migrating From Flat Files

Loading files is only part of a migration. Allow time to agree on dataset ownership, definitions, quality checks, and access.

Maintain shared, documented datasets. The arrangement worth building toward is a small number of curated, documented, authoritative datasets that all projects read from, replacing the pattern where each project pulls its own extract on its own schedule. This is what resolves disagreement between reports, and it is an organizational commitment more than a technical one: each dataset has an owner, its refresh cadence is documented, and projects read from it.

Translate existing QC into the database. Every flat-file workflow contains quality checks, some documented and many residing in an analyst's habits. Before migrating, document them: the row count that should fall within a range, the date field that should never hold a future value, the county codes that should match a known list. Then reimplement them as validation that runs on load (Chapter 3.). Teams that devote a working session to this documentation routinely discover checks nobody had ever written down.

Specify training datasets in the same effort. While defining the shared datasets, define synthetic training versions with the same structure for onboarding, development, and debugging (Section 23.6). The schema work is already being done.

Budget for the SQL learning curve. Analysts moving from flat files reasonably worry about learning SQL, organizing the data, maintaining QC, and sustaining collaboration through the transition. Allocate training time explicitly (Section B.2), and expect a period during which the old flat-file process runs in parallel with the new one, as in a SAS migration (Section 19.6).

Agree on data handling conventions. Inconsistency in how team members handle data tends to emerge as they begin working more independently. A migration is an opportunity to correct it, because it forces conventions to be written down. It is equally an opportunity to encode the inconsistency permanently in a schema, if nobody does that work.

17.10 Practical Guidance

Measure before shopping. Largest file, total non-redundant volume, growth rate. Use these measurements to avoid buying unnecessary capacity.

Evaluate the free option seriously. Parquet with DuckDB, for two weeks, against real data. If it resolves the problem, that is the answer. If it does not, the platform request now rests on a specific, defensible finding.

Configure cost controls on day one. Configure auto-suspend, warehouse quotas with a suspension buffer, query timeouts, and alerts covering other billed services before the first analyst receives access.

Settle funding before settling on a product. The budget conversation is slower than the technical one and is the one that stalls projects.

Execute the BAA and read the DUA before moving protected data, confirming coverage for each service.

Treat the data organization work as the project. Shared datasets, documented QC, and agreed metric definitions determine whether the infrastructure is worth having. Include this work in the migration plan.

18. APIs and Public Data Sources

Not every analysis starts with data your agency collects. The denominators for your rates come from the Census Bureau, the national counts you compare against come from CDC, and both live on someone else's server, published for exactly this kind of use.

The usual way to get that data is to browse to the portal, click through some filters, hit Export, and download a CSV, which can leave the download settings unrecorded. Six months later, nobody remembers which filters were applied or whether the provider has revised the numbers since, and the download is the one step of the analysis that cannot be re-run.

Scripted downloads record the request and make it repeatable. In R, this means using a dedicated package when one exists, building requests with `httr2` when one does not, handling API keys, and structuring the pull so the rest of the analysis stays reproducible.

Note

This chapter is about *public* data sources, where the data is intended for download and the main concerns are reproducibility and politeness. Getting access to your agency's internal databases is covered in Chapter 16., and the governance process around restricted data in Chapter 25..

18.1 What an API Request Looks Like

A web API (application programming interface) lets code request data or operations from a service. Its endpoints are URLs; the example here returns data. CDC publishes NNDSS weekly notifiable disease counts on its open data portal, and this URL asks for the 2024 pertussis records:

```
https://data.cdc.gov/resource/x9gk-5huc.json?label=Pertussis&year=2024
```

The pieces: `data.cdc.gov` is the portal, `x9gk-5huc` identifies the dataset, `.json` is the response format, and everything after the `?` is a set of query parameters that filter the result. Paste it into a browser to see JSON, a plain-text format for structured data. The response contains records and named fields.

JSON values can be text even when they represent numbers. Inspect the provider's field definitions before converting types (Section 18.7).

18.2 Where Public Health Data Lives

A few sources cover most of what public health teams pull from outside their agency:

Source	What's there	How to access it
data.cdc.gov	CDC's open data portal: NNDSS, provisional deaths, vaccination coverage, BRFSS, and hundreds more	Socrata API; <code>RSocrata</code> or <code>httr2</code>
US Census Bureau	ACS, decennial census, population estimates (your rate denominators)	<code>tidycensus</code> (free API key required)
CDC WONDER	Mortality, natality, and other query systems	Web query tool; an API that accepts XML-formatted requests
HealthData.gov	Cross-agency HHS datasets	Varies by dataset
State and local open data portals	Many jurisdictions run their own Socrata or CKAN portals	Same Socrata patterns as data.cdc.gov

18.3 Use a Package When One Exists

Check for a maintained package before implementing requests yourself. `tidycensus` handles Census queries and returns estimates with margins of error. `RSocrata` handles pagination and date conversion for Socrata portals. Confirm the package supports the dataset and fields you need; a wrapper does not remove the need to validate the result.

For example, this `tidycensus` call requests county population estimates for Virginia from the 2023 five-year ACS. Request a free key at https://api.census.gov/data/key_signup.html and store it with `tidycensus::census_api_key("YOUR_KEY", install = TRUE)`. This writes the key to `.Renviron` for future sessions (Section 18.5). Restart R, or run `readRenviron("~/.Renviron")`, before making a request in the current session. Then run:

```
va_pop <- tidycensus::get_acs(
  geography = "county",
  state = "VA",
  variables = c(population = "B01003_001"),
  year = 2023,
  survey = "acs5"
)
```

For a Socrata dataset, `RSocrata` can handle the page requests:

```
pertussis <- RSocrata::read.socrata(
  "https://data.cdc.gov/resource/x9gk-5huc.json?label=Pertussis&year=2024"
)
```

Kyle Walker's *Analyzing US Census Data* [10] covers Census data and `tidycensus` in detail. The `httr2` example below gives you control over the request and archiving steps when a package does not provide the options you need.

18.4 Building Requests with `httr2`

When no suitable wrapper exists, `httr2` lets you construct and inspect a request before sending it. Record the endpoint, filters, selected fields, and any version parameter. Keep requests in a separate acquisition script so changing a report does not repeatedly contact the service.

The complete example below uses `httr2`, `jsonlite`, and `dplyr`. Install those packages before running it. The function definitions can be run without contacting the API; the download happens only when you call `fetch_nndss()` in Section 18.8.

First, define the request for the 2024 pertussis records:

```
nndss_request <- function() {
  req <- httr2::request("https://data.cdc.gov/resource/x9gk-5huc.json") |>
    httr2::req_url_query(
      `$where` = "label='Pertussis' AND year='2024'",
      `limit` = 5000,
      `order` = ":id"
    ) |>
    httr2::req_retry(max_tries = 3) |>
    httr2::req_throttle(capacity = 30, fill_time_s = 60)
  token <- Sys.getenv("SOCRATA_APP_TOKEN")
  if (nzchar(token)) {
    req <- httr2::req_headers(req, `X-App-Token` = token)
  }
  req
}
```

`req_url_query()` encodes the filter and other query parameters. The page size and ordering will stay fixed while the download function changes the offset. The optional token comes from your environment (Section 18.5); it is not part of the URL or the saved metadata.

18.4.1 Pagination

Check the API's row limit and completion rule. A successful HTTP response can contain only the first page. For offset-based pagination, use a stable ordering and continue until the documented completion condition is met. Socrata recommends an explicit `order such as :id`. Changes to the dataset during pagination can still affect results; use a provider snapshot when available.

Set a maximum number of requests to catch accidental loops, and treat reaching it as an incomplete pull. Compare the final row count and coverage with expectations. A round count is a reason to investigate, but neither a round nor an irregular count proves completeness.

18.4.2 Retries and rate limits

Use bounded retries for transient errors and obey the provider's request limits. An authentication failure or invalid query requires correction, not repeated requests. The request above allows at most three attempts per page. By default, `req_retry()` retries HTTP 429 and 503 responses and respects a `Retry-After` header when the server supplies one. `req_throttle()` limits the request rate to 30 requests per minute here; choose a rate appropriate to the provider. Section 18.8 shows how to archive each page and record whether the pull completed.

18.5 API Keys and Tokens

Many APIs require a key, and others work better with one. The Census API requires a free key. Socrata portals work anonymously but throttle anonymous traffic aggressively; a free app token moves you to a much more generous rate limit. Keys are credentials, and the discipline is the same as for database passwords in Section 16.5: keep them in `.Renviron`, read them with `Sys.getenv()`, and never commit them.

In a scheduled pipeline, supply the credential through the job's approved secret store (Section 11.3). Avoid writing credentials into request logs, filenames, or archived metadata.

18.6 Separate the Pull from the Analysis

Do not put an API call at the top of an analysis script. Put it in its own script, write the raw response to `data/raw/` under a dated filename or directory (Section 2.3), and have the analysis read the file.

Saving the response preserves the data used by the analysis. An API reflects the data as of right now, and many public health sources revise: CDC's provisional death counts, for example, are updated for recent weeks as certificates arrive, so the same query run a month apart returns different numbers. The archived pages record what you retrieved in that run. Read and combine those files when reproducing the analysis. Separating the pull also means the analysis renders offline and keeps working when the API is down or the dataset moves, and iterating on a report no longer re-downloads the same data on every render.

In an automated pipeline, the pull becomes its own step, scheduled with the tools in Chapter 11. or declared as a `targets` node (Section 11.4) so downstream steps rerun only when a new file arrives.

18.7 Validate What Comes Back

Check the response schema, types, and values before using the data. Required columns may be renamed or removed; a field may disappear from every record when all its values are missing. Distinguish an allowed all-missing field from a missing required identifier.

Define how to interpret missing values and suppression flags from the provider's documentation. Do not turn a suppressed count into zero. Reject unexpected text during numeric conversion; a warning that becomes `NA` can otherwise disappear into later calculations.

Also check the requested year, geography, disease, row count, and reporting-period coverage. These checks catch a syntactically valid response to the wrong query. Chapter 3. covers implementing checks and stopping the pipeline when required checks fail.

For this example, require the disease label, year, and reporting week. Convert the count field `m1` only after checking that its nonmissing values contain integer text. The validator allows an entirely missing count field but rejects missing identifiers:

```

validate_nndss <- function(x) {
  required <- c("label", "year", "week")
  if (!is.data.frame(x) || !all(required %in% names(x)) || nrow(x) == 0L) {
    stop("Expected a nonempty table with label, year, and week")
  }
  if (anyNA(x[required]) || any(x$label != "Pertussis")) {
    stop("Unexpected or missing label, year, or week")
  }
  # Socrata can omit a field from every row when all values are missing.
  if (!"m1" %in% names(x)) {
    x$m1 <- NA_character_
  }
  for (field in c("year", "week", "m1")) {
    raw <- as.character(x[[field]])
    present <- !is.na(raw)
    if (any(!grepl("^[0-9]+$", raw[present]))) {
      stop("Unexpected non-integer text in ", field)
    }
    value <- suppressWarnings(as.integer(raw))
    if (any(present & is.na(value))) {
      stop("Integer conversion failed in ", field)
    }
    x[[field]] <- value
  }
  if (any(x$year != 2024L) || any(x$week < 1L | x$week > 53L)) {
    stop("Unexpected year or reporting week")
  }
  # Missing m1 values remain missing; retain provider flag fields for interpretation.
  x
}

```

The original response files will retain the provider's fields and values. This validator leaves missing counts missing and keeps any flag fields in the table. Check those flags and the dataset's field definitions before interpreting a missing value. The checks here establish the expected schema and basic values; they do not establish geographic coverage or completeness of reporting.

You can check the missing-count behavior without making a request:

```

example_records <- data.frame(
  label = c("Pertussis", "Pertussis"),
  year = c("2024", "2024"),
  week = c("1", "2"),
  m1 = c("3", NA_character_)
)

validate_nndss(example_records)

```

```

  label year week m1
1 Pertussis 2024    1 3
2 Pertussis 2024    2 NA

```

The count becomes numeric, and the missing count stays missing. Replacing "3" with an unexpected value such as "suppressed" would stop validation. Decide how to interpret such a flag from the provider's documentation before adding a conversion rule.

18.8 Download and Reuse

Use one reader for both a new download and later reuse. It combines the saved JSON pages and applies the validator:

```
read_nndss_pages <- function(paths) {
  pages <- lapply(paths, function(path) {
    jsonlite::fromJSON(path, simplifyVector = TRUE)
  })
  validate_nndss(dplyr::bind_rows(pages))
}
```

The download function creates a new archive directory and saves each response before moving to the next page. It continues until the API returns an empty page. Reaching `max_pages` stops the pull with an error, so a loop limit cannot be mistaken for successful completion:

```
fetch_nndss <- function(out_dir, max_pages = 100L) {
  stopifnot(
    length(max_pages) == 1L,
    is.finite(max_pages),
    max_pages >= 1,
    max_pages == floor(max_pages)
  )
  if (!dir.create(out_dir, recursive = TRUE, showWarnings = FALSE)) {
    stop("Choose a new output directory")
  }
  req <- nndss_request()
  paths <- character()
  complete <- FALSE
  for (i in seq_len(max_pages)) {
    resp <- httr2::req_perform(httr2::req_url_query(
      req,
      `offset` = (i - 1L) * 5000L
    ))
    path <- file.path(out_dir, sprintf("page-%04d.json", i))
    writeBin(httr2::resp_body_raw(resp), path)
    paths <- c(paths, path)
    if (length(httr2::resp_body_json(resp)) == 0L) {
      complete <- TRUE
      break
    }
  }
  if (!complete) {
    stop("Page limit reached; incomplete archive, do not analyze")
  }
  result <- read_nndss_pages(paths)
  metadata <- list(
    retrieved_utc = format(Sys.time(), tz = "UTC", usetz = TRUE),
    endpoint = "https://data.cdc.gov/resource/x9gk-5huc.json",
    filter = "label='Pertussis' AND year='2024'",
    order = ":id",
    page_size = 5000L,
    rows = nrow(result),
    complete = TRUE
  )
  # No credential is written to the archive.
  jsonlite::write_json(
    metadata,
    file.path(out_dir, "manifest.json"),
    auto_unbox = TRUE,
    pretty = TRUE
  )
  result
}
```

The completion manifest records the endpoint, filter, ordering, page size, retrieval time, and row count. It is written only after pagination and validation succeed. If a request fails partway through, the pages

already downloaded remain available for inspection, but there is no completion manifest. Start a fresh archive when retrying so files from different pulls are not mixed.

To download, run the four function definitions above, then call `fetch_nndss()` from an acquisition script or the console. This example puts the archive in a directory named with its UTC retrieval time:

```
archive_dir <- file.path(
  "data",
  "raw",
  format(Sys.time(), "%Y%m%dT%H%M%S", tz = "UTC")
)
pertussis <- fetch_nndss(archive_dir)
```

Record the chosen archive path with the analysis. For a later offline run, set `archive_dir` to that existing directory and use the saved pages:

```
metadata <- jsonlite::read_json(file.path(archive_dir, "manifest.json"))
stopifnot(isTRUE(metadata$complete))
page_files <- sort(list.files(
  archive_dir,
  pattern = "^page-[0-9]+[.]json$",
  full.names = TRUE
))
pertussis <- read_nndss_pages(page_files)
stopifnot(nrow(pertussis) == metadata$rows)
```

A missing manifest stops this reuse step. The row-count check detects some accidental changes to an archive; it does not prove the source data was complete or unchanged during pagination. Keep the saved responses with the analysis and apply the coverage checks described in Section 18.7. Report rendering should read this archive without making a new API call.

19. Migrating from Legacy Workflows

Most public health data work predates the tools in this book. State health departments run surveillance on SAS programs written over twenty years by people who have since retired, weekly reports come out of Excel workbooks where the analysis lives in formula chains and pivot tables, and case data sits in Access databases on a shared drive. These workflows produce real, trusted outputs every week. The concern is whether the team can continue to maintain and staff them.

Translating syntax is only part of a migration. Even a familiar operation such as `PROC MEANS` requires checking how the R equivalent handles grouping and missing values. What takes judgment is deciding what to migrate and in what order, proving that the new pipeline produces the same numbers as the old one, and retiring the old system without losing the institutional knowledge embedded in it. This chapter is about that process.

19.1 Why Migrate, and Why Not

The case for migrating usually starts with money: SAS licensing is a recurring line item that R eliminates, and for a small team the savings alone can fund a training program. The hiring pool tilts the same direction. New analysts arrive knowing R or Python, the people who know SAS are retiring, and every year a workflow stays in SAS it gets harder to find someone who can maintain it. Beyond cost and staffing, an R workflow can integrate with the tools covered earlier: parameterized Quarto reports (Chapter 6.), dashboards (Chapter 9.), scheduled pipelines on GitHub Actions (Chapter 11.), or a version-controlled analysis that can be reviewed and handed off (Chapter 1.).

There are equally good reasons to leave a particular workflow alone, at least for now. A report that is being retired should not be rewritten; use migration planning to identify products that can be retired. A rewrite the team cannot maintain is worse than the status quo: if the only SAS analyst retires in six months and nobody else knows R yet, the first investment is training (Section 22.3). Timing matters too, since a rewrite in the middle of an outbreak response adds risk exactly when the team can least absorb it.

Account for the risk that a migration rewrites software that works. The legacy program's numbers are trusted; the rewrite's are not until you prove they match. Plan time for those comparisons.

19.2 Inventory and Triage

Start by listing every recurring product the legacy stack produces: each report, each dataset handed to another program, each dashboard feed. For each one, record what runs it, how often, who consumes the output, and who understands the code. This inventory is usually revealing on its own. Most teams find a handful of products that matter enormously, a long tail that runs out of habit, and at least one program nobody can explain.

Then triage. Good first candidates for migration score high on all of these:

- It runs on a schedule (weekly, monthly), so the payoff recurs and there are regular opportunities to compare old against new.
- Someone who understands the current logic is still available to answer questions.
- The output has an engaged consumer who will notice, and care, if numbers change.
- It is painful today: manual steps, fragile hand-offs, a single person who can run it.

One-off historical analyses generally should not be migrated at all. Archive the code and its outputs, document what they were for, and leave them alone.

Migrate one product end to end before starting a second. A complete pilot, from raw data to delivered report, running in parallel with the legacy version, teaches the team more than partially migrating ten programs, and it gives stakeholders a working example to assess.

19.3 Reading Legacy Data Formats

Whatever else migrates, the data has to. The `haven` package reads SAS (`.sas7bdat`, `.xpt`), SPSS (`.sav`), and Stata (`.dta`) files directly:

```
library(haven)

path <- system.file("examples", "iris.sas7bdat", package = "haven")
read_sas(path)
```

```
# A tibble: 150 × 5
  Sepal_Length Sepal_Width Petal_Length Petal_Width Species
      <dbl>      <dbl>      <dbl>      <dbl> <chr>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5          5         3.6         1.4         0.2 setosa
6         5.4         3.9         1.7         0.4 setosa
7         4.6         3.4         1.4         0.3 setosa
8          5         3.4         1.5         0.2 setosa
9         4.4         2.9         1.4         0.2 setosa
10        4.9         3.1         1.5         0.1 setosa
# i 140 more rows
```

Check labels, missing values, and dates during import. SAS stores value labels (formats) separately from the data, in a catalog file typically named `formats.sas7bcat`; pass it to `read_sas()` and the labels arrive as labelled columns, which `as_factor()` converts to R factors:

```
cases <- read_sas(
  "data/raw/cases.sas7bdat",
  catalog_file = "data/raw/formats.sas7bcat"
)
cases <- as_factor(cases)
```

Missing values need more care. SAS distinguishes 28 numeric missing values (`.`, `._`, and `.A` through `.Z`), often used to encode *why* a value is missing: refused, unknown, not applicable. R has one `NA`. `haven` preserves the distinctions as tagged NAs (see `?haven::tagged_na`), but most downstream R code will treat them all as plain `NA`, so if the distinctions carry meaning, convert them to an explicit column before they are lost.

Dates come through correctly, because `haven` knows that SAS counts days from 1960-01-01 while R counts from 1970-01-01. Hand-rolled imports do not get that conversion: a CSV exported from SAS with raw date numbers will be off by exactly 3,653 days, an error distinctive enough to recognize on sight.

For Excel sources, `readxl` reads `.xlsx` and `.xls` files, and the spreadsheet-hygiene guidance in Chapter 2. applies doubly to inherited workbooks. For Access, the `odbc` package can connect directly on Windows, where the Access ODBC driver is usually already installed:

```
library(DBI)

con <- dbConnect(
  odbc::odbc(),
  .connection_string = paste0(
    "Driver={Microsoft Access Driver (*.mdb, *.accdb)};",
    "Dbq=S:/surveillance/cases.accdb;"
  )
)
dbListTables(con)
```

Whichever format the data arrives in, convert it once and save an analysis-ready copy in an open format (CSV or Parquet, per Chapter 2.), keeping the original file as the untouched raw archive. Proprietary formats depend on software that will not always be there to read them.

19.4 Translating SAS Code

SAS and R differ more in mindset than in capability. A SAS program is a sequence of DATA steps (implicit row-by-row loops that read one dataset and write another) punctuated by PROC calls. Idiomatic R is vectorized: operations apply to whole columns at once, chained with pipes. A literal line-by-line translation produces R code that works but reads like SAS; the better approach is to translate what each step accomplishes.

Common operations have R counterparts, though their defaults and edge cases can differ:

SAS	R equivalent
DATA step assignments, IF/THEN	<code>mutate()</code> with <code>if_else()</code> or <code>case_when()</code>
PROC SORT (and NODUPKEY)	<code>arrange()</code> , <code>distinct()</code>
PROC MEANS, PROC SUMMARY	<code>group_by()</code> + <code>summarize()</code>
PROC FREQ	<code>count()</code> , <code>janitor::tabyl()</code>
PROC TRANSPOSE	<code>tidyr::pivot_longer()</code> , <code>pivot_wider()</code>
MERGE in a DATA step	Joins only after checking BY-key multiplicity and row-matching semantics
PROC SQL	<code>dplyr</code> , or SQL directly via DBI (Chapter 16.)
PROC FORMAT	factor levels and labels
PROC EXPORT	<code>readr::write_csv()</code>
%MACRO	a function

Macros require a different approach. SAS macros are text substitution; R functions evaluate arguments and return values, and they are easier to test and document. A macro library that grew over years often translates into a small set of R functions, which is the point at which a team package (Chapter 14.) starts to make sense.

19.4.1 Check merge and missing-value semantics

A SAS DATA-step match-merge is not a SQL join. If one BY group has two records on the left and three on the right, a match-merge produces three records; an all-matches join produces six. Check duplicate keys, row order, retained values, and unmatched records before choosing an R join. The [SAS combining-data documentation](#) explains the match-merge behavior.

[SAS numeric missing values compare below ordinary numbers](#). R missing-value comparisons return NA. For example, SAS evaluates `count < 0` as true for `.` and `.A`, as well as for a negative count. This small check makes that difference explicit:

```
count <- c(NA_real_, haven::tagged_na("a"), -1, 0, 5)
sas_expected <- c(TRUE, TRUE, TRUE, FALSE, FALSE)
r_comparison <- count < 0
sas_comparison <- is.na(count) | count < 0
stopifnot(identical(sas_comparison, sas_expected))
data.frame(count, r_comparison, sas_comparison)
```

	count	r_comparison	sas_comparison
1	NA	NA	TRUE
2	NA	NA	TRUE
3	-1	TRUE	TRUE
4	0	FALSE	FALSE
5	5	FALSE	FALSE

This reproduces that particular SAS expression; it does not define the rule your analysis should use. If missing counts should be excluded, write and test that requirement in both versions. Tagged missing values preserve distinctions during import, but do not give R SAS's comparison semantics. Other operations need their own parity cases.

19.4.2 AI-assisted translation

Legacy translation is a task AI coding assistants (Chapter 8.) handle well. Claude Code (Section 8.3) can take a several-hundred-line SAS program and produce a credible R draft in minutes, and it is equally useful in the other direction: asking it to *explain* an inherited program, step by step, before you translate anything. When the original author is gone, an AI walkthrough of what a complex DATA step does can help you investigate it.

Treat the output as a draft, though. Allow time for verification. AI translation can produce code that looks right and runs clean but handles an edge case (usually missing values) differently than the original. Nothing that comes out of an assistant should reach production without passing the parity checks in the next section. Check code as well as data before sharing it: scripts can contain credentials, internal paths, and sensitive values. The privacy considerations in Section 8.4 apply to both. Confirm this use is permitted under your agency's AI policy before you start, and note that an assistant may read data files in your project directory (Section 26.4).

19.5 Verifying Parity

The legacy program's output is your test oracle. Freeze an input dataset, run it through both pipelines, and compare the outputs with code. The `waldo` package gives readable, precise diffs:

```
sas_out <- data.frame(
  county = c("Fairfax", "Arlington", "Loudoun"),
  cases = c(23L, 41L, 18L),
  rate = c(2.0, 17.2, 4.3)
)

r_out <- data.frame(
  county = c("Fairfax", "Arlington", "Loudoun"),
  cases = c(23L, 41L, 18L),
  rate = c(2.0, 17.2, 4.2)
)

waldo::compare(sas_out, r_out)
```

```
old vs new
      rate
old[1, ] 2.0
old[2, ] 17.2
- old[3, ] 4.3
+ new[3, ] 4.2

`old$rate`: 2.00 17.20 4.30
`new$rate`: 2.00 17.20 4.20
```

The diff points straight at the discrepancy. Counts should match exactly; derived numbers like rates should match within a small tolerance (`waldo::compare()` takes a `tolerance` argument), because the two engines can legitimately differ in the last decimal places of floating-point arithmetic.

When outputs differ, the cause is usually a place where SAS and R behave differently by design. Check these differences:

Behavior	SAS	R
Rounding halves	Away from zero: <code>ROUND(2.5)</code> is 3	Half to even: <code>round(2.5)</code> is 2
Missing values	<code>.</code> , <code>._</code> , and <code>.A-.Z</code>	A single <code>NA</code>
Missing values in sorts	Sort first (missing is smallest)	<code>arrange()</code> puts <code>NA</code> last
Blank strings	<code>" "</code> is missing	<code>" "</code> is a value, distinct from <code>NA</code>
Date origin	1960-01-01	1970-01-01

The rounding difference is worth seeing once, because it looks so much like a bug:

```
round(c(0.5, 1.5, 2.5))
```

```
[1] 0 2 2
```

R follows the IEEE standard and rounds halves to the nearest even digit, so 0.5 rounds to 0 and 2.5 rounds to 2, where SAS would give 1 and 3. If the legacy program rounds before a suppression threshold or publishes rounded rates, the two pipelines will disagree by one in the last digit on exactly the values ending in five, and only those.

Every discrepancy gets one of two resolutions: fix the R code to match, or accept the difference and write down why. Keep the comparison script and a short parity report in the repository alongside the migrated code, listing each accepted difference and its justification. That document is what lets you tell a program manager, with evidence, that the new numbers are trustworthy, and it is exactly the kind of artifact analytical peer review (Chapter 27.) should examine.

19.6 Running in Parallel and Cutting Over

Parity on a frozen test dataset is necessary but not sufficient, because live data keeps finding cases the test data did not contain. So run both pipelines in parallel on real data for a fixed number of cycles: four to eight weeks for a weekly report is typical. Each cycle, produce both outputs, run the comparison script, and investigate anything new. These investigations are where the legacy program's undocumented behavior surfaces: the special-casing of one bad data year, the county recode nobody mentioned. Each one either becomes deliberate logic in the new pipeline or a documented, accepted difference.

Agree on the cutover criteria before the parallel period starts (for example: N consecutive cycles with no unexplained discrepancies), so the team can assess readiness against agreed criteria. Then, at cutover:

- Move the legacy code into the repository under a `legacy/` directory, read-only, with a README mapping each old program to its replacement (Section 21.1).
- Run the legacy pipeline one final time and archive its outputs as the last reference point.
- Actually decommission it. Remove it from the schedule, and cancel the license when the last workflow that needs it is gone.

The decommissioning step is the one teams skip, and skipping it is costly. Two live pipelines mean two sources of truth, and they will eventually diverge, at which point someone has to figure out which one the program has been using. Cancel unused licenses after confirming that no retained workflow needs them.

19.7 Excel and Access Workflows

Spreadsheet migrations differ from SAS migrations because there is no program to translate. The analysis may be spread across formula chains, pivot caches, and cells someone updates by hand, which can be difficult to compare and review: one inserted row can silently break a range reference three sheets away.

The pattern that works is to split the workbook's two jobs. If data entry happens in Excel, let it keep happening there, in a workbook restructured to the rectangular, one-thing-per-cell standard of Chapter 2.. Everything after entry moves to a script: the workbook becomes an *input* that `readxl` reads, and R produces the outputs. The translations are mostly one-to-one. A `VLOOKUP` chain is a `left_join()`, a pivot table is `group_by()` plus `summarize()` (with `pivot_wider()` for the layout), and logic encoded in conditional formatting becomes an explicit column. Validate imported data (Chapter 3.) to detect entry errors before the analysis runs.

Access databases play two roles that migrate separately. As storage, the tables can be read directly via `odbc` as shown above, or exported once if the database is being retired. Simple select and aggregation queries may translate directly. [Microsoft documents differences between Access SQL and ANSI SQL](#). Inventory Access-specific functions, date syntax, crosstab queries, query parameters supplied by forms, and VBA dependencies separately; these can require substantial changes. Validate the translated query outputs before retiring the original application. If several people still need to enter and look up cases interactively, Access can keep that front-end job for now, with R connecting read-only for analysis; the longer-term home for shared, structured case data is an IT-managed database, which is a conversation to start with the people in Section 25.4.

19.8 The People Side

Legacy programs often contain undocumented analytical decisions. Somewhere in that SAS code is the exclusion rule from a 2015 data quality incident and the workaround for one lab's misformatted files, none of it written down anywhere else. The person who wrote the program holds the other half of that memory.

Involve the legacy author in the migration. Pair them with the team's strongest R programmer: the veteran explains what each step is for and reviews parity discrepancies, while the R partner writes idiomatic new code, and both learn. This is upskilling (Section 22.3) with a concrete deliverable attached, and it gives the author an opportunity to learn R, which can support retention (Section 22.6). When the author is already gone, the code and existing documentation may be your main sources, which makes the careful reading, and the AI-assisted explanation described earlier, that much more important.

Finally, pick the timing like the project decision it is (Chapter 24.). A migration has a long middle where both systems run and the team carries double work. Schedule the parallel runs for a period with fewer reporting demands.

19.9 Further Reading

- The [haven documentation](#) covers the details of reading SAS, SPSS, and Stata files, including labelled values and tagged missings.
- [R for Excel Users](#) (Lowndes and Horst) is a free, workshop-format introduction to R aimed specifically at people whose current workflow is Excel.
- [The Epidemiologist R Handbook](#) includes a "Transition to R" chapter with practical advice, and R equivalents for common tasks, aimed at epidemiologists coming from SAS, Stata, and SPSS.

20. Code Style

Consistent code style helps colleagues read, review, and modify an analysis. It also makes your own work easier to revisit months later.

Style conventions specify how code is written: how things are named, where spaces go, how long lines get before they break. None of these decisions affect what the code computes, but all of them affect how quickly a reader can understand it and how cleanly changes show up in version control.

This chapter covers the [Tidyverse Style Guide](#) as the standard for R, the [Air formatter](#) for automatically applying that standard, and the `lintr` package for catching issues that a formatter cannot.

Record the team's style conventions in the handbook (Section 23.4) and automate formatting so reviewers can focus on the analysis (Section 27.2).

20.1 The Tidyverse Style Guide

The [Tidyverse Style Guide](#) is the de facto standard for R code in data science. It is opinionated but well-reasoned, and following it means your code will look immediately familiar to anyone else working in R.

The most important conventions:

- Names: use `snake_case`, all lowercase, words separated by underscores. Prefer `flu_cases` over `fluCases` or `FluCases`. Object names should be nouns; function names should be verbs.
- Spacing: put spaces around most operators (`<-`, `+`, `-`, `==`, and others) and after commas. Do not put spaces inside parentheses or before a comma.
- Line length: keep lines to 80 characters or fewer. Long expressions should be broken across lines with consistent indentation.
- Indentation: two spaces per level. Not tabs.
- Assignment: use `<-` for assignment, not `=`.
- Pipes: when chaining multiple operations, put each step on its own line, indented two spaces.

The full guide covers function naming, documentation, `ggplot2` layering, and much more. It is worth reading once and returning to when questions arise. The key practical point: you do not have to memorize all of it. A formatter handles most of the mechanical rules automatically.

20.2 Formatting with Air

[Air](#) is an R code formatter written in Rust. When you save a file, it reformats the code to follow consistent conventions for spacing, indentation, and line breaks. It does not rename variables or change logic, only whitespace and layout.

The value of a formatter is that it removes style decisions from the development loop entirely. You write code, save, and Air handles presentation. Code review conversations shift from “this line is too long” or “there should be a space here” to the actual logic. Diffs in version control (see Chapter 1.) show only meaningful changes, not incidental reformatting.

20.2.1 Setting Up Format on Save in Positron

Air ships bundled with [Positron](#), so no separate installation is needed. To enable format on save, configure a `.vscode/settings.json` file in your project root. The quickest way:

```
usethis::use_air()
```

This creates the settings file with the correct entries. Alternatively, create or edit `.vscode/settings.json` manually:

```
{
  "[r]": {
    "editor.formatOnSave": true,
```

```

    "editor.defaultFormatter": "Posit.air-vscode"
  }
}

```

Committing `.vscode/settings.json` to version control means every collaborator on the project gets format-on-save behavior automatically, without any manual setup.

For Quarto documents, add a second entry so that R code cells inside `.qmd` files are also formatted:

```

{
  "[r]": {
    "editor.formatOnSave": true,
    "editor.defaultFormatter": "Posit.air-vscode"
  },
  "[quarto]": {
    "editor.formatOnSave": true,
    "editor.defaultFormatter": "quarto.quarto"
  }
}

```

To format a single R cell inside a `.qmd` file without saving the whole document, place your cursor in the cell and press `Cmd+K Cmd+F` (Mac) or `Ctrl+K Ctrl+F` (Windows/Linux).

Formatting a whole project at once

To format every R file in a project at once, open the Command Palette (`Cmd+Shift+P` on Mac, `Ctrl+Shift+P` on Windows/Linux) and run **Air: Format Workspace Folder**. This is useful when first introducing Air to an existing codebase.

20.2.2 Before and After

The following two blocks show the same code before and after Air formats it. The computation is identical in both versions; only the layout changes.

Before:

```

flu_summary<-flu|>filter(cases>0,!is.na(county))|>group_by(county,disease)|
>summarise(total_cases=sum(cases),avg_rate=mean(rate,na.rm=TRUE),.groups="drop")|
>arrange(desc(total_cases))

```

After:

```

flu_summary <- flu |>
  filter(cases > 0, !is.na(county)) |>
  group_by(county, disease) |>
  summarise(
    total_cases = sum(cases),
    avg_rate = mean(rate, na.rm = TRUE),
    .groups = "drop"
  ) |>
  arrange(desc(total_cases))

```

The formatted version uses more lines to show the steps separately. Each transformation appears on its own line, arguments that belong together are grouped, and indentation reflects the structure of the pipeline. When this code appears in a pull request, a reviewer can identify which step changed.

Note

Air's configuration lives in an optional `air.toml` file at the project root. The defaults (2-space indentation, 80-character line width) follow the Tidyverse Style Guide and are suitable for most projects. The most common reason to configure `air.toml` is to exclude specific functions from Air's table-formatting behavior, using the `skip` field.

20.3 Linting with lintr

A formatter handles whitespace. A linter goes further: it reads code statically and flags patterns that are likely wrong or poor style, without running the code and without changing anything.

The `lintr` package is the standard linting tool for R. It checks for issues including:

- Assignment with `=` instead of `<-`
- Missing or extra spaces around operators
- Use of `T` or `F` instead of `TRUE` or `FALSE` (fragile, because `T` can be overwritten as a variable name)
- Redundant comparisons like `x == TRUE`
- Lines that exceed the recommended length
- Variable names that do not follow `snake_case` convention

Unlike Air, `lintr` does not fix anything. It reports what it finds and leaves decisions to you. This is useful for catching issues that a formatter cannot handle: semantic shortcuts, naming violations, and patterns that are syntactically valid but likely to cause confusion.

20.3.1 Getting Started

Install `lintr` from CRAN:

```
install.packages("lintr")
```

The primary interface is `lintr::lint("your_file.R")`, which reads a file from disk and prints a list of warnings. To lint every R file in a project at once, use `lintr::lint_dir()`. Each line of output identifies the file, the line and column, the severity, and the name of the rule that triggered it.

20.3.2 Examples

Consider an R script, `analysis.R`, with several common problems:

```
x = 10
y<-x*2
flag = T
if (flag == TRUE) print(y)
```

Running `lintr::lint("analysis.R")` produces:

```
analysis.R:2:3: style: [assignment_linter] Use one of <-, <<- for assignment, not =.
x = 10
^
analysis.R:3:2: style: [infix_spaces_linter] Put spaces around all infix operators.
y<-x*2
  ^~
analysis.R:3:5: style: [infix_spaces_linter] Put spaces around all infix operators.
y<-x*2
  ^
analysis.R:4:6: style: [assignment_linter] Use one of <-, <<- for assignment, not =.
flag = T
  ^
analysis.R:4:9: style: [T_and_F_symbol_linter] Use TRUE instead of the symbol T.
flag = T
      ~^
analysis.R:5:27: style: [trailing_blank_lines_linter] Add a terminal newline.
if (flag == TRUE) print(y)
```

Working through these one at a time: `=` for assignment is flagged because the style guide prefers `<-` for assignment, although R also permits `=` in this context. The missing spaces around `<-` and `*` on the second line are spacing violations. `T` as a shorthand for `TRUE` is flagged because `T` is just a variable that happens to default to `TRUE` and can be overwritten (`T <- 42` is legal R). And `flag == TRUE` is redundant: if `flag` is already a logical value, `if (flag)` says exactly the same thing.

A second example shows naming convention warnings. Given a file with:

```
totalCases <- 100
meanRate <- totalCases / 30
ReportTitle <- paste("Weekly report:", format(Sys.Date(), "%B %Y"))
```

lintr flags all three names:

```
analysis.R:2:1: style: [object_name_linter] Variable and function name style should
match snake_case or symbols.
totalCases <- 100
^~~~~~
analysis.R:3:1: style: [object_name_linter] Variable and function name style should
match snake_case or symbols.
meanRate <- totalCases / 30
^~~~~~
analysis.R:4:1: style: [object_name_linter] Variable and function name style should
match snake_case or symbols.
ReportTitle <- paste("Weekly report:", format(Sys.Date(), "%B %Y"))
^~~~~~
analysis.R:4:68: style: [trailing_blank_lines_linter] Add a terminal newline.
ReportTitle <- paste("Weekly report:", format(Sys.Date(), "%B %Y"))
```

All three names are flagged for camelCase or PascalCase. In an analysis that will be maintained and extended, consistent snake_case naming is worth the small upfront effort of renaming.

Using Air and lintr together

Air and lintr complement each other. Air handles layout automatically on every save. lintr catches things that layout cannot: semantic shortcuts like `T` for `TRUE`, naming convention violations, and redundant expressions. A practical workflow is to let Air format on save and run `lintr::lint_dir()` periodically to catch the issues Air does not touch.

21. Documentation

Code that runs correctly but cannot be understood is a liability. Six months after writing an analysis, even the original author may not remember what a variable represents, why a particular filter was applied, or where a dataset came from. For a new team member inheriting existing work, undocumented code is essentially opaque.

Documentation is the investment that makes analytical work reusable, reviewable, and transferable. Leave enough information that the next person (or your future self) can pick up the work without starting over.

21.1 READMEs

Every project repository should have a README at the root. This is the first thing anyone sees when they open a project, and it should answer the questions a new reader has immediately:

- What does this project do?
- What data does it use, and where does that data come from?
- How do you run the analysis?
- What are the outputs and where do they go?

A README does not need to be long. A one-page README that answers these questions is more valuable than a ten-page document that describes the history of the project and the background literature.

21.1.1 A Minimal README Template

```
# Project Title

One or two sentences describing what this analysis does and why it was done.

## Data

What data is used. Where it comes from (source, date accessed, any access
requirements). If data files are not in the repository (e.g., because
they contain PII), explain where they should be placed.

## Running the Analysis

How to reproduce the results. What to run first, what runs next.
Any packages or environment requirements (see environments chapter).

## Outputs

What the analysis produces. Where outputs are written.
```

Commit the README with the project. Update it when something material changes. A stale README is worse than a brief one, because it creates false confidence that the instructions are current.

21.1.2 README for Shared Utility Code

If a repository contains functions or tools used by other projects, the README should also document the interface: what the key functions are, what arguments they take, and a short example showing typical use.

21.2 Inline Comments

Use comments to explain the reasons for decisions. The code itself shows what is happening; a comment adds value by explaining the reason behind a decision that might otherwise look arbitrary.

A comment that adds no information:

```
# filter to active cases
cases <- cases |> filter(status == "active")
```

The code already says this. The comment is noise.

A comment that adds information:

```
# exclude cases with status "pending" and "transferred": these are not yet
# reportable and would inflate counts. see data dictionary for full status codes.
cases <- cases |> filter(status == "active")
```

This explains why only active cases are kept, information that is not in the code itself and that a reviewer or future maintainer needs.

Comment when:

- A non-obvious decision was made and the reason matters (a filter that excludes something, a constant that came from an external source)
- A known data quality issue is being worked around
- An approach was chosen over an obvious alternative for a specific reason

Do not comment:

- Code that reads naturally and does what it says
- Every line as a matter of habit
- To explain what R functions do (that is what documentation is for)

21.2.1 Sectioning Longer Scripts

For analysis scripts that run longer than a screen, section headers help readers navigate. In R, headers recognized by RStudio and Positron use `# ----` or `# ====` suffixes:

```
# Load data ----
# Clean and filter ----
# Calculate rates ----
# Produce outputs ----
```

These create an outline in the document navigator (the panel at the bottom of the script editor in Positron) that lets readers jump directly to a section without scrolling.

21.3 Documenting Functions with roxygen2

When a function will be reused across scripts, projects, or team members, it should be documented at the function definition, not in a separate file. The standard tool for this in R is [roxygen2](#).

`roxygen2` documentation lives in comments immediately above the function definition, starting with `#'`. The most important tags:

- `@param`: documents each argument: its name, expected type, and what it does
- `@return`: describes what the function returns
- `@examples`: shows how to call the function, ideally with a runnable example

A minimal documented function:

```
#' Calculate age-adjusted rate per 100,000
#'
#' Applies direct standardization to compute an age-adjusted rate using
#' the 2000 U.S. standard population weights.
```

```
#'
#' @param cases A data frame with columns `age_group`, `events`, and `population`.
#' @param std_pop A data frame with columns `age_group` and `std_weight`. Defaults
#'   to the 2000 U.S. standard population.
#'
#' @return A single numeric value: the age-adjusted rate per 100,000 population.
#'
#' @examples
#' calculate_age_adjusted_rate(flu_cases_2023)
calculate_age_adjusted_rate <- function(cases, std_pop = us_std_pop_2000) {
  # ... function body
}
```

Even a brief description, one `@param` per argument, and one `@return` is substantially better than no documentation at all. A future reader knows what the function expects without having to read and mentally execute the implementation.

21.3.1 When Functions Are More Than Utilities

Keep project-specific helpers with the analysis. For functions shared across projects, Chapter 14. explains packaging and distribution; Section 13.9 covers how to divide the shared code into maintainable packages.

21.4 Project-Level Documentation

Analysis projects involve more than code: there are data sources, methodological decisions, known data quality issues, and interpretive context that do not belong in code comments or README files but that someone needs to know.

21.4.1 Data Dictionaries

If a project uses a dataset with non-obvious columns (abbreviated names, coded values, or context-dependent meanings), maintain a data dictionary alongside the analysis. This can be a simple table in a Markdown or Quarto document:

Column	Type	Description
<code>case_id</code>	character	Unique case identifier; not persistent across system updates
<code>epi_class</code>	character	Epidemiologic classification: <code>confirmed</code> , <code>probable</code> , <code>suspect</code>
<code>rpt_cnty_cd</code>	character	FIPS code of the county of report, stored as text to preserve leading zeros (not necessarily the county of residence)
<code>onset_dt</code>	date	Symptom onset date as reported; missing if not collected for this condition

A data dictionary does not need to document every column. Document the columns that are confusing, the ones that carry important caveats, and the ones where the name does not tell the full story.

A machine-readable dictionary can also supply labels and definitions to the pipeline. Keeping metric definitions in a machine-readable file that both the R code and the dashboard consume is covered in Section 10.7.

21.4.2 Decision Logs

Analyses involve choices that are not visible in the code: why a particular date range was chosen, why a jurisdiction was excluded, why one method was preferred over another when both were defensible. These

decisions are worth recording, because they will be questioned by a reviewer, by a stakeholder, or by a future analyst who is maintaining the work.

A simple decision log entry might look like:

2024-11-14: Excluded 2020 data from trend analysis. COVID-related disruptions to case reporting caused substantial undercounting across multiple conditions; the 2020 data points would distort trend estimates in ways that are not informative about the underlying epidemiology. Analysis covers 2018–2019 and 2021–2023.

This does not need to be elaborate. A short paragraph with a date and a rationale is enough to reconstruct the reasoning later.

21.4.3 Methods Documentation

For analyses with non-trivial statistical methods, a methods document that describes the approach in plain language (separate from the code) is worth maintaining. This is the document a reviewer (see Chapter 27.) or stakeholder can read to understand what was done without reading R code. It should describe the data sources, the analytical approach, known limitations, and any validation steps taken.

This document can be a section of the final report, a standalone Quarto document in the repository, or a README section, wherever it is most likely to be found and updated.

21.5 Documentation as a Team Practice

Update project documentation in the same change as the code or data definition it describes. A reviewer should be able to find the inputs, understand the decisions, and reproduce the expected outputs (Section 27.2).

Record the team's minimum documentation requirements in its handbook (Section 23.4), and link to each project's README and data dictionary. Keep setup and team procedures in the handbook; keep project-specific explanations with the project.

II

Nontechnical

22	Building and Sustaining a Data Science Team	139
23	Onboarding and the Team Handbook	143
24	Project Management	147
25	Working with IT and Data Governance	151
26	AI Governance in a Health Agency .	157
27	Peer Review for Data Analysis	163
28	Communicating Findings	167

22. Building and Sustaining a Data Science Team

A health department may depend on one analyst to maintain a surveillance pipeline and explain its assumptions. If that person leaves, colleagues may struggle to update it. A department building a new team faces related questions: which skills to hire for, how to classify the positions, and where the team should report.

The aim is building a data science capability that does not depend on any single person and that fits the realities of a public health agency. The project management chapter (Chapter 24.) covers running a project well. This one covers building and keeping the team that runs the projects.

22.1 Roles on a Public Health Data Science Team

The titles overlap and the boundaries are fuzzy. An epidemiologist, a data analyst, a data scientist, a data engineer, and an informatician can have job descriptions that read almost identically, and in a small agency the same person may answer to all five. It helps to think less about titles and more about the functions that have to be covered:

- Subject-matter framing: turning a program question into an answerable analytic question. This requires epidemiology and domain expertise.
- Data engineering: getting data out of source systems, moving it, and keeping pipelines running. Often the least visible function and the one whose absence hurts most.
- Analysis and modeling: the statistical and computational work of producing the result.
- Communication and delivery: reports, dashboards, and the translation of findings for people who did not run the analysis (Chapter 28.).

Analysts often specialize in one of these areas while maintaining working knowledge of the others. The practices in this book (version control, reproducible reporting, validation) are the working competence that lets one person cover more than their specialty while maintaining quality.

On a small team, each person may cover several functions. Check that the team can handle all four. A lightweight skills matrix (functions down one axis, people across the other, proficiency in the cells) is a quick way to see where training or additional staff would help.

22.2 Hiring and Position Descriptions

Hiring a data scientist into a government agency runs straight into a classification problem. Many civil-service systems have no “data scientist” job classification at all, so the role gets filed under whatever existing title is closest: epidemiologist, IT specialist, statistician, research analyst. Each comes with a salary band and a set of required credentials that may not match the work. A position classified as “epidemiologist III” may require a degree the best candidate lacks, or cap salary below what the labor market commands for the skills you actually need.

You cannot always fix the classification, but you can write the position description around the work. State what the person will do: build reproducible analytic pipelines, work in version control, query institutional databases, produce reports and dashboards, communicate findings to program staff. These descriptions help applicants assess whether they are qualified and give interviewers specific skills to evaluate.

When evaluating candidates, a portfolio tells you more than a transcript. A GitHub profile, a published dashboard, or a short take-home exercise¹ that mirrors the actual work reveals whether someone can do the job in a way that a list of degrees cannot. A junior hire should show they can write clean, working

¹Take-home exercises should be followed up with an oral interview to walk through the exercise, since candidates may use AI tools to complete them.

code and learn quickly; a senior hire should show judgment about when an analysis is good enough, how to structure a project so others can pick it up, and how to mentor.

Choose between a full-time hire and a contractor based on the duration and type of work. A contractor can deliver specialized skills quickly and is the right call for a bounded, well-specified build. A full-time employee accumulates the institutional knowledge (which data sources have known limitations, which stakeholders need what) that makes the next project faster. For anything ongoing, especially recurring surveillance work, continuity usually wins.

22.3 Growing Skills on the Team You Have

Hiring is not the only way, and often not the best way, to build capacity. Experienced analysts may know Excel, SAS, or SPSS well and need time to learn R, Git, and Quarto. (Migrating the workflows themselves, as opposed to the people who run them, is the subject of Chapter 19.) Upskilling the people you already have is frequently more practical than competing for scarce outside hires, and it builds on institutional knowledge that a new hire would have to acquire from scratch.

Free resources are available (Appendix A.), but staff also need protected learning time. Procurement rules may affect access to paid training (Appendix B.). Learning to work reproducibly while also delivering the weekly reports is nearly impossible if the weekly reports consume every hour. Capacity-building that is not given real, defended time on the calendar does not happen, no matter how good the intentions.

Teams can combine several approaches:

- Structured curricula for foundational skills, working through a book or course as a group (Appendix A.).
- Programs built for this purpose, like the [DSTT program](#) this book grew out of, which pair training with coaching on the team's actual projects.
- Communities of practice inside or across agencies, where people doing similar work share patterns and help each other solve problems.
- Pairing and code review (Chapter 27.) for learning as well as quality control: reviewing a colleague's pull request is one of the most effective ways for both people to learn.

It also helps to be explicit that the practices in this book are learned incrementally. Nobody adopts version control, reproducible reporting, validation, and reproducible environments all at once. A team that picks up one practice per quarter is moving at a healthy pace.

22.4 Onboarding and Knowledge Continuity

One measure of this risk is the [bus factor](#): how many people could leave before the work stops? A bus factor of one, the lone analyst who is the only one who understands the pipeline, leaves the team vulnerable to an absence or departure. The goal of onboarding and continuity practices is to get that number above one and keep it there.

Assign backup coverage for each recurring product and give staff time to learn work outside their specialty. Track which products still depend on one person. Chapter 23. covers the handbook, practice runs, and handoff procedures that make this coverage usable.

22.5 Where the Team Sits

Where a data science function reports within an agency shapes what it can do. There is no single right answer, but the common options trade off in predictable ways:

- Inside an epidemiology or surveillance program keeps the team close to the subject-matter questions and the people who will use the findings, at the cost of distance from the data infrastructure and the IT relationships that provision it.
- Inside informatics or IT puts the team next to the databases, servers, and access controls, at the cost of distance from the program questions and sometimes from the analytic culture.
- A standalone analytics unit serving the whole agency can set its own standards and serve many programs, at the cost of having to build relationships with each program.

A related choice is the service model. A centralized team is a shared resource that programs request work from; it builds deep technical capability and consistent standards but can become a bottleneck and can drift away from any one program's needs. Embedded analysts sit inside programs, close to the work, but can become isolated and reinvent each other's solutions. Many agencies land on a hybrid: a small central

team that sets standards and handles cross-cutting infrastructure, with analysts embedded in the largest programs.

Work with IT and data governance staff regardless of where the team reports. The team's ability to access data, run pipelines, and publish results depends on it, which is why Section 24.4 and Chapter 25. treat those relationships as part of project planning.

22.6 Retention and Sustainability

Public sector salaries may be lower than industry salaries. Agencies should also consider the working conditions they can offer. People stay in public health data science for the mission, for autonomy and ownership over meaningful work, for the chance to keep growing, and for the conditions that let them do the work well. Each of those is something an agency can actually offer.

Tooling is one of those conditions, and an underrated one. Skilled people who are forced to work in outdated, click-heavy, irreproducible environments (exporting from one system, pasting into another, hand-checking numbers) will leave for somewhere that lets them work the way they know how. The modern practices in this book are not only about output quality; giving capable people good tools is itself a retention strategy.

Succession planning, documentation, and cross-training help the team continue its work when someone leaves. Shared knowledge also lets staff take leave or change responsibilities without leaving a critical system unsupported.

22.7 Further Reading

- *Executive Data Science* [11] is a short, practical book on structuring and managing data science teams and working with executive stakeholders. Useful for team leads and for analysts who want to understand the decisions being made above them.
- *Build a Career in Data Science* [12] covers hiring, job descriptions, interviewing, and career growth from both sides of the table, and is a useful reference for managers writing position descriptions and evaluating candidates.
- CSTE and the Public Health Informatics Institute publish workforce and competency resources specific to public health data and informatics roles, which are helpful when mapping these general practices onto agency classifications.

23. Onboarding and the Team Handbook

Staff leave with two weeks' notice. Fellows rotate on an annual cycle. Someone goes on parental leave in the middle of a surveillance season, and the person covering has never opened the script. In each case a recurring product has to change hands in a window too short to write documentation from scratch.

Section 22.4 describes the risks of relying on one person. Maintain a written handbook a new analyst can follow on the first day, which continues to function after its author departs.

23.1 Why Keep a Handbook

Most teams know they should document their work. What they produce is scattered: a README in one repository, a data dictionary in a shared drive folder, setup instructions in an old email thread, and procedures that only one person knows.

A handbook differs in having a single entry point. A new staff member is directed to one document, and everything required is either in it or linked from it. This gives a new analyst instructions they can follow even when the person who wrote them is unavailable.

A handbook reduces time spent repeating setup instructions, leaving colleagues more time to discuss the work itself.

Note

A handbook is distinct from project documentation. Chapter 21. covers READMEs, inline comments, and data dictionaries, which document specific code and specific datasets. The handbook documents how the team operates: where things are, who approves what, and what to do when an extract fails.

23.2 Contents of the Handbook

A workable table of contents for a small public health analytic team:

1. Orientation: what the team does, which products it delivers and on what cadence, and who the stakeholders are.
2. Environment setup: installing and configuring R, Positron, Git, and any database drivers on an agency-managed machine. Chapter 15. covers the reproducible-environment side of this.
3. Access requests: every system the team touches, who owns it, who approves access, which form to file, and typical approval time. This section saves the most calendar time, because a new analyst who files all access requests on the first day becomes productive weeks earlier than one who discovers each requirement upon encountering it.
4. Repository layout and naming conventions: where code lives, where data lives, and how projects are named (Chapter 2., Chapter 1.).
5. Update protocols, one document per recurring product, covered below.
6. Data source inventory: every source the team uses, its origin, its update frequency, its upstream owner, and its known defects.
7. Glossary and metric definitions: what the team means by a case, a rate, a denominator, and a reporting week. Disagreements here produce figures that do not reconcile across products.
8. Contacts: names and roles for upstream data owners, IT contacts, the disclosure reviewer, and whoever understands why the county denominators are adjusted.

Keep the handbook in version control alongside the code, written in markdown and rendered if desired. Whatever format you choose, record changes and assign responsibility for keeping it current.

23.3 Testing Setup Documentation

Test the setup instructions with a colleague.

Write the setup instructions, then hand them to someone who has never performed the setup, sit beside them, and observe. Provide no assistance. Record every point at which they stop, guess, or ask a question. Those points are the gaps in the documentation, including assumptions the author may have overlooked.

An experienced colleague unfamiliar with the setup can be a useful first reader. Make clear that you are testing the instructions, so readers feel comfortable reporting unclear steps.

Common gaps include assumed software or permissions, screenshots from another version, Unix commands that do not work on a managed Windows machine, and Git instructions that assume prior experience.

Tip

Date screenshots, or record the software version they depict. Screenshots can clarify instructions, but interfaces change. A line reading “screenshots current as of GitLab 17.2, October 2025” tells the reader whether to trust what they are looking at.

23.4 Conventions

Record the choices specific to your team: branch naming, commit size, file naming, documentation requirements, and where restricted data belongs. Link to Section 1.2, Chapter 2., and Chapter 21. for the general practices. Use examples from your own repositories where a rule is easy to misinterpret.

23.5 Update Protocols

For each recurring product the team delivers, write one document covering the steps required to produce it, the inputs and their sources, the expected outputs and their appearance, the known failure modes and the response to each, and whom to contact for anything not listed.

These documents serve two purposes. They are the coverage plan when the owner is unavailable, and they are the training material when someone new assumes the product.

Writing the protocols can reveal knowledge gaps between roles. Where a team has specialized, with one person in biosurveillance, one writing R, and one maintaining the dashboard, the incomplete section of the protocol document may concern a handoff between those roles. Gaps in protocol documentation indicate where knowledge is not shared.

Warning

Include instructions for known failures. A line reading “if the query returns zero rows, check whether the facility list changed; this occurs roughly twice a year” gives the next analyst a specific check to try. Add an entry each time something breaks.

23.6 Practice Repositories and Training Data

New analysts require somewhere to make mistakes. A practice repository seeded with a small realistic project allows someone to learn the team’s Git workflow by branching, breaking something, and opening a merge request without touching a production pipeline. This gives new analysts practice with the workflow.

The data equivalent is a set of standardized training datasets mirroring the structure of real data without the protected information: the same column names, the same types, the same awkward date formats, the same known quirks, with fabricated values. Their utility extends well beyond onboarding. They are what you develop against before entering a secure environment (Section 25.5), what you use to demonstrate a problem to a colleague, and what you provide to a support request or an AI assistant in place of real records (Chapter 26.).

Building them is not free, and the task never becomes urgent on its own. The argument that funds it is that the same datasets serve onboarding, development, and debugging. A team establishing shared enterprise datasets (Section 17.9) is already performing most of the specification work, so the training versions should be built in the same effort.

23.7 Cross-Training

Specialization occurs without anyone deciding on it. One person becomes the dashboard person, another the R person, another the one who understands the syndromic surveillance feed. The arrangement is efficient until one of them is unavailable.

Rotating ownership of recurring products is the practical countermeasure, so reserve time for it. Select one product per quarter. The current owner does not run it; they sit with a colleague while that colleague runs it from the protocol document, and every inaccuracy in the document is corrected as it is found. The result is a tested protocol and a second person who has performed the work, which differs from a second person who has read about it.

Complete a rotation before a known absence. Teams that handle parental leave, overlapping PTO, and the question of who covers a script have generally answered that question in advance.

Note

Pair rotation with code review (Chapter 27.). Running a product yourself gives you useful context for reviewing changes to it.

23.8 Handoff and Offboarding

Two weeks is the customary notice period and is insufficient for writing documentation from scratch. It is sufficient for executing a checklist that already exists.

Inventory what the departing person owns: every recurring product, repository, scheduled job, credential, and relationship with an upstream data owner. Compile this before it is needed and keep it in the handbook.

Name the person taking over each item.

Run each recurring product once, with the departing person observing and the new owner operating from the protocol document, correcting the document wherever it fails.

Transfer or reissue credentials. Scheduled jobs running under a departing employee's account are a common cause of a pipeline dying several weeks after their last day, and the failure is particularly damaging because nothing errors visibly until an expected report does not arrive (Section 11.6).

Record the undocumented reasoning. Half an hour spent asking why a county is excluded and why the analysis uses the March file recovers knowledge that is otherwise lost.

Update the handbook, including the contacts section, before the last day.

23.9 Adapting an Existing Handbook

Look for existing handbooks you can adapt. Other agencies have written most of what you need, and many publish it.

Adapting a borrowed handbook is faster than drafting one, and the translation is where much of the value lies. Converting instructions written for a hosted Git service into instructions for a self-hosted instance requires stating how your instance differs. Converting instructions written for a team with a managed analytics server requires stating where your team actually executes code. Each substitution helps identify assumptions your handbook should explain.

Ask through whatever professional network or community platform your team participates in. Agencies are generally willing to share, and they are solving the same problems in parallel without any means of discovering one another.

23.10 Maintaining the Handbook

Without regular maintenance, a handbook can give new staff outdated instructions.

Assign an owner by name. Review on a schedule, quarterly being reasonable for a small team, and attach the review to something that already occurs so that it requires no independent motivation. The natural triggers are onboarding a new person, which provides a chance to test the instructions, and any occasion when a protocol document proves wrong during an incident.

Keep edits inexpensive. A handbook requiring a merge request, three reviewers, and a rendering step will not be updated during the ten minutes when someone notices a broken instruction. Markdown files in the team repository with a low review threshold strike an appropriate balance.

 Tip

The best occasion to write a handbook section is the first time you explain something to a colleague. The explanation is already being composed; recording it is cheaper than delivering it twice.

23.11 The IT Liaison

Where possible, involve a named IT representative early enough to understand the team's systems and priorities. They can identify approval lead times and suggest supported alternatives before infrastructure constraints delay a project. Record the contact's responsibilities in the handbook, along with the procedures for requesting access, installing approved software, and escalating incidents. Section 24.4 covers planning these requests.

23.12 Further Reading

- Chapter 21. covers code and project documentation; link to those documents from the handbook.
- Chapter 22. covers roles, hiring, and retention, of which onboarding is one component.
- Appendix B. covers funding the training that a new analyst's onboarding plan identifies as a gap.
- GitLab publishes its complete company handbook at handbook.gitlab.com. It is far larger than a small analytic team requires and shows how an organization can document its operations in a shared handbook. The sections describing how the handbook itself is maintained are the relevant ones.

24. Project Management

Unclear scope, conflicting expectations, and poor coordination can prevent a public health analysis from being useful. A rigorous analysis of the wrong question, delivered six months late to an audience that can't act on it, is a failed project regardless of its technical quality. This chapter covers the non-technical practices that help projects succeed: defining scope before work begins, collaborating as a team, working with IT and data governance partners, and communicating findings to people who will actually use them.

For a broader treatment of managing data science work, Roger Peng, Brian Caffo, and Jeff Leek's *Executive Data Science* [11] is a concise, practical resource for both team leads and the analysts working alongside them.

24.1 Defining the Project

24.1.1 Start with the question

The most important work in a data science project often happens before any data is touched. Peng and Matsui's *The Art of Data Science* [13] frames the entire analysis process around a specific task: state the question precisely. Vague questions produce vague analyses. "Can you look at overdose trends?" is not a question that will lead anywhere useful. "Has the age distribution of overdose deaths in our state shifted since 2019, and does the pattern differ by rural versus urban counties?" is a question that can be answered.

Before agreeing to take on a project, work with the requestor to articulate:

- What specific question are we answering?
- Who will use the findings, and what decision or action do they enable?
- What would a satisfying answer look like: a number, a chart, a report, or a recommendation?
- What does success look like, and how will we know when we've reached it?

Use the answers to agree on the project's scope and deliverables.

24.1.2 Project charters

For anything beyond a quick turnaround request, a brief project charter documents the shared understanding between the data science team and its stakeholders before work begins. A charter doesn't need to be a formal document; a shared notes page or a few paragraphs in a project tracking tool is sufficient. What matters is that the key parties have read it and agreed to it before anyone opens a dataset.

A useful project charter for public health data science covers:

- The question: stated precisely, as described above.
- Data sources and access: what data will be used, where it lives, and what approvals are needed (data use agreements, IRB review, IT provisioning).
- Deliverables: what will actually be produced and in what format (report, dashboard, dataset, presentation slides)?
- Timeline: key milestones and a realistic completion date, with explicit acknowledgment of dependencies.
- Stakeholders: who the primary requestor is, who the audience is, who has final say on scope.
- Out of scope: what the project will explicitly *not* address; this is often as important as what it will.

The charter is also the right place to surface risks early. If the analysis depends on data that hasn't been collected yet, or on linking two systems that have never been joined, that should be visible before commitments are made.

24.2 Managing Scope

Scope creep, the gradual expansion of a project beyond its original definition, is endemic to data science work and is usually well-intentioned. Stakeholders see early results and ask "can you also look at...?", and analysts, wanting to be helpful, say yes. Over time, a focused project becomes an unfocused one.

Make scope changes explicit. When a new request comes in mid-project, treat it as a scope change. Acknowledge it, discuss whether it belongs in the current project or a future one, and adjust the timeline if it's added. The charter is the reference point for these conversations.

Plan time for delivery. The gap between “the analysis is done” and “the findings are in front of the people who need them” is consistently larger than anticipated. Time for writing, review, revision, and presentation needs to be included in project plans.

Know when to say “not yet.” Not every analytic question can be answered well with available data, methods, or time. It is better to scope a project to what can be answered defensibly than to produce a shaky analysis of a broader question. A clear, well-supported answer to a narrow question is more useful to a program than a hedged non-answer to a large one.

24.3 Collaborating as a Team

Agree on who owns each deliverable, who reviews it, and who covers an absence. Use a shared repository and a review workflow (Chapter 1., Section 27.2). Record dependencies and decisions where the team can find them.

Include reproducibility in the acceptance criteria: a colleague should be able to rerun the analysis from the recorded inputs and environment. Allocate time for that check before delivery. Chapter 15. covers restoring the environment; Chapter 21. covers the project instructions; Section 23.5 covers recurring-product handoffs.

24.4 Working with Your IT Team

IT is a necessary partner in institutional public health work. Server access, database credentials, software installation, network permissions, and data transfer approvals all flow through IT. Projects that treat IT as an afterthought routinely stall at the moment they are otherwise ready to move.

24.4.1 Start early

IT requests take time, and often more time than anticipated. Getting new software approved and installed on a shared server may take days or weeks. Database access for a sensitive dataset may require formal requests, security reviews, and management sign-off at multiple levels. If the analysis depends on accessing a new data source or running on infrastructure you haven't used before, initiate those conversations at project kickoff.

24.4.2 Be specific about what you need

A clear, specific request is far easier to fulfill than a vague one. When approaching IT, be prepared to specify:

- What software or tools are needed, including version numbers if relevant.
- What data needs to be accessed, where it lives, and in what format.
- What level of compute and storage the analysis requires.
- Whether the work involves sensitive or regulated data (HIPAA, PII, restricted-use files) that requires special handling.
- Any hard deadlines driven by reporting requirements or funding timelines.

Framing requests in terms of the program need (e.g., “this analysis supports the quarterly overdose surveillance report due to the state health officer in March”) helps IT understand the stakes and prioritize accordingly.

Two requests deserve special preparation because they carry a cost and not just a permission. Standing up or joining a database platform is a budget decision as much as a technical one, and a request should include measured storage and compute needs (Chapter 17.). Approval for an AI tool depends on agency policy (Chapter 26.).

Where team composition is within your control, include an IT representative in project planning to identify resource and access requirements early (Section 23.11).

24.4.3 Secure data environments

Much of the data used in public health analytics is sensitive: vital records, case surveillance data, insurance claims, linked longitudinal datasets. Familiarize yourself with your organization's data governance policies and the specific terms of any data use agreements governing the datasets you work with. Some analyses must be conducted within designated secure environments (managed servers, data enclaves, air-gapped systems) and outputs may need to pass a disclosure review before leaving.

These constraints shape what tools and workflows are actually feasible. It is better to understand them at project start than to build an analysis pipeline that cannot be used where the data actually lives. See Chapter 25. for a more detailed treatment of data use agreements, HIPAA, disclosure review, and working in secure data environments.

24.5 Communicating Findings

Findings need to reach the people who can use them. Public health data science ultimately serves program and policy decisions made by people who did not run the analysis and may not read a methods section.

24.5.1 Know your audience

Different stakeholders need different things:

- Program staff doing day-to-day work need actionable findings at the right level of detail, often a summary with supporting data available for those who want to dig in.
- Program directors and health officers need the bottom line and its implications, typically in a brief format (one page, one slide) with uncertainty communicated plainly.
- Methodologically sophisticated collaborators (epidemiologists, biostatisticians, peer reviewers) need full methods, sensitivity analyses, and honest discussion of limitations.

One analysis rarely serves all three audiences from a single document. Plan to produce multiple outputs from the same underlying work.

24.5.2 Communicate uncertainty honestly

Public health decisions are made under uncertainty, and analysis should convey what is known, what is estimated, and with what confidence. An administrator told “overdose deaths increased by exactly 12.3%” who later learns the estimate carried substantial uncertainty will trust the team less the next time. One given “we estimate a 10–15% increase, with the uncertainty driven primarily by changes in reporting completeness” has something they can actually act on.

That said, excessive hedging is its own failure mode. When the data clearly shows something, say so clearly. Uncertainty does not mean ambiguity about everything.

24.5.3 Match the output format to the need

A written report, an interactive dashboard, a two-page brief, and a slide deck are all appropriate in different contexts. Choose a format that helps the intended audience use the findings. See Chapter 9. for guidance on building dashboards for data that needs to be explored or updated regularly. See Chapter 28. for a more detailed treatment of message structure, visualization choices, and communicating uncertainty.

24.6 Further Reading

- *Executive Data Science* [11]. A short, practical book on managing data science teams and working with executive stakeholders. Essential reading for team leads; useful context for analysts.
- *The Art of Data Science* [13]. Covers the full data analysis process from question formulation through communication. The epicycles-of-analysis framework is particularly useful for understanding why projects rarely proceed in a straight line, and why that is normal.

25. Working with IT and Data Governance

Before beginning an analysis, locate the data and confirm the approvals needed to use it. Infrastructure, legal agreements, and agency policies determine where and how you can work. Section 24.4 introduces IT as a project management concern; this chapter goes into the operational detail: the types of environments public health data scientists actually work in, the legal frameworks that govern data use, and the practical steps for getting access to data and meeting those requirements.

25.1 Data Environments in Public Health

Public health data science happens across a wide range of computing environments, and the environment shapes what tools and workflows are feasible. Understanding where data lives, and what constraints that imposes, is the starting point.

Laptops and local files are the simplest setup. Data is on your machine, code runs locally, and there are no network permissions to navigate. This works for small, non-sensitive datasets (public-use files, aggregated data, synthetic data) but fails for anything regulated: sensitive data should not live on a personal laptop without explicit authorization.

Shared network drives are the most common data storage arrangement at state and local health departments. The data lives on a server managed by IT, mapped as a drive letter or network path on each authorized user's machine. As described in Chapter 2., this creates a specific challenge: code lives in version control, but data cannot leave the drive. Use configurable paths to the shared drive as described in Section 2.3.2.

Institutional databases (SQL Server, Oracle, PostgreSQL) are increasingly common for high-volume surveillance data, vital records, and claims. Queries run on the server and return only results, so raw data never has to move. This is generally preferable to a shared drive for structured, regularly updated data. Chapter 16. covers querying databases with DBI and dbplyr.

Secure remote desktops and data enclaves are used for the most sensitive data, including restricted-use vital records, Medicaid claims, and linked longitudinal datasets. Analysts connect via remote desktop to a server where the data lives. Copy-paste and file transfer are often disabled. Internet access may be restricted. The data never leaves the server; only approved outputs do, after passing a disclosure review. CDC's Research Data Center and many state-level restricted data programs work this way.

Air-gapped systems go further: no external network connection at all. These are rare but exist for certain federal and state datasets. Code and packages must be brought in through an approved intake process, and every output leaves through a formal review.

Tip

The more restrictive the environment, the more planning is required upfront. In a data enclave, you cannot install a package on a whim or look up documentation online. Test your code on approved public-use or synthetic data in a less restricted environment first, resolve all dependency questions, and then bring the finalized analysis into the enclave.

25.2 Data Use Agreements

A data use agreement (DUA) is a legal contract between the organization providing data and the organization receiving it. DUAs define what data is being shared, who is allowed to access it, what it can be used for, and what happens to it at the end of the project. They are routine in public health. Any time data is obtained from another agency, a federal program, a health system, or a research partner, expect a DUA.

25.2.1 What DUAs Cover

The most consequential terms are:

Permitted uses. DUAs typically specify the exact purpose for which the data was obtained. Using data for a secondary purpose (even one that seems obviously fine) may require an amendment. Read this section carefully before scoping the analysis.

Authorized users. The agreement usually names individuals or roles who may access the data. Adding a new team member or sharing data with a collaborator at another agency requires notifying the data provider and may require an amendment.

Data environment requirements. Many DUAs specify where data may be stored and processed, for example only on a designated secure server, only within the jurisdiction's network, or only in a HIPAA-compliant environment. "I'll just put it on my laptop for a quick look" is a DUA violation if the agreement prohibits local storage.

Dissemination restrictions. Some DUAs restrict publication (requiring advance review by the data provider), prohibit releasing record-level data, or limit which findings can be publicly reported. Know these terms before drafting a report or dashboard.

Data destruction. Most DUAs require that data be destroyed or returned at the end of the project period, and that destruction be documented. This includes copies, backups, and any data that was transferred for analysis.

25.2.2 Practical Guidance

Get a copy of the DUA before the data arrives. It is much easier to negotiate terms before signing than to discover a constraint mid-project. Loop in your supervisor and, if the terms are unfamiliar or restrictive, your agency's legal or compliance office. When in doubt about whether a specific use is permitted, ask the data provider in writing (email creates a record).

Warning

Data received informally (as an email attachment, a Dropbox link, or a thumb drive) still carries the same governance obligations as data received through a formal DUA process. "We didn't sign anything" does not mean "there are no restrictions." Ask how the data can be used before you use it.

25.3 HIPAA and Protected Health Information

The Health Insurance Portability and Accountability Act (HIPAA) establishes federal standards for protecting the privacy and security of individually identifiable health information. This section is not a comprehensive HIPAA guide (your organization's privacy officer is the right resource for that), but public health data scientists need a working understanding of what HIPAA covers and how it applies to their work.

25.3.1 What HIPAA Covers

HIPAA applies to *covered entities* (healthcare providers, health plans, and healthcare clearinghouses) and their *business associates* (organizations that handle protected health information on their behalf). Many public health agencies are covered entities or business associates; many are not. The rules differ across jurisdictions, and some public health functions have explicit carve-outs under the HIPAA Privacy Rule (disease reporting, for example, is generally permitted without individual authorization).

Protected health information (PHI) is any individually identifiable health information held or transmitted by a covered entity. The HIPAA Safe Harbor method specifies 18 categories of identifiers to remove from health information. The list below groups related categories; consult the [HHS guidance](#) for the full requirements:

- Names
- Geographic subdivisions smaller than a state (including ZIP codes, street addresses, cities in some cases)
- Date elements other than year directly related to an individual; ages over 89 and dates indicating such ages must be grouped as age 90 or older
- Telephone numbers, fax numbers, email addresses
- Social Security numbers
- Medical record numbers, health plan beneficiary numbers, account numbers
- Certificate and license numbers
- Vehicle identifiers (license plates, VINs)

- Device identifiers and serial numbers
- URLs and IP addresses
- Biometric identifiers (fingerprints, voiceprints)
- Full-face photographs
- Any other unique identifying number or code

Whether information is PHI also depends on who holds it and applicable HIPAA exclusions. Ask the privacy officer to assess the dataset.

25.3.2 The Minimum Necessary Standard

HIPAA's minimum necessary standard requires that covered entities use, disclose, or request only the minimum amount of PHI needed to accomplish the intended purpose. For a data scientist, this means requesting only the variables needed for the analysis, not pulling a complete medical record when you only need diagnosis codes and dates.

25.3.3 De-identification

Properly de-identified data is not PHI and is not subject to HIPAA restrictions. HIPAA provides two paths to de-identification:

Safe harbor: Remove all 18 identifiers listed above, plus ensure that the data provider has no actual knowledge that the remaining information could identify an individual. Only qualifying three-digit ZIP codes may be retained; replace other ZIP prefixes with 000 and remove other geographic subdivisions smaller than a state.

Expert determination: A qualified statistician applies generally accepted statistical principles to certify that the risk of identifying any individual is very small. This allows more flexibility than safe harbor (for example, three-digit ZIP codes or specific dates) but requires documented statistical analysis.

Note

De-identification under HIPAA is a formal process, not a judgment call. Simply removing a name is not de-identification. If you are preparing data for release or for use in a less restricted environment, work with your privacy officer to ensure the de-identification is compliant.

25.4 Getting Data Access

Data access in public health institutions involves more parties than most analysts expect. A clear map of who needs to be involved, and what each party needs to hear, prevents the most common delays.

The data steward owns or is responsible for the dataset within the source organization. They can tell you what is in the data, how it is structured, its known quality issues, and what access process is required. For internal datasets (data your own agency collects), the steward is typically in the program that runs the surveillance system. For external datasets (from another state agency, a federal program, or a partner organization), the steward is at the providing organization.

IT controls the infrastructure: server access, database credentials, software installation, network permissions, and VPN. A request to IT should be specific: what software is needed (including version numbers), which database or server needs to be accessed, what level of compute and storage the analysis requires, and whether the data is sensitive or regulated.

Legal and compliance reviews DUAs and data sharing agreements. Loop them in early (DUA review can take weeks at some institutions) and provide the full draft agreement.

Your supervisor can help identify the official authorized to sign DUAs and data sharing agreements on behalf of the agency. They should know about any data access requests that involve legal agreements before those agreements are presented for signature.

25.4.1 Framing Requests

A request framed in terms of program need is easier to prioritize than a vague technical ask. Compare:

"I need access to the vital records SQL Server database."

versus:

“This analysis supports the quarterly overdose mortality report due to the State Health Officer in March. I need read access to the `vr_deaths` table in the vital records database and the ability to install `odbc` and `DBI` in my R environment. The data is governed by our existing MOU with vital records. I need this by February 15 to have time to validate before the report deadline.”

The second version tells IT what the deadline is, what the legal basis is, and exactly what is needed. It also signals that someone outside IT will notice if the request is not addressed.

25.5 Working in Secure Data Environments

When data cannot leave a controlled environment, the analysis has to come to the data. Secure remote desktops and data enclaves impose constraints that require adapting the workflows described elsewhere in this book.

25.5.1 Common Constraints

- No internet access, or access limited to specific approved sites
- No USB or removable media, preventing local file transfer
- Copy-paste disabled between the remote session and the local machine
- No administrator rights, preventing ad-hoc software or package installation
- Output review, meaning no file can leave the environment until it passes a disclosure review

25.5.2 Adapting Your Workflow

Resolve package dependencies before entering the environment. Use `renv` (see Chapter 15.) to capture your project’s exact package list. Submit the lockfile to IT for review; they can pre-install those packages in the secure environment, or arrange an approved intake path. Discovering mid-analysis that a package is unavailable is a significant delay.

Plan the analysis before entering. Write and test your code on a representative mock dataset (public-use data with similar structure, or synthetic data generated from the real data’s structure) before bringing the code into the secure environment. Plan for further exploration inside the enclave when the real data raises questions that mock data could not reveal. The same mock datasets are what make it safe to use an AI coding assistant while you develop (Section 26.4).

Keep code in version control accessible from within the environment. Some enclaves have access to an internal Git server, or allow code to be transferred via an approved intake process. Maintain your code in a repository (see Chapter 1.) so it can be moved in as a single unit.

Document outputs before requesting export. Every output (tables, figures, and reports) that leaves the environment will be reviewed. Organize outputs clearly, understand the suppression rules that apply (see Section 25.6), and apply them yourself before submission. Self-review before the formal review reduces round trips.

25.6 Disclosure Review and Small Number Suppression

Even properly de-identified data can allow re-identification when cell counts are small. A table showing that a specific rural county had 2 opioid overdose deaths among people aged 25–34 may effectively identify those individuals in a small community. Disclosure review is the process of checking whether outputs could allow such re-identification before they are released.

25.6.1 When Disclosure Review Applies

Disclosure review is typically required for:

- Any output leaving a secure data environment
- Outputs derived from restricted-use data, regardless of environment
- Public-facing reports and dashboards based on individual-level surveillance data
- Analyses submitted for publication when the underlying data is restricted

Your DUA and your agency's data governance policies will specify what review process applies. For internally-collected surveillance data, the process is usually internal. For federally restricted data (NCHS, CMS), the providing agency conducts the review.

25.6.2 Small Number Suppression

Suppression thresholds vary by data provider, dataset, and agency policy. Follow the applicable rule, including whether zero counts may be shown. The illustrative rule in Chapter 4. suppresses counts from 1 through 4 and retains zero; it is not a universal public health standard.

Complementary suppression is required when suppressing one cell would allow back-calculation of another. If a row shows a total and several cells, and only one cell is suppressed, the suppressed value can be calculated by subtraction. The fix is to suppress an additional cell (typically the next-smallest) so that the suppressed value cannot be derived. This often requires checking row and column totals as well as individual cells.

i Note

Example: A county reports 7 total influenza deaths. By age group: 25–34: 4 (suppressed, < 5), 35–44: 2 (suppressed), 45+: 1 (suppressed). The total (7) may still be releasable if it is not small, but all three age group cells must be suppressed. If only the first cell were suppressed, the other two would reveal the suppressed value.

In practice, suppression logic can be complex for multi-dimensional tables. Use code or review tools to check suppression, followed by disclosure review.

25.6.3 What Disclosure Review Examines

A reviewer will typically check:

- All cells meet the minimum count threshold
- Complementary suppression has been applied correctly
- Geographic or demographic combinations that are highly specific (a single rural county × a specific age group × a specific cause) do not expose individuals even if each cell alone is above the threshold
- Percentages, rates, and proportions are checked: a rate computed from a suppressed count is itself suppressed

When reporting suppressed data to a lay audience, explain why: “Data suppressed to protect individual privacy” is preferable to an unexplained asterisk or blank cell.

Suppression rules also constrain design decisions upstream of the output. A dashboard permitting users to filter to a single county, a single month, and a single age group can produce a cell below the threshold, whatever the default view displays. Quarterly aggregation may reduce small cells, but it still requires disclosure checks. Choosing the aggregation grain with these thresholds in mind, and applying suppression in code on the way out, is covered in Section 10.2.

26. AI Governance in a Health Agency

Chapter 8. covers the tools: what a completion service does, what an agent can do, which model to select. Agency policy should specify permitted uses, where staff may send data, who reviews AI-assisted analyses, and what records to retain.

Policy and approval requirements can affect when a team can use a tool. Agencies that have stood up formal review processes for AI use of agency data have found that approval timelines affect when modeling work begins. Review cycles measured in weeks are common, and a project that budgets no time for them will lose that time from analysis.

26.1 The Need for a Written Policy

Staff are already using these tools, in many cases on personal accounts and personal devices, because no institutional option was available and the work required doing.

When tools become available before an agency has issued guidance, staff may be unsure what is permitted. Absent guidance, individual staff draw their own lines, and some of those lines are sound. Using a commercial assistant on a personal account to generate code that downloads and merges public, non-identifiable files, while deliberately declining to use it for interpretation, is a defensible position. The agency still needs to state which accounts and uses it authorizes.

A written policy converts individual judgment into an agency position. It protects staff, who otherwise have no way to determine whether their practice is permitted. It gives supervisors something to reference other than instinct. And it means the first difficult question is not answered under pressure during an incident.

26.2 Contents of an Agency AI Policy

A workable policy addresses eight areas.

Scope. Which staff, which systems, and which activities are covered, and whether the policy applies only to generative AI or also to predictive modeling. A policy drafted for chatbots that inadvertently captures every regression model will be ignored.

Permitted and prohibited data classes. Make these instructions easy to find. Name the classes explicitly: public data, internal non-sensitive data, de-identified data, PII, PHI, and any restricted-use data governed by its own agreement. State which classes may be sent to which categories of tool. Vague instructions produce inconsistent practice.

Approved tools and the approval route. A list, and a defined path for adding to it. A policy naming three approved tools without a route for a fourth leaves staff without a process for requesting another tool.

Logging and retention. What is recorded when an approved tool is used, who may see those records, and how long they persist.

Human review requirements. The level of review an AI-assisted output requires before leaving the team, scaled to consequence. Code producing a figure in a published report warrants more review than code that reformats a file.

Procurement and cost ownership. Who funds API usage or seat licenses, what the approval threshold is, and who monitors spend. API billing is consumption-based and carries the same absence of a default ceiling described in Section 17.5.

Incident handling. The procedure when someone recognizes that they have disclosed something they should not have. Make reporting straightforward and handle reports fairly so staff report mistakes promptly.

Review cadence. Tools and terms change, so review the policy regularly. Assign an owner and a review date.

 Tip

The [NIST AI Risk Management Framework \[14\]](#) supplies vocabulary and structure that security and compliance reviewers recognize, and several states have published agency AI policies suitable for adaptation. Adapting an existing policy is faster than drafting one, and adaptation helps identify requirements specific to your agency, as with the team handbook (Section 23.9).

26.3 Formal Review Processes

Agencies establishing a formal review for AI use of agency data generally construct something resembling an institutional review board: a written proposal, a committee, a scheduled meeting, and an approval that gates the work. Proposals of ten pages or more, establishing ownership and responsibility for AI use of data in an agency warehouse, are typical.

Several design choices determine whether such a process functions.

Scale review to risk. Applying IRB-weight review to every use of a coding assistant will either exceed the committee's capacity or be circumvented. A tiered process, with light registration for low-risk uses and full review for anything touching protected data or informing a public decision, is sustainable.

Include methodological expertise on the panel. Review requires relevant technical expertise. Review panels are frequently composed to require a subject-matter expert in the relevant health topic without requiring anyone with expertise in the method under review. A behavioral health expert can assess whether an outcome definition is sensible. Also include someone able to assess model validation, data leakage, and performance measures. A panel lacking that expertise approves and rejects proposals for reasons unrelated to their actual risk.

Publish evaluation criteria in advance. Applicants who know what will be asked submit better proposals, and the committee spends its time evaluating the proposed use.

Set a turnaround expectation. A review with no stated timeline becomes an indefinite hold. Two weeks is achievable for a tiered process. A month per project means programs will stop submitting.

Separate tool approval from use approval. Approving a tool once, for a class of uses, differs substantially from reviewing each analysis. Most agencies need both processes, and conflating them is what makes review slow.

26.4 Protected Data and AI Tools

State which tools, if any, are approved for protected data and explain what counts as sharing. A prohibition on pasting patient records does not address files, console output, screenshots, or other context an assistant may receive.

An assistant may receive sensitive context beyond the typed prompt. Coding assistants read files in the project directory. A data file containing real records in the working directory has been transmitted by an agent that reads it, whether or not anything was typed about it. This is invisible unless one already knows to look for it, which is why it belongs in training and not only in policy.

Debugging is the other frequent route. An error that prints a data frame, a `head()` output pasted into a chat, a traceback carrying values. Debugging is precisely when people paste output without reading it closely, and a screenshot of the IDE sent to ask about a plotting problem carries the contents of the environment pane and console with it. File uploads are the obvious case and warrant explicit mention in the policy because staff ask about them.

Some features transmit data by design. A natural-language query interface built into a data platform runs against real data; that is the product. Whether such use is permitted depends on the terms covering that specific service, which is a separate question from whether the underlying platform is covered (Section 17.8).

Use mock data for development when the assistant is not approved for the real data. Section 25.5 recommends this for secure environments, and the same discipline applies here: build and debug against synthetic or public-use data of the same structure, then execute the finished code against real data in an environment where no assistant is observing. Training and calibration datasets built for onboarding (Section 23.6) serve this purpose directly.

On de-identification, Section 25.3 enumerates the eighteen HIPAA identifiers. Safe Harbor also requires that the covered entity have no actual knowledge that the remaining information could identify a person. See the [HHS de-identification guidance](#). A dataset stripped of direct identifiers can remain effectively

identifying where the combination of county, age, and a rare condition points to one person, which is the re-identification logic underlying small-number suppression (Section 25.6). Data too small-celled to publish is generally too small-celled to paste into a chat.

 Warning

Treat anything transmitted to an external model as potentially persistent, regardless of the vendor's stated retention policy. Zero-retention terms are real and worth negotiating for, and the agreement must specify which services and data it covers. Confirm that the data owner permits the disclosure before transmitting it.

26.5 Cloud and Self-Hosted Inference

A hosted service runs inference on a vendor's infrastructure. A self-hosted deployment runs it on infrastructure your organization controls, which can include leased cloud hardware. Compare the actual model, workload, and controls in each proposal.

For a hosted service, estimate request volume, input and output tokens, and repeated calls during agent sessions. Include subscription fees, usage limits, and the costs of connected services. For self-hosting, include hardware purchase or rental, utilization, electricity, maintenance, security updates, and staff support. Measure response time and answer quality on the same representative tasks. Neither location establishes which option is cheaper or more capable.

An enterprise subscription does not by itself authorize PHI use. Confirm that any business associate agreement covers the specific service and features, and that the data owner permits their use. Retention settings, training policies, regions, and logging are separate checks. Section 8.4 links the API data policies; Chapter 25. covers the approval process.

With self-hosting, verify network access, telemetry, logs, and connected tools before relying on isolation. A locally running model can still send data through a tool or logging service. Record who operates the deployment and maintains its controls.

Check existing agency agreements before starting a procurement. Compare approved options against the workload and data requirements, including staff capacity to support them.

26.6 Transparency in Modeling Tools

Automated machine learning tools differ in what they expose. Some provide candidate models, evaluation metrics, and explanations; [H2O AutoML](#) is one example. An interface alone tells you little about whether an analysis is reproducible or reviewable.

Before using a tool's result, require a record of the input data, preprocessing, candidate models, tuning settings, selection metric, and evaluation design. Check whether the fitted model and its predictions can be exported or reproduced, and whether reviewers can inspect the assumptions relevant to the decision. Reimplement a model when necessary to meet those requirements, and compare predictions to verify the translation.

Use validation data or cross-validation to select and tune models. Evaluate the selected approach on an untouched test set. If you use test results to choose a winner or revise the analysis, those observations have become part of model development; a new evaluation is needed. Repeated tuning and limited data may call for nested cross-validation. Preserve relevant time, facility, or person groupings when defining splits. The [scikit-learn evaluation guide](#) explains these distinctions.

For models informing public health decisions, also examine performance across relevant groups, calibration, and the consequences of errors. Document the reasons for the final choice so a reviewer can assess it independently of the software used.

26.7 Generated Code

Generated code can run without errors and still produce wrong answers. This is what makes accepting generated code without understanding it hazardous: the failure is silent, and these tools are most useful in exploratory analysis, which is exactly where oversight is weakest.

Never accept code you cannot read. If a line is not understood, either learn what it does or do not use it. Understanding the code helps you evaluate it alongside tests and peer review.

Verify against a known result. Execute generated code against data where the answer is already known: a count computed by hand, a previously published figure, a small test case. Chapter 4. covers building this verification into the project so it runs automatically.

Review it as ordinary code. Section 27.4 covers using AI as a review aid. The reciprocal case matters more: generated code belongs in the same peer review as hand-written code, and reviewers benefit from knowing which is which. Ask the author how they verified the code, including edge cases. Neither writing a function by hand nor accepting a generated version establishes correctness.

Certain categories warrant particular attention because generated code is confidently wrong in them: date and time zone handling, joins that silently drop or duplicate rows, aggregations that disregard missing values, and any statistical method where a consequential assumption is embedded in a default argument. Chapter 3. exists for these failures.

i Note

Translating legacy SAS programs is among the strongest applications of these tools and among those where verification matters most (Section 19.4). A model can translate a thousand-line SAS program in minutes. Whether it translated the missing-value semantics correctly is answered only by parity testing (Section 19.5).

26.8 Reproducibility

A prompt is not a method section, and an analysis that cannot be reconstructed is not defensible regardless of how it was produced.

The reproducibility standard does not change because a model was involved. The final artifact remains code in version control that runs from raw data to result (Chapter 6.). Where an assistant wrote the code and the code is committed, reviewed, and reproducible, the code provides a reproducible record; follow any additional agency or publication requirements for disclosure and logging.

The case that differs is where the analysis uses model outputs directly. Where a model categorized free-text records, selected variables, or produced a reported figure, reconstructing that result later requires more than the code. Record the model and version, the prompts, the date, any parameters set, and what was changed in the output. Keep nonsensitive configuration and decision records in version control (Section 21.4). Store sensitive prompts, model outputs, and logs in approved storage with appropriate access controls and retention rules. Put a nonsensitive record identifier or approved storage reference in the repository; do not copy protected content into it.

Two caveats apply. Models are not deterministic by default, so complete records do not guarantee an identical rerun; they provide an auditable account of what occurred. Model versions are also retired. Archive model outputs used by recurring surveillance products and verify them before use. If a recurring product depends on model output, document how the team will monitor the output and replace the model when its behavior changes or its version becomes unavailable.

26.9 Staff Capability

Training is the question that follows policy, and general AI literacy is a better investment than tool-specific instruction.

Tool-specific training becomes obsolete on the vendor's release schedule. What does not become obsolete is understanding what these models do, why they produce confident errors, what a context window is and why it constrains results, the difference between a model and a product, and how to evaluate a performance claim. An analyst with that understanding can adopt any tool; an analyst trained on one interface requires retraining when it changes.

The second capability worth developing is the judgment to recognize when not to use these tools. This is harder to teach than the tools themselves, and it is what a policy ultimately depends on, since no written rule anticipates every case. Section B.1 covers where this fits within a training budget.

26.10 Further Reading

- The [NIST AI Risk Management Framework \[14\]](#) is the reference document security and compliance reviewers are most likely to recognize. Its four functions (govern, map, measure, manage) supply a policy's structure, and the companion playbook contains concrete suggested actions.

- The Office of Management and Budget issues memoranda governing federal agency use of AI. These are revised periodically and by administration, so check the current [OMB memoranda index](#). State and local agencies are not bound by them, and they shape expectations that reach agencies through federal partners and cooperative agreements.
- HHS publishes AI strategy and guidance specific to health data, and several states have published agency AI use policies. Both are worth reviewing as templates before drafting.
- Chapter 25. covers the underlying data governance on which all of this rests: data use agreements, HIPAA, secure environments, and disclosure review. An AI policy cannot override restrictions in a data use agreement.

27. Peer Review for Data Analysis

Peer review helps catch errors in analysis code, methods, and interpretation before the results inform a decision. It also gives colleagues a chance to learn how the analysis works and share responsibility for its quality.

This chapter covers what analytical peer review looks like in practice, how to structure a review, and the tools that make it easier, including AI as a review aid.

27.1 Scope of Analytical Peer Review

Peer review in this context means having someone other than the original analyst read and evaluate the analysis before its results are communicated. This is different from:

- Software code review, which focuses on correctness, security, and maintainability of code that runs in production. Analytical code has different concerns: Does the question match the data? Are the methods appropriate? Are the results interpreted correctly?
- Statistical peer review in the academic sense, where a journal reviewer evaluates methodological rigor. Internal review can be narrower and faster, with depth matched to the consequences of error.
- Proofreading a report, which catches writing problems but misses analytical ones. A clearly written report can present a wrong answer clearly.

A good analytical peer review evaluates three things:

1. Correctness of the code: Does the code do what the analyst says it does? Are there off-by-one errors, filter inversions, or joins that silently drop records?
2. Appropriateness of the methods: Is the analysis designed to answer the question being asked? Are there obvious alternative interpretations the analyst did not consider?
3. Validity of the conclusions: Do the stated findings follow from the results? Are uncertainty and limitations communicated accurately?

None of these require the reviewer to rerun the analysis from scratch, though doing so is valuable when stakes are high.

27.2 Code Review Mechanics

The practical vehicle for analytical peer review is a pull request (see Section 1.2). When an analyst finishes a piece of work, they open a pull request from their branch to the main branch. The reviewer reads the diff, leaves comments, and either approves or requests changes before the code is merged.

This workflow has several advantages over emailing scripts back and forth or reviewing a report after the fact:

- The review is tied to the specific changes that were made, not a reconstruction from memory.
- Comments are attached to specific lines of code, making feedback precise.
- The history of the review (what was flagged, what was addressed) is preserved alongside the code.
- Analysis results are not incorporated into a deliverable until the review is complete.

27.2.1 What to Look for in a Code Review

When reviewing analytical code, focus on:

Data handling

- Are filters applied correctly? A common error is filtering to the wrong values (e.g., keeping `status != "active"` when the intent was to keep only active records).
- Are joins producing the expected number of rows? Silent duplications or dropped records from a many-to-many join or a misspecified key are among the most consequential errors in analytical work.
- Is missing data handled explicitly, or silently dropped? If dropped, is that appropriate?
- Are date ranges and cutoffs correct? Off-by-one errors on dates are easy to introduce and easy to miss.

Calculations

- Do aggregations (sums, means, rates) match what is described in the analysis?
- Are rates calculated with the correct denominators? Are numerator and denominator from the same population?
- Are any constants hard-coded where they should be calculated from the data?

Outputs

- Do the summary statistics match reasonable expectations for the data? A mean that seems implausibly high or low is worth investigating.
- Do the plots match what the code is computing? A chart labeled “rate per 100,000” should be computing and displaying a per-100,000 rate.
- Are the conclusions stated in the narrative consistent with the numbers in the output?

27.2.2 Giving and Receiving Feedback

A review is useful only if feedback is acted on, and feedback is acted on only if it is received well. A few principles that help:

For reviewers: Be specific about what you are flagging and why. “This join might be producing duplicates: there are multiple rows per `case_id` in `contact_table`, which means a left join on `case_id` will multiply the case rows. Can you verify the row count before and after?” is more actionable than “check the join.”

For analysts: Investigate review comments and explain your reasoning. The reviewer may be wrong; if so, a short explanation resolves it. But the reviewer may have caught something real and the fact that the analysis worked correctly in a different case does not mean it works correctly in this one.

For teams: Make review a routine part of analytical work. A team where review is standard for all analysis will catch errors earlier and distribute knowledge more evenly than one where review is triggered only by anxiety.

27.3 Checklists for Analytical Review

Checklists help reviewers maintain consistency and avoid the bias of reading the analysis looking to confirm that it is correct. Reviewing analytically requires some skepticism, checking how the code could fail.

A basic checklist for most analytical peer reviews:

Data

1. The data source is correctly identified and loaded
2. The date range and population scope match the stated analysis parameters
3. Filters are correctly specified and applied in the right order
4. Missing values are accounted for
5. Join keys are unique (or duplicates are intentional and handled)
6. Row counts at key steps are plausible

Analysis

1. Numerators and denominators are correctly specified for any rates
2. Grouping variables match the level of the desired analysis
3. Any statistical tests or models use methods appropriate for the data and question
4. Results match expectations from prior knowledge of the data

Outputs

1. Chart labels, titles, and units are correct
2. Summary tables match what the code computes
3. Stated conclusions follow from the results
4. Uncertainty and limitations are acknowledged appropriately

Not every item applies to every analysis. For a simple descriptive summary, several of the statistical items are irrelevant. For a complex linked dataset with multiple joins and rates, the data section deserves more attention. Use judgment.

27.4 AI as a Review Aid

AI coding tools (see Chapter 8.) can supplement but not replace human peer review. They are useful for certain review tasks and unreliable for others.

The reciprocal case matters too: code produced by these tools should go through the same review as code written by hand, and reviewers benefit from knowing which is which. Section 26.7 covers that side, along with the categories where generated code tends to be confidently wrong.

Where AI tools add value in review:

- Explaining what a section of code does, in plain language, so a reviewer can quickly verify that the code matches the analyst's stated intent
- Identifying common patterns that are likely to be errors (filter inversions, joins without explicit key specification, implicit coercions)
- Checking that variable names, column references, and function arguments are used consistently
- Generating a summary of what changed between versions of a script

Where AI tools are not reliable:

- Verifying that the analytical methods are appropriate for the question (this requires domain knowledge)
- Detecting errors that require understanding the real-world data (e.g., knowing that a particular jurisdiction stopped reporting in 2021, so a sudden drop is a consequence of missing reports)
- Evaluating whether conclusions are well-supported (this requires both analytical judgment and knowledge of the context)

A practical workflow: use Posit Assistant (see Section 8.2) or Claude Code (see Section 8.3) to review a script for mechanical issues (does the logic match the description, are there obvious error patterns) and then have a human reviewer focus on whether the analysis is asking the right question and drawing the right conclusions from the results.

Note

AI tools will sometimes confidently identify a “problem” that is not actually a problem. Investigate AI-generated review comments before accepting them. The human reviewer is responsible for the review.

27.5 High-Stakes Analyses

Routine analyses, i.e. a weekly case count report, a descriptive summary for an internal meeting, benefit from light-touch review. Analyses that inform significant decisions warrant more thorough review.

Indicators that an analysis warrants a higher review standard:

- The results will be used to allocate resources or change programs
- The results will be released publicly or to the press
- The analysis involves a method the team has not used before on this data
- The analysis contradicts prior findings and there is no obvious explanation

For high-stakes work, consider independent replication: a second analyst replicates the key computations independently and compares results. Discrepancies must be resolved before the analysis is finalized. This is more expensive than a standard review but is warranted when errors would have significant consequences.

27.6 Building a Review Culture

Agree on review expectations as a team. Teams that review well share a few characteristics:

Review is not optional for consequential work. There is a clear, shared expectation that analyses above a certain threshold of importance are reviewed before results are communicated.

Review is timely. A review request that sits for two weeks creates pressure to skip it. Teams that review quickly, i.e. same or next business day for most requests, make time for review in the workflow.

Reviewers are recognized for good reviews. Catching an error that would have been embarrassing is a contribution. Teams that treat review as invisible overhead underinvest in it.

Review is a learning mechanism. Use review to help analysts improve their work. Analysts who receive good reviews get better. Reviewers who read a lot of others' code develop a broader perspective on the team's methods and practices.

28. Communicating Findings

A technically correct analysis that does not reach its audience (or reaches them in a form they cannot act on) has not done its job. In public health, findings inform decisions made by program directors, health officers, elected officials, and practitioners who did not run the analysis and may not read a methods section. Getting from rigorous analysis to useful communication is its own skill, and one that is easy to underinvest in when the pressure is on to finish the analysis itself.

Section 24.5 introduces the core principles: know your audience, communicate uncertainty honestly, match the output format to the need. This chapter goes deeper on each, with concrete guidance on structure, visualization, and how to handle the cases that are hardest to communicate well.

28.1 Lead with the Finding

The most common failure mode in communicating analysis results is burying the finding. Reports structured like academic papers (Introduction, Methods, Results, Discussion) force busy decision-makers to read through to the end to learn what the analysis concluded. Many do not make it that far.

Lead with the finding, then provide the supporting evidence and context.

Compare these two openings for a hypothetical analysis:

Version A: “Overdose mortality in the state has been a growing public health concern. This analysis used death certificate data from the National Vital Statistics System for 2015–2023 to examine trends in overdose deaths by county and age group. The results showed that rates in rural counties increased substantially.”

Version B: “Overdose death rates in rural counties increased 34% from 2019 to 2023, compared to a 22% increase in urban counties, a change from 2015–2019, when rates increased faster in urban counties. The shift is driven primarily by illicitly manufactured fentanyl, which first appeared in rural county toxicology reports at scale in 2020.”

Version B leads with what changed, by how much, and what is driving it. A health officer who reads only the first two sentences of Version B has the main finding. Version A makes them work for it.

28.1.1 The “So What” Test

After stating a finding, ask: *So what?* What should a program manager, health officer, or policymaker consider doing differently because of this finding? The analysis team often knows the answer but leaves it unstated, assuming the audience will draw the right implication. They often do not.

A finding without an implication is incomplete:

Finding only: “Rural overdose death rates increased 34%.”

Finding with implication: “Rural overdose death rates increased 34%, and rural counties currently have significantly less harm reduction infrastructure per capita than urban counties, suggesting that expanding naloxone distribution and syringe services to rural areas is a priority intervention point.”

This does not mean advocating for a specific policy. It means stating what the data implies for programmatic decisions. The program can decide what to do with that implication; the analyst’s job is to make it explicit.

28.2 Audience and Format

Different stakeholders need different outputs from the same underlying analysis. Plan to produce more than one. The analysis runs once; the communication takes multiple forms.

28.2.1 Decision-Makers

Health officers, board members, program directors, and elected officials typically need:

- The bottom line first (what is the finding and what does it imply?)
- One or two key numbers, not a table of 40 statistics
- Uncertainty stated plainly in language they can repeat to others
- A clear sense of what decision or consideration the finding should inform
- One page, or one slide

They do not need methods details unless they specifically ask. If the methods are rigorous, say so briefly (“These findings are based on complete death certificate data; preliminary estimates from provisional data are consistent with these results”) and move on.

28.2.2 Program Staff

Staff doing day-to-day work (case investigators, disease intervention specialists, program coordinators) need findings they can act on. The most useful outputs for this audience:

- Are specific enough to change what they do tomorrow
- Include enough supporting data that they can answer follow-up questions from the people they work with
- Are formatted for reuse in their own presentations and communications

A table they can sort, a chart they can copy into a slide, or a brief they can forward to a partner agency is often more valuable than a polished report they cannot extract from.

28.2.3 Technical Peers

Epidemiologists, biostatisticians, and methodologically sophisticated collaborators need the full picture: data sources, methods, sensitivity analyses, limitations, and honest uncertainty. For this audience, the concerns are whether the analysis was done correctly and whether the conclusions are defensible, not whether the one-page summary is clear.

When sharing work with technical peers for review (see Chapter 1. for making code available), include the analysis code alongside the report. A methods section that says “code available at [repository]” is more credible than one that does not.

28.2.4 An Executive Summary Template

For findings that need to reach multiple audiences, a structured one-page executive summary gives decision-makers what they need while pointing technical readers to the full report. A minimal template:

SUMMARY

[One to two sentences: the main finding and its most important implication.]

KEY FINDINGS

- [Finding 1, stated plainly, with the key number]
- [Finding 2]
- [Finding 3]

IMPLICATIONS

[One to two sentences on what the findings suggest for programs, policy, or practice. Frame as considerations, not mandates.]

DATA AND METHODS

[One sentence: data source, time period, geographic scope, population.]
Full methods available at [link or report title].

The summary fits on one page. The full report follows. Decision-makers read the summary; technical reviewers read both.

28.3 Choosing the Right Output

Choose a format that helps the intended audience use the findings.

Situation	Best format
One-time finding for a decision	Written brief or slide deck
Ongoing data that updates regularly	Parameterized report (Chapter 6.) or dashboard (Chapter 9.)
Data that stakeholders need to explore	Interactive dashboard
Findings for public release	Report with executive summary
Technical findings for peer review	Full report with methods appendix and code
Data that will be used in others' presentations	Table or chart files, clearly labeled

A few questions that sharpen the choice:

Will the audience consume a summary or explore the data? If the finding can be stated in a few sentences and a chart, a brief is right. If stakeholders will want to slice by county, time period, or demographic group, a dashboard lets them do that without requiring a new analysis each time.

Is this a one-time analysis or will it recur? If the analysis will be repeated monthly or annually with updated data, building a reproducible, parameterized report (see Chapter 6.) costs more upfront and dramatically less over time. If it is truly one-time, a polished static output is fine.

Will this be printed? HTML dashboards and interactive reports do not print well. If physical copies or PDFs are a requirement, design for that from the start.

28.4 Visualizing to Communicate

Visualization for communication is different from visualization for exploration. Exploratory charts help you understand data; communication charts help an audience understand a specific finding. The design goal shifts from “show me what is in this data” to “make this one finding impossible to miss.”

28.4.1 Match Chart Type to Message

Message	Chart type
Compare values across categories	Horizontal bar chart (especially with many categories)
Show change over time	Line chart
Show a part of a whole	Stacked bar, waffle chart; avoid pie charts with more than 3 slices
Show a relationship between two variables	Scatter plot
Show geographic distribution	Choropleth map (with caution, since land area ≠ population)
Show a single number	Large text, value box

No chart type is universally wrong, but some common combinations fail reliably:

- Pie charts with many slices: humans cannot judge angles accurately; a bar chart is almost always clearer

- Dual y-axes: charts with two y-axes are nearly always misleading because the visual relationship between the two lines depends entirely on how the axes are scaled, which the designer chooses
- 3D charts: the added dimension carries no data but introduces distortion; never use them
- Truncated y-axes: starting a bar chart's y-axis above zero exaggerates differences; if differences are small, describe their size in text

28.4.2 Make the Finding Clear

A communication chart has one job: make the finding clear. Every element that does not support that job should be removed.

Highlight what matters. Use color, annotations, or callout lines to draw attention to the specific element that supports the finding. A line chart showing trends for 50 states, all in gray, with a single state highlighted in the primary color, is far more effective than 50 differently-colored lines competing for attention.

Label directly. Annotations on the chart (for example, “34% increase” at the relevant data point, or a text label at the end of each line) are clearer than legends the reader has to match to the chart. If the chart requires a legend to interpret, consider whether direct labeling is possible.

Use accessible colors. Approximately 8% of men have color vision deficiency. Red/green combinations are the most common problem. Colorblind-friendly palettes (such as the Okabe-Ito palette used in some ggplot2 themes) are safe choices. Check any palette at a tool like [Coblis](#) before finalizing. Section 7.3 covers contrast ratios and the requirement that color never be the only channel carrying information.

Remove decoration that carries no information. Heavy gridlines, background colors, borders, and 3D effects subtract from clarity. The data should be the most visually prominent element on the chart.

28.5 Communicating Uncertainty

Explain uncertainty in terms the audience can use, without overstating precision or obscuring a clear finding. Section 24.5 introduces this; what follows is more specific guidance.

28.5.1 Natural Language Over Statistical Formalism

Decision-makers who did not take a statistics course will not correctly interpret “95% confidence interval.” Many will read a CI as a margin of error or, worse, as the probability that the true value falls in the range. Natural language is clearer:

Avoid: “The estimated increase was 34.2% (95% CI: 28.1%–40.3%).”

Prefer: “We estimate the increase was between 28% and 40%, with our best estimate at 34%.”

Or: “We estimate the increase was roughly 34%, though the true figure could reasonably be as low as 28% or as high as 40%.”

When explaining what drives the uncertainty, be specific: “The estimate is imprecise because few deaths occurred in this county” is more useful than “the confidence interval is wide” or “ $p = 0.07$.”

28.5.2 Null Results Are Findings

A nonsignificant result can be consistent with both small and practically important effects. Report the estimate and interval, and state what difference would matter for the decision. Statistical significance does not measure effect size or practical importance, as the [ASA statement on p-values](#) explains.

For a hypothetical comparison of vaccination coverage, suppose the team specified a five-percentage-point difference as practically important before analysis:

“Estimated coverage was 2 percentage points higher in group A than in group B, with a 95% confidence interval from 3 points lower to 7 points higher. This interval includes both no difference and

an increase exceeding our five-point threshold. The data do not resolve whether the groups differ by an amount that would change the program decision.”

If discussing power, name the assumed effect size and the probability of detecting it under the study design. Power is not a cutoff below which detection is impossible. A claim of equivalence needs a prespecified equivalence margin and an analysis that supports it.

28.5.3 Preliminary and Provisional Data

Provisional counts can change as reports arrive and records are revised. Name the data product, extraction date, and period covered. Use the provider’s completeness estimates or a documented comparison of successive releases to describe likely revisions. Do not assume reporting delays are the same across periods or groups.

A reusable statement with values to fill from that evidence:

“These are provisional [data product] counts retrieved on [date], covering [period]. In [documented comparison periods], counts changed by [range] between releases [specified time apart]. Completeness for the current period is [assessment]. We will reassess the apparent trend after [next release or completeness criterion].”

If revision evidence is unavailable, say that its direction or magnitude is unknown. Avoid assuring readers that a trend will persist unless you have checked its sensitivity to incomplete reporting.

28.5.4 Data Quality Problems

When data has known quality issues (a reporting system that changed, a year with underreporting, a jurisdiction that did not report), be transparent about them and explain how they affect the finding.

A pattern that holds despite a known data quality problem is stronger evidence than one that depends on a single year or jurisdiction. Point that out explicitly:

“Reporting completeness improved in 2021 after the new case management system was deployed, which contributes to the apparent increase that year. The trend from 2022 onward, when reporting was stable, shows a continued increase of approximately 8% per year.”

28.5.5 Suppression

When cells are suppressed due to small counts (see Section 25.6), explain why in plain language:

“Data for counties with fewer than 5 deaths in a given year are not shown to protect individual privacy.”

If the suppression is consequential (meaning the reader might draw incorrect conclusions from the visible cells because suppressed cells would change the picture), say so:

“Several small rural counties are not shown due to small counts, but their inclusion would not change the overall trend.”

Or, if suppression does change the picture:

“Data for several small counties are suppressed. Statewide totals include these counties; county-level rates for the suppressed jurisdictions are not available.”

Appendices

References

A. Additional Resources

This appendix points to external resources for topics that go deeper than what this book covers. The selections here are oriented toward public health practitioners building data science skills in R, with emphasis on free and openly available material.

Nearly everything listed here is free. Appendix B. covers the separate question of what to buy when you have a professional development budget and a deadline.

A.1 Learning R

R for Data Science (Wickham, Çetinkaya-Rundel, and Grolemund) is the standard starting point for learning the tidyverse. It covers data import, transformation with `dplyr`, visualization with `ggplot2`, and the broader workflow of doing data analysis in R. The second edition is freely available online. Most DSTT participants will benefit from working through at least the first half.

What They Forgot to Teach You About R (Bryan and Hester) covers working in R effectively: project-oriented workflows, path discipline, R startup files, and the habits that separate reproducible work from fragile one-off scripts. Short and practical.

The tidyverse style guide documents naming conventions, spacing, and code organization conventions used across the tidyverse. Following a consistent style makes code easier to read, review, and maintain. The `styler` package can apply many of these rules automatically.

A.2 Going Deeper in R

Advanced R (Wickham) covers R's object systems, functional programming, metaprogramming, and performance. It is aimed at people who already use R regularly and want to understand why things work the way they do, write more expressive code, and debug problems that go beyond "my package isn't loading."

R Packages (Wickham and Bryan) is the definitive guide to writing R packages. A package is the right container for analysis code that needs to be shared across projects or team members. It bundles functions, documentation, and tests in a standardized way that makes everything easier to reuse. Even teams that do not plan to publish to CRAN benefit from the structure packages impose. This book walks through the complete workflow using `devtools` and `usethis`.

Mastering Shiny (Wickham) covers building interactive web applications with Shiny. Shiny apps require a running R process (unlike Quarto dashboards) but support interactivity such as user-driven filtering, on-the-fly modeling, custom inputs. This book covers reactive programming, app architecture, and deployment.

A.3 Visualization

ggplot2: Elegant Graphics for Data Analysis (Wickham, Navarro, and Pedersen) goes beyond the basics of `geom_*` functions to cover the grammar of graphics underlying `ggplot2`, how to extend it, and how to build custom themes and scales. Useful for teams that produce a lot of publication-quality figures and want consistent, polished output.

R Graphics Cookbook (Chang) is organized as a collection of recipes: a problem followed by a solution. Less conceptual than the `ggplot2` book, more useful as a quick reference when you know what you want a plot to look like but not how to produce it.

Storytelling with Data (Knaflitz) is about the decisions upstream of the plotting code: what to show, what to leave out, and how to structure a chart so the audience reaches the point you intended. Tool-agnostic, and the closest thing to a standard reference for the design side of Section 28.4. The companion exercise book provides the practice.

A.4 Modeling and Statistics

Tidy Modeling with R (Kuhn and Silge) is the companion book to the [tidymodels](#) ecosystem, a collection of packages that bring tidyverse conventions to statistical modeling: splitting data, specifying model workflows, tuning hyperparameters, and evaluating performance. Tidymodels works with hundreds of model types through a consistent interface. The book and the tidymodels website are the primary resources for teams moving from exploratory analysis into predictive modeling.

A.5 Public Health Data Science

The Epidemiologist R Handbook is a comprehensive, freely available reference manual written specifically for epidemiologists and public health practitioners using R. It covers data management, descriptive analyses, outbreak investigation, time series, spatial analysis, and reporting, with worked examples using realistic public health data. For DSTT teams, this is the most directly applicable reference for day-to-day analytical work.

Building Reproducible Analytical Pipelines with R (Rodrigues) covers functional programming, package-based project structure, Docker, and Nix as a complete framework for building analyses that can be re-run reliably. It connects topics from the environments chapter (Chapter 15.) and goes substantially deeper on each.

A.6 Databases from R

Many agencies store surveillance data in relational databases such as SQL Server, PostgreSQL, Oracle, or SQLite. R can query these databases directly, returning results as data frames without any manual export step.

Posit's database guide is the best starting point. It covers the overall architecture (DBI as the connection layer, specific driver packages per database type), connection setup in Positron and RStudio, and best practices for credentials and connection management.

DBI is the foundational package. It defines the interface that all R database drivers implement: `dbConnect()` to open a connection, `dbGetQuery()` to run a SQL query and return results, `dbDisconnect()` to close the connection. If you know the SQL you want to run, DBI is the direct path.

dbplyr extends dplyr to work against database tables. Instead of writing SQL, you write normal dplyr code (`filter()`, `mutate()`, `summarize()`) and dbplyr translates it to SQL and runs it on the database. This is useful when the team knows dplyr well and the database queries are not highly complex. `collect()` pulls the results into R as a local data frame.

A.7 APIs and Public Data

httr2 is the package for talking to web APIs from R, introduced in Chapter 18.. Beyond the basics covered there, its documentation includes a “Wrapping APIs” vignette that walks through building a reusable interface to an API, useful if your team pulls from the same source in many projects.

Analyzing US Census Data (Walker) [10] is the definitive guide to working with census data in R via the `tidycensus` package: ACS and decennial data, geography and mapping, and the margins-of-error handling that census estimates require. Freely available online. For public health teams, this is where rate denominators come from.

The Socrata developer documentation covers the API behind `data.cdc.gov` and many state open data portals, including the SoQL query language for filtering and aggregating on the server.

A.8 Data Infrastructure

The DuckDB documentation is unusually good, and the sections on reading Parquet, the CSV sniffer, and performance tuning cover most of what a public health team needs. [9] is the original paper if you want the design rationale for columnar analytical engines, which is the background to Section 17.3.

MotherDuck documents the hosted, shared version of DuckDB, which is the smallest step up from a local file when several people need to query the same data.

The FinOps Foundation publishes vendor-neutral material on managing consumption-based cloud spend. Most of it is written for organizations far larger than a health department, and the core practices

(attributing cost to a team, setting budgets with alerts, right-sizing compute) apply directly to the pricing problem described in Section 17.5.

A.9 BI Platforms

[The Tableau REST API reference](#) documents authentication, publishing, and extract refresh, which are the endpoints worth automating from R (Section 10.3). The vendor documentation is the only reliable source here, because these APIs version independently of any R wrapper.

[The Power BI REST API reference](#) covers dataset refresh and workspace management. The [Fabric REST API](#) documents additional Fabric services.

A.10 Package Dependencies

[Dependencies: Mindset and Background](#) in *R Packages* explains the costs and benefits of package dependencies, complementing Section 13.1.

[The lifecycle package documentation](#) explains what each stage badge (stable, experimental, superseded, deprecated) commits a maintainer to, which is what you need to interpret a function's badge before building on it.

[pak](#) installs packages and, more usefully here, shows dependency trees with `pkg_deps_tree()` before you commit to anything.

A.11 Testing and Validation

[The testthat documentation](#) and the [testing chapters of R Packages \[7\]](#) cover the framework introduced in Chapter 4., including test organization, fixtures, and snapshot testing.

[pointblank](#) is the validation package used in Chapter 3.. Its documentation covers agents, action levels, and the multi-agent reporting that makes validation results reviewable by non-programmers.

A.12 Automation and Reproducible Environments

[The targets user manual \[4\]](#) is the reference for the pipeline tool introduced in Section 11.4. The chapters on debugging pipelines and branching over many inputs are useful for recurring surveillance work.

[The Rocker Project](#) documents the Docker images used in Section 15.2.3, including which image to start from and how the versioned tags work.

[renv documentation](#) covers lockfiles, restoring environments, and the cache, in more detail than Section 15.1.

A.13 Accessibility

[The WCAG 2.2 quick reference](#) is the practical, filterable version of the standard behind the requirements in Section 7.1. Filter to Level AA and to the guidelines relevant to your output type.

[The Section508.gov training materials](#) are written for federal staff and cover documents, spreadsheets, and presentations.

A.14 AI Tools and Governance

[The NIST AI Risk Management Framework \[14\]](#) and its companion playbook give an agency AI policy a structure that compliance reviewers recognize. The starting point for Section 26.2.

[The OMB memoranda index](#) is where current federal guidance on agency AI use lives. These are revised by administration, so check the index for current guidance.

[The Posit Assistant documentation](#) and the [Claude Code documentation](#) cover the tools in Chapter 8. in more depth than that chapter does, including configuration, permissions, and what context each tool sends.

A.15 Team Practice and Onboarding

The GitLab Handbook documents how GitLab operates. Its scope exceeds a small epidemiology team's needs, but its structure and maintenance practices provide useful examples (Section 23.10).

The Turing Way is a community-built guide to reproducible, ethical, collaborative data science. Its guides on project design, collaboration, and communication overlap substantially with the nontechnical part of this book and go deeper on several topics.

The Carpentries publish workshop curricula on R, the shell, Git, and SQL that a team can work through together. The lessons are openly licensed, so they can be adapted for your own onboarding material.

A.16 Migrating from Legacy Tools

R for Excel Users (Lowndes and Horst) is a free, workshop-format course aimed at analysts whose current workflow is Excel, covering the transition to R, RStudio, and reproducible project habits. A good group-study resource for teams working through the migration described in Chapter 19.

haven reads SAS, SPSS, and Stata files into R, preserving value labels and special missing values. Its documentation covers the details of labelled data that matter when converting legacy datasets to open formats.

A.17 Online Courses

The **Johns Hopkins Data Science Specialization** on Coursera is a 10-course sequence by Roger Peng, Jeff Leek, and Brian Caffo covering the complete applied data science workflow in R: tooling, data acquisition, exploratory analysis, reproducible research, statistical inference, regression, machine learning, and building data products. Check each course's current access options; preview, audit, and certificate terms vary. Public health practitioners may find these individual courses useful:

- **R Programming**: the core language, data structures, control flow, and functions. A solid foundation if you are new to R or coming from another language.
- **Exploratory Data Analysis**: visualization and summarization as an investigative tool, not a reporting tool.
- **Reproducible Research**: the rationale and practice of literate programming in R, closely aligned with the approach in Chapter 6.

The **Johns Hopkins Executive Data Science Specialization** (also by Peng, Leek, and Caffo) is a shorter, less technical sequence aimed at managers who lead or commission data science work. It pairs well with Chapter 24, and is useful for team leads who want to communicate more effectively with analysts.

A.18 Version Control

Happy Git and GitHub for the useR (Bryan) is the companion reference to Chapter 1. It covers installation and configuration in detail, the full range of workflows for collaborating on GitHub, and a substantial troubleshooting section for the inevitable moments when Git does something unexpected. Useful when you want to move beyond the introductory workflow.

A.19 Quarto

The **Quarto documentation** at quarto.org is comprehensive and well-maintained. Key sections beyond what this book covers:

- **Quarto Projects**: organizing multiple documents with shared configuration, a prerequisite for generating many parameterized reports at once.
- **GitHub Actions for Quarto**: automating rendering and publishing on push, so reports update without manual intervention.
- **Quarto Extensions**: community-contributed output formats, shortcodes, and filters that extend what Quarto can produce.

B. Spending a Training Budget

Professional development funds (both from DSTT and possibly from your employer) typically have a deadline attached and sometimes little guidance. The common outcome is that a team identifies what it wants in the final weeks, discovers a procurement constraint it did not know about, and either buys whatever can be approved quickly or returns the money.

Plan training purchases around the team's needs and the procurement deadline. Section 22.3 covers staff development, and Appendix A. lists free material to consider alongside paid options.

Tip

These guidelines are written for a general public health practitioner with a training budget looking to continue upskilling and professional development, and not necessarily guidelines specific for the DSTT training budget. Check with DSTT program staff for any DSTT-specific guidance.

B.1 Principles

Start with the skills the team needs for its current work.

Buy skills that transfer. SQL, R fundamentals, and version control will remain useful in five years, and will remain useful if the agency replaces its BI platform, its warehouse, or its surveillance system. Training in a particular interface may need updating when the product changes.

The same reasoning applies to AI training. A general course covering how these models work, what they do well, and where they fail will age better than instruction in the prompt syntax of whichever assistant the agency licensed this year. Chapter 26. explains why understanding the method matters more than knowing the interface.

Choose training that includes practice. For SQL, look for exercises in which learners write queries and inspect the results.

B.2 Skills Worth Buying

SQL comes first. It is chronically underrated and becomes unavoidable as agencies move surveillance data into warehouses (Chapter 17.). An analyst who can only receive extracts depends on whoever writes them, which adds days to every ad hoc question. Include hands-on practice.

R fundamentals follow, specifically functions. Most public health analysts learn R as a sequence of scripts executing top to bottom. The transition from scripts to functions is what makes testing (Chapter 4.), packaging (Chapter 14.), and substantive code review (Chapter 27.) possible, and it helps analysts reuse and test their work.

Version control comes next. It is inexpensive to learn relative to how much it changes a team's working practice (Chapter 1.).

Data visualization and communication follow, including selecting a visual encoding and structuring a message (Chapter 28.). Start with the free reproducible reporting documentation before paying for additional instruction. General AI literacy completes the list.

B.3 Courses and Subscriptions

Public-health-specific providers. [Applied Epi](#) offers Intro to R for Applied Epidemiology and related courses taught by epidemiologists, with examples such as line lists and case counts. CSTE offers training tied to its own programs. These catalogs are narrow. Where an agency restricts purchases to public-health-approved content, the GIS or Shiny course a team actually wants may be unavailable, so check eligibility before selecting a course.

General platforms. [DataCamp](#) is strong for hands-on SQL and R practice in a browser sandbox requiring no local installation, which can help when staff cannot install software locally. Check current

prices and whether the advertised rate requires annual billing. [Coursera](#) hosts the Johns Hopkins Data Science Specialization and the Executive Data Science sequence, both described in Appendix A.. Udemy and Codecademy are inexpensive and uneven; evaluate the specific course you intend to use.

R-focused training companies. [Jumping Rivers](#) and comparable firms sell instructor-led private training. The per-seat cost is high, and a single-day workshop delivered to an entire team can represent better value than five individual subscriptions, with content built around the team's own data.

Free and community options. [The Carpentries](#) publish workshop curricula on R, Git, and the shell that a team can work through independently. [Workshops for Ukraine](#) sells individual sessions on specific topics for a small donation and maintains a deep back catalog. [The Epidemiologist R Handbook](#) is free and is the most directly applicable R reference for this audience.

Conferences. [posit::conf](#) offers R and Python sessions for practitioners, CSTE's annual conference offers opportunities to meet other public health practitioners, and vendor summits are useful primarily to organizations already committed to that vendor. Include registration, travel, and staff time when comparing conference costs. Where the choice is one conference trip for one person against a year of hands-on training for the team, the training is the better purchase.

💡 Tip

Before purchasing a subscription, determine whether the state health department, a university partner, or the library system already holds an enterprise license. Several agencies pay for these platforms at the department level without informing program staff.

B.4 Books

Appendix A. covers the technical R books worth owning, most of which are free online. Books remain worth purchasing for a team shelf, and they are the easiest item to buy quickly against a closing deadline.

Beyond the R canon, consider books on communication: *Storytelling with Data* and its companion exercise book, *Talking Health*, and Edward Tufte's visualization books. For R specifically, *R for Data Science* [15], the functions and control flow chapters of *Advanced R*, and *Hands-On Programming with R* cover the fundamentals identified above. Where anyone on the team works in Python, *Python for Data Analysis* is the standard reference.

B.5 Consulting

Some of what a team needs is not sold as a course. Where the gap is one specific workflow, a pipeline that needs to be made reproducible, a dashboard nobody has time to finish, or training built around your own data, a short consulting engagement can be a better use of the money than another subscription.

i Training and consulting from the author

I am available for that kind of work, including after a team's DSTT program year ends. I have worked with DSTT teams for years as a project coach, and I consult on data science and bioinformatics across a range of fields. I am a professor in the University of Virginia School of Data Science, and I was previously a professor of public health sciences in the UVA School of Medicine. The fastest way to reach me is [stephenturner.us/email](mailto:stephenturner.us@email).

B.6 Procurement Constraints

Confirm the purchasing rules before committing to a course or event.

Establish what "spent" means to your finance office. Committed and spent are different states. Some offices count a purchase order issued before the deadline; others require the invoice paid and reconciled. Ask in writing and ask early, because the answer changes what can be purchased.

Establish whether a purchasing blackout applies. Many local governments freeze non-essential purchasing near the end of a fiscal year, which frequently coincides with training deadlines. Do not assume a personal purchase will be reimbursed. Obtain written approval before anyone incurs an expense.

Map travel restrictions before planning travel. Out-of-state travel bans are common and sometimes absolute, including for conferences, which can require an entire stipend to be spent in state. Virtual conference registration and subscription training are the usual substitutes when travel is unavailable.

Determine whether registering now for a later event counts. Registering and paying for a conference before a spending deadline for an event occurring months afterward is often possible, and whether a finance office accepts it varies. Ask, and hold a backup plan, since travel approval and scheduling can still prevent attendance after the money is committed.

Plan reconciliation where multiple payers are involved. A program sponsor, a county finance office, a grant, and a personal reimbursement within one purchase produce four sets of documentation, and reconciliation across several payers can consume more staff time than the training itself. Retain every receipt and record which fund each purchase draws on as it occurs.

Begin earlier than appears necessary. Approval lead times exceed expectations consistently. Sole-source justifications, vendor onboarding, and tax documentation for a small training company can require weeks. Six weeks before a deadline is late.

 Warning

A delayed approval can leave a team choosing from an approved catalog in the final week, even when none of the courses meets its needs. Verify the approval path before reviewing the catalog.

B.7 A Worked Plan

Consider \$3,000, six weeks, and a three-person team.

In week one, confirm the rules: what constitutes spending, whether a blackout applies, whether travel is permitted, and what catalog restrictions exist. In parallel, ask each staff member which of the skills above most limits their current work.

In week two, order books if the required approvals are in place. Allocate roughly \$300 in this example.

In weeks two and three, submit the subscription request. Reserve roughly \$500 for subscriptions in this example, subject to a current quote and comparison with free practice materials. Submit early, because vendor onboarding is the slow component.

In weeks three through five, allocate the remaining \$2,200 to the team's highest-priority training need: a private instructor-led R or SQL workshop for the full team, a course seat for each analyst, or virtual conference registration for each. Obtain quotes and start approval for instructor-led training in week one; its lead time may exceed six weeks.

In week six, reconcile. Do not plan to purchase anything in week six.

 Note

This plan excludes individual conference travel deliberately. Travel could consume much of this example budget. Purchase travel when the objective is the professional relationships, and state that as the objective.

B.8 Requesting Next Year's Funding

Tie the request to a specific improvement. For example: "Three SQL course seats will help analysts query the warehouse directly. We currently wait four days for each extract request." Use measured delays from your own team.

Record what was purchased and what changed as a result. Where last year's version control training is the reason a weekly report survived an analyst's departure, document that outcome in this year's request. The continuity argument in Chapter 23, and Section 22.6 is the strongest case available for training funds, because turnover is a risk leadership is already concerned about.

Bibliography

- [1] K. W. Broman and K. H. Woo, “Data Organization in Spreadsheets,” *The American Statistician*, vol. 72, no. 1, pp. 2–10, 2018, doi: [10.1080/00031305.2017.1375989](https://doi.org/10.1080/00031305.2017.1375989).
- [2] H. Wickham, “Tidy Data,” *Journal of Statistical Software*, vol. 59, no. 10, pp. 1–23, 2014, doi: [10.18637/jss.v059.i10](https://doi.org/10.18637/jss.v059.i10).
- [3] J. Bryan, “How to Name Files.” [Online]. Available: <https://github.com/jennybc/how-to-name-files>
- [4] W. M. Landau, “targets: Dynamic Function-Oriented 'Make'-Like Declarative Pipelines.” 2021. [Online]. Available: <https://docs.ropensci.org/targets/>
- [5] H. Wickham, “conflicted: An Alternative Conflict Resolution Strategy.” 2023. [Online]. Available: <https://conflicted.r-lib.org/>
- [6] V. P. Nagraj, K. H. Siller, T. Stewart, N. Magee, and S. D. Turner, “Ten Quick Tips to SNIFF out Sustainable and Secure Scientific Software,” *PLOS Computational Biology*, vol. 22, no. 7, p. e1014510, July 2026, doi: [10.1371/journal.pcbi.1014510](https://doi.org/10.1371/journal.pcbi.1014510).
- [7] H. Wickham and J. Bryan, *R Packages*, 2nd ed. O'Reilly Media, 2023. [Online]. Available: <https://r-pkgs.org/>
- [8] V. P. Nagraj and S. D. Turner, “Pracpac: Practical R Packaging with Docker,” no. arXiv:2303.07876. arXiv, Mar. 2023. doi: [10.48550/arXiv.2303.07876](https://doi.org/10.48550/arXiv.2303.07876).
- [9] M. Raasveldt and H. Mühleisen, “DuckDB: An Embeddable Analytical Database,” in *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*, ACM, 2019, pp. 1981–1984. doi: [10.1145/3299869.3320212](https://doi.org/10.1145/3299869.3320212).
- [10] K. Walker, *Analyzing US Census Data: Methods, Maps, and Models in R*. CRC Press, 2023. [Online]. Available: <https://walker-data.com/census-r/>
- [11] R. D. Peng, B. Caffo, and J. T. Leek, *Executive Data Science*. Leanpub, 2015. [Online]. Available: <https://leanpub.com/eds>
- [12] E. Robinson and J. Nolis, *Build a Career in Data Science*. Manning Publications, 2020. [Online]. Available: <https://www.manning.com/books/build-a-career-in-data-science>
- [13] R. D. Peng and E. Matsui, *The Art of Data Science*. Leanpub, 2015. [Online]. Available: <https://leanpub.com/artofdatascience>
- [14] National Institute of Standards and Technology, “Artificial Intelligence Risk Management Framework (AI RMF 1.0),” technical report NIST AI 100-1, Jan. 2023. doi: [10.6028/NIST.AI.100-1](https://doi.org/10.6028/NIST.AI.100-1).
- [15] H. Wickham, M. Cetinkaya-Rundel, and G. Grolemund, *R for Data Science*, 2nd ed. O'Reilly Media, 2023. [Online]. Available: <https://r4ds.hadley.nz/>